2026-01

数据结构 · 单项选择题 · 2 分

线性表 L 含有 n（$n \geq 3$）个元素，采用顺序存储方式保存。在 L 中原有元素相对次序保持不变且仍有空闲存储空间的情况下，下列操作中，一定会移动多个元素的是

I. 在表头插入一个元素

II. 在表头删除一个元素

III. 在表尾插入一个元素

IV. 在表尾删除一个元素

A. 仅 I、II

B. 仅 I、III

C. 仅 II、IV

D. 仅 III、IV

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】2.2.2_1_顺序表的插入删除 · 原PDF第6页
原课件 2.2.2_1_顺序表的插入删除 · PDF p6

插入操作的时间复杂度

```
bool ListInsert(SqList &L,int i,int e){
    if(i<1||i>L.length+1)          //判断i的范围是否有效
        return  false;
    if(L.length>=MaxSize)          //当前存储空间已满，不能插入
        return  false;
    for(int j=L.length;j>=i;j--)   //将第i个元素及之后的元素后移
        L.data[j]=L.data[j-1];
    L.data[i-1]=e;                 //
    L.length++;                    //长度加1
    return  true;
}
```

关注最深层循环语句的执行次数与问题规模 n 的关系

问题规模 n = L.length （表长）

最好情况：新元素插入到表尾，不需要移动元素

i = n+1，循环0次；最好时间复杂度 = O(1)

最坏情况：新元素插入到表头，需要将原有的 n 个元素全都向后移动

i = 1，循环 n 次；最坏时间复杂度 = O(n);

平均情况：假设新元素插入到任何一个位置的概率相同，即 i = 1,2,3, … , length+1 的概率都是 $p = \frac{1}{n+1}$

i = 1，循环 n 次；i=2 时，循环 n-1 次；i=3，循环 n-2 次 …… i =n+1时，循环0次

平均循环次数 = $np + (n-1)p + (n-2)p + \cdots\cdots + 1\cdot p = \frac{n(n+1)}{2}\frac{1}{n+1} = \frac{n}{2}$ ⇒ 平均时间复杂度 = O(n)

王道考研/CSKAOYAN.COM

对应原题本第2页；点击此行返回题目。

【对应知识】2.2.2_1_顺序表的插入删除 · 原PDF第9页
原课件 2.2.2_1_顺序表的插入删除 · PDF p9

删除操作的时间复杂度

```
bool ListDelete(SqList &L,int i,int &e){
    if(i<1||i>L.length)          //判断i的范围是否有效
        return  false;
     e=L.data[i-1];              //将被删除的元素赋值给e
    for(int j=i;j<L.length;j++)  //将第i个位置后的元素前移
        L.data[j-1]=L.data[j];
    L.length--;                  //
    return  true;
}
```

关注最深层循环语句的执行次数与问题规模 n 的关系

问题规模 n = L.length （表长）

最好情况：删除表尾元素，不需要移动其他元素
i = n，循环 0 次；最好时间复杂度 = O(1)

最坏情况：删除表头元素，需要将后续的 n-1 个元素全都向前移动
i = 1，循环 n-1 次；最坏时间复杂度 = O(n);

平均情况：假设删除任何一个元素的概率相同，即 i = 1,2,3, … , length 的概率都是 $p = \frac{1}{n}$
i = 1，循环 n-1 次；i=2 时，循环 n-2 次；i=3，循环 n-3 次 …… i =n 时，循环0次

平均循环次数 = (n-1)p + (n-2)p + …… + 1·p = $\frac{n(n-1)}{2}\frac{1}{n} = \frac{n-1}{2}$ ⇒ 平均时间复杂度 = O(n)

王道考研/CSKAOYAN.COM

对应原题本第2页；点击此行返回题目。

2026-02

数据结构 · 单项选择题 · 2 分

已知带头结点的双向链表 L 含有 n（$n \geq 3$）个元素，结点结构为

p2	d	p1

其中，p2 和 p1 分别是指向其直接前驱和直接后继结点的指针，指针 head 指向头结点，指针 cu = head。现要将 L 各结点中的 p2 指针修改为指向 p1 所指结点的直接后继结点。当指针指向的结点不存在时，相应的指针值为 NULL。下列选项中，可以完成修改功能的是

A. `while (cu != NULL) { cu->p2 = cu->p1->p1; cu = cu->p1; }`

B. `while (cu != NULL && cu->p2 != NULL) { cu->p2 = cu->p1->p1; cu = cu->p1; }`

C. `while (cu != NULL) { if (cu->p1 != NULL) { cu->p2 = cu->p1->p1; cu = cu->p1; } }`

D. `while (cu != NULL) { if (cu->p1 != NULL) cu->p2 = cu->p1->p1; else cu->p2 = NULL; cu = cu->p1; }`

王道知识与解析 · 未定位原解析，仅附知识

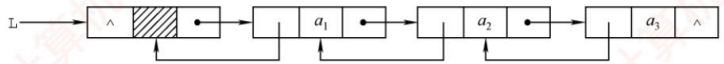
【对应知识】王道2027数据结构 · 原PDF第48页
王道2027数据结构 · 双链表及插入操作 · PDF p48

注 意

单链表是链式存储结构的基础，建议读者熟练掌握其基本操作。在设计算法时，可先通过画图理清指针变化逻辑，再编写代码，有助于避免常见错误（如指针丢失、内存泄漏等）。

2.3.3 双链表

单链表的每个结点仅包含一个指向其后继的指针，故只能从前往后依次遍历。若需访问某结点的前驱（插入或删除操作时），则要从头开始遍历，时间复杂度为$O(n)$。为了克服这一局限，引入了双链表，双链表中的每个结点包含两个指针prior和next，分别指向直接前驱和直接后继，如图2.9所示。其中，头结点的prior为NULL，尾结点的next也为NULL。



【对应知识】王道2027数据结构 · 原PDF第48页（续）
王道2027数据结构 · 双链表及插入操作 · PDF p48

图 2.9 双链表示意图

双链表结点类型的定义如下：

```
typedef struct DNode{                    //定义双链表结点类型
        ElemType data;                   //数据域
        struct DNode *prior,*next;       //前驱和后继指针
}DNode, *DLinklist;
```

双链表的按值查找操作和按位查找操作与单链表的相同，通常也需要从头结点开始顺序查找。由于增加了指向前驱的指针，双链表在插入操作和删除操作中要同时维护前驱与后继两个方向的链接，因此其实现方式与单链表的有较大差异。关键在于：**修改指针时不能造成断链**。得益于对前驱结点的直接访问，在**已知目标结点**的前提下，双链表的**插入和删除操作的时间复杂度可降至 $O(1)$**。

1．双链表的插入操作

考点追踪 ▶▶ 双链表中插入操作的实现（2023）

【对应知识】王道2027数据结构 · 原PDF第48页（续）
王道2027数据结构 · 双链表及插入操作 · PDF p48

在双链表的结点*p 之后插入新结点*s，其指针变化过程如图 2.10 所示。代码片段如下：

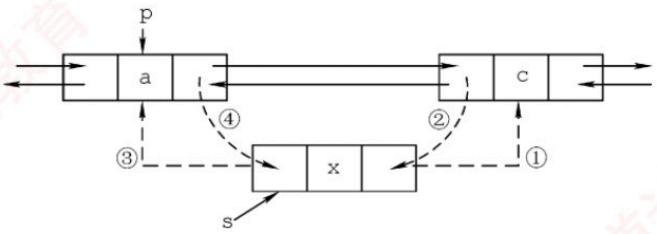


图 2.10　双链表插入结点过程

```
① s->next=p->next;              //将结点*s 插入到结点*p 之后
② p->next->prior=s;
③ s->prior=p;
④ p->next=s;
```

上述语句的执行顺序**并不是任意的**：步骤①必须在步骤④之前，否则 p->next 被覆盖后，无法访问原后继结点，导致指针丢失，插入失败。其余步骤可在保证逻辑正确的前提下适当调整（例如③可在①前执行）。若问题改成要求在结点*p 之前插入结点*s，则要如何处理？请读者自行推导操作步骤。

【对应知识】王道2027数据结构 · 原PDF第49页
王道2027数据结构 · 双链表删除及空指针边界 · PDF p49

2．双链表的删除操作

考点追踪 ▶▶ 双链表中删除操作的实现（2016）

删除双链表中结点*p 的后继结点*q，其指针的变化过程如图 2.11 所示。

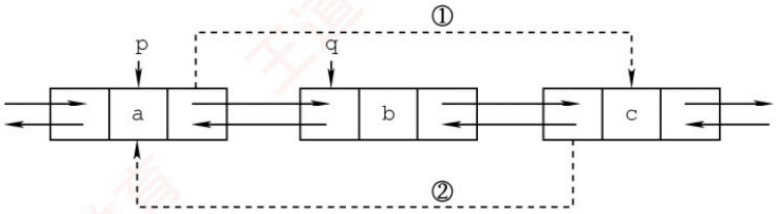


图 2.11　双链表删除结点过程

```
p->next=q->next;            //图 2.11 中步骤①
q->next->prior=p;           //图 2.11 中步骤②
free(q);                    //释放结点空间
```

若问题改成要求删除结点*q 的前驱结点*p，请读者尝试写出对应的操作步骤。

建立双链表时，同样可以采用类似单链表的头插法或尾插法，但要**注意**：每次插入新结点时，要同时正确设置 next 和 prior 两个指针，以维持双向链接的完整性。

2.3.4　循环链表

1．循环单链表

循环单链表与普通单链表的**主要区别在于**：表中最后一个结点的next域不再为NULL，而是指向头结点，从而使整个链表形成一个环。这种结构使得从任意一个结点出发均可遍历整个循环

【对应知识】王道2027数据结构 · 原PDF第49页（续）
王道2027数据结构 · 双链表删除及空指针边界 · PDF p49

单链表，而不仅限于从表头开始，如图2.12所示。

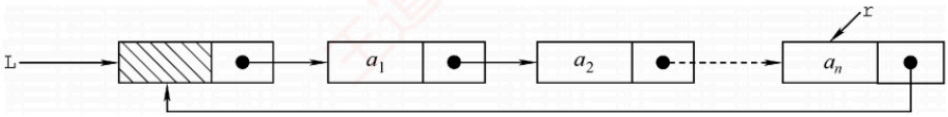


图 2.12　循环单链表

尾结点*r 的 next 指向头结点，因此链表中**不存在 next 为 NULL 的结点。判空条件**不再是检查头指针 L 是否为空，而是检查头结点的 next 是否指向自身（L->next==L）。

考点追踪 ▶▶ **循环单链表中删除首元素的操作（2021）**

循环单链表的插入和删除操作与普通单链表基本相同，但有一个**关键差异**：在表尾进行操作时，需要特别处理以维持链表的循环特性。然而，正是**因为循环单链表是一个环，在任何位置上的插入和删除操作都是等价的**，因此无须判断是否到达表尾。

为了提高效率，循环单链表有时不设头指针，而**仅设置尾指针 r**。此时，在表头或表尾插入元素的时间复杂度均为 $O(1)$。若使用头指针，则表尾插入需要遍历整个链表，时间复杂度为 $O(n)$。例如，在表尾插入新结点*s 时，需要执行以下步骤：令 s->next=r->next（使新结点的后继指向头结点，以维持环状结构）；将 r->next 指向新结点*s；更新 r 为新结点*s。

2．循环双链表

基于循环单链表的概念，不难推导出循环双链表。**不同之处**在于：循环双链表中的每个结点不仅包含指向下一个结点的next域，还包含指向前一个结点的prior域。特别地，头结点的prior

2026-03

数据结构 · 单项选择题 · 2 分

若二叉树 T 的中序遍历序列是 b,e,d,f,c,a,g，层序遍历序列是 a,b,g,c,d,e,f，则 T 的后序遍历序列是

A. c,e,d,f,b,g,a

B. c,e,f,d,b,g,a

C. e,f,d,c,b,g,a

D. e,g,f,d,b,c,a

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】5.3.1_3 由遍历序列构造二叉树 · 原PDF第37页
原课件 5.3.1_3 由遍历序列构造二叉树 · PDF p37

知识回顾与重要考点

由二叉树的遍历序列构造二叉树

- 前序+中序遍历序列
- 后序+中序遍历序列
- 层序+中序遍历序列

前序遍历序列	根结点	左子树的前序遍历序列	右子树的前序遍历序列	
后序遍历序列	左子树的后序遍历序列	右子树的后序遍历序列	根结点	
中序遍历序列	左子树的中序遍历序列	根结点	右子树的中序遍历序列	
层序遍历序列	根结点	左子树的根	右子树的根	.....

Key：找到树的根节点，并根据中序序列划分左右子树，再找到左右子树根节点

王道考研/CSKAOYAN.COM

对应原题本第4页；点击此行返回题目。

【对应知识】王道2027数据结构 · 原PDF第153页
王道2027数据结构 · 二叉树中序、后序遍历及递归实现 · PDF p153

```
void PreOrder(BiTree T){
    if(T!=NULL){
        visit(T);                    //访问根结点
        PreOrder(T->lchild);         //递归遍历左子树
        PreOrder(T->rchild);         //递归遍历右子树
    }
}
```

2．中序遍历（InOrder）

若二叉树为空，则不执行任何操作；否则，

1）中序遍历左子树；

2）访问根结点；

3）中序遍历右子树。

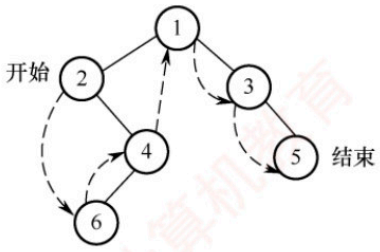


图 5.8　二叉树的中序遍历

考点追踪 ▶▶ **中序序列中结点关系的分析（2017、2024）**

图 5.8 中的虚线表示对该二叉树进行中序遍历的访问路径，所得中序遍历序列为 264135。

对应的递归算法如下：

```
void InOrder(BiTree T){
    if(T!=NULL){
        InOrder(T->lchild);          //递归遍历左子树
        visit(T);                    //访问根结点
        InOrder(T->rchild);          //递归遍历右子树
    }
}
```

3．后序遍历（PostOrder）

若二叉树为空，则不执行任何操作；否则，

1）后序遍历左子树；

2）后序遍历右子树；

3）访问根结点。

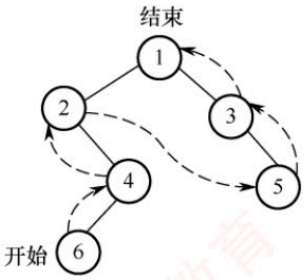


图 5.9　二叉树的后序遍历

图 5.9 中的虚线表示对该二叉树进行后序遍历的访问路径，所得后序遍历序列为 642531。

对应的递归算法如下：

```
void PostOrder(BiTree T){
    if(T!=NULL){
        PostOrder(T->lchild);        //递归遍历左子树
        PostOrder(T->rchild);        //递归遍历右子树
        visit(T);                    //访问根结点
    }
}
```

上述三种遍历算法中，递归遍历左右子树的顺序都是固定的，区别仅在于访问根结点的时机不同。不论采用哪种遍历方法，**每个结点都被访问一次且仅一次**，时间复杂度均为 $O(n)$。在递归实现中，**系统工作栈的深度等于树的高度**，在最坏情况下（如单支树），空间复杂度为 $O(n)$。

4．层次遍历

图 5.10 展示了二叉树的层次遍历过程，即按照 1, 2, 3, 4 的层次顺序，以及箭头所指方向，从上到下、从左到右逐层访问二叉树中的各结点。

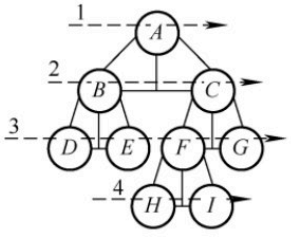


图 5.10　二叉树的层次遍历

对应原题本第4页；点击此行返回题目。

【对应知识】王道2027数据结构 · 原PDF第153页（续）
王道2027数据结构 · 二叉树中序、后序遍历及递归实现 · PDF p153

对应原题本第4页；点击此行返回题目。

数据结构 · 单项选择题 · 2 分

已知由 5 棵树组成的森林 F 中各棵树所含的结点个数分别是 2，3，4，5，7。若 F 中各棵树的排列次序任意，则与 F 对应的二叉树的高度至少是

A. 5

B. 6

C. 8

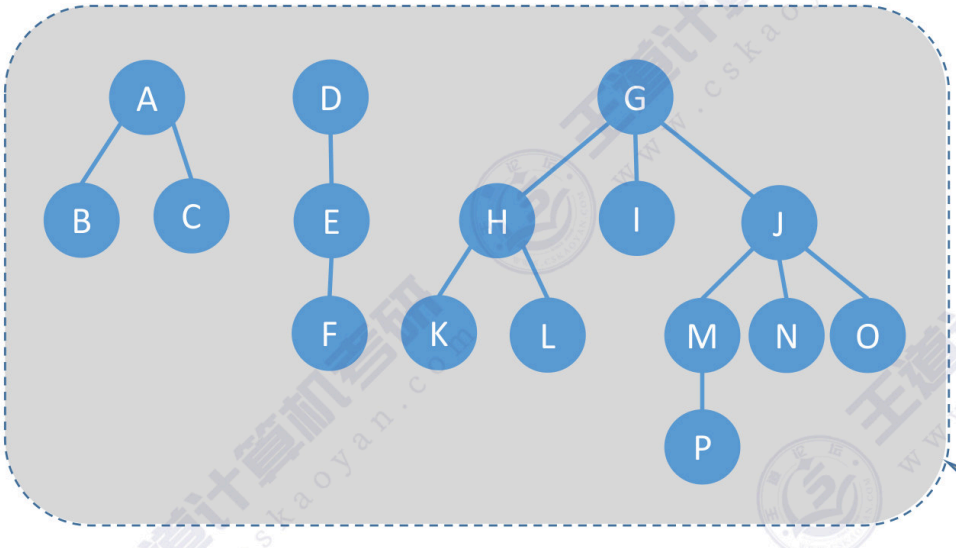
D. 10

王道知识与解析 · 未定位原解析，仅附知识

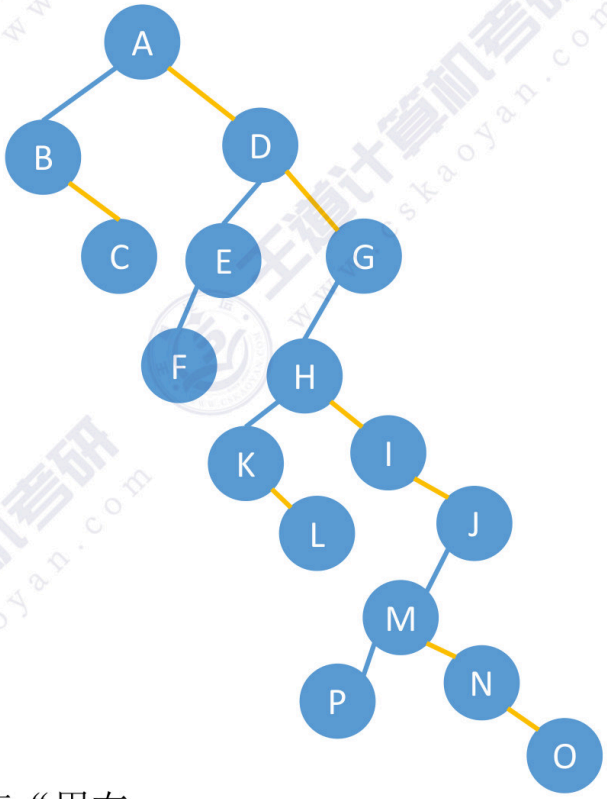
【对应知识】5.4.2 树、森林与二叉树的转换 · 原PDF第5页
原课件 5.4.2 树、森林与二叉树的转换 · PDF p5

森林→二叉树 的转换

注意：森林中各棵树的根节点视为平级的兄弟关系



孩子兄弟表示法



森林→二叉树 转换技巧：

①先把所有树的根结点画出来，在二叉树中用右指针串成糖葫芦。

②按“森林的层序”依次处理每个结点。

处理一个结点的方法是：如果当前处理的结点在树中有孩子，就把所有孩子结点“用右指针串成糖葫芦”，并在二叉树中把第一个孩子挂在当前结点的左指针下方

王道考研/CSKAOYAN.COM

对应原题本第5页；点击此行返回题目。

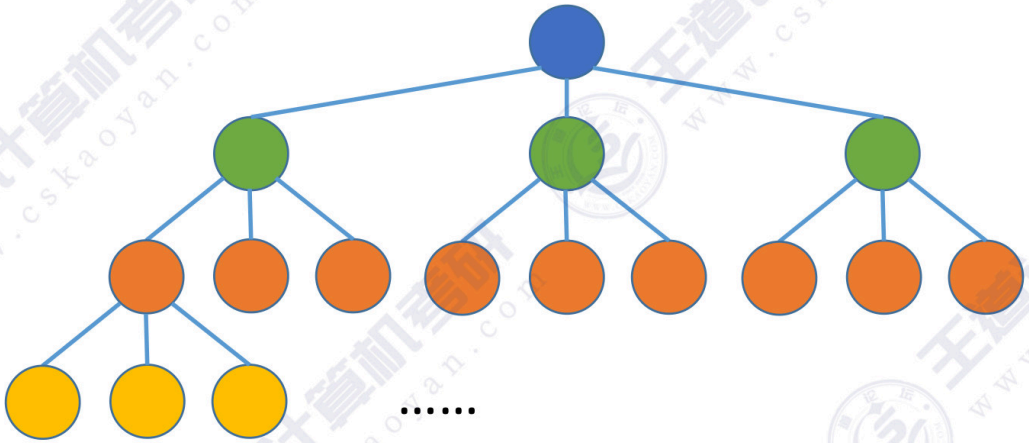
【对应知识】5.2.1_2 二叉树的性质 · 原PDF第4页
原课件 5.2.1_2 二叉树的性质 · PDF p4

二叉树的常考性质

常见考点3：高度为h的二叉树至多有 2^h-1 个结点（满二叉树）

高度为h的m叉树至多有 $\frac{m^h-1}{m-1}$ 个结点

等比数列求和公式：$a+aq+aq^2+\cdots+aq^{n-1}=\frac{a(1-qn)}{1-q}$



第 1 层：m^0

第 2 层：m^1

第 3 层：m^2

第 4 层：m^3

王道考研/CSKAOYAN.COM

2026-05

数据结构 · 单项选择题 · 2 分

已知二叉树 T 中 7 个叶结点 a,b,c,d,e,f,g 的权值分别是 1，2，4，5，8，10，12，当 T 的带权路径长度最小时，与结点 e 深度相同的叶结点仅有

A. d

B. g

C. d、f

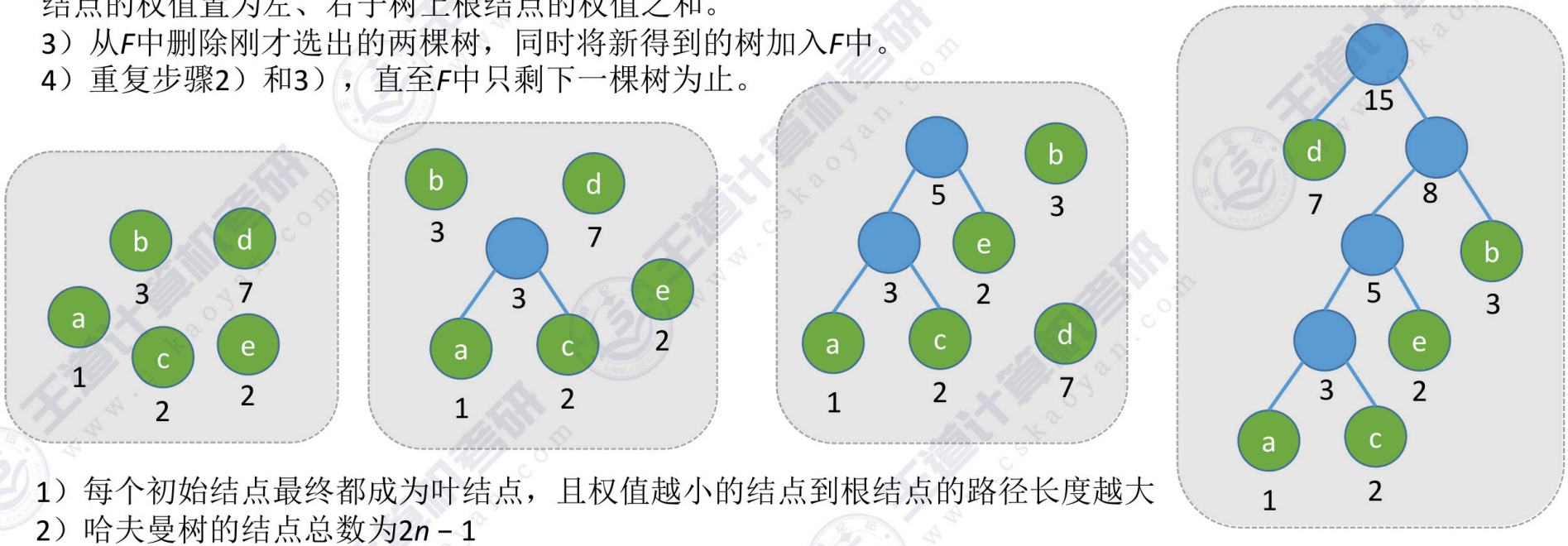
D. f、g

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】5.5.1 哈夫曼树 · 原PDF第5页
原课件 5.5.1 哈夫曼树 · PDF p5

哈夫曼树的构造

给定n个权值分别为$w_1, w_2,\ldots, w_n$的结点，构造哈夫曼树的算法描述如下：
1）将这n个结点分别作为n棵仅含一个结点的二叉树，构成森林F。
2）构造一个新结点，从F中选取两棵根结点权值最小的树作为新结点的左、右子树，并且将新结点的权值置为左、右子树上根结点的权值之和。
3）从F中删除刚才选出的两棵树，同时将新得到的树加入F中。
4）重复步骤2）和3），直至F中只剩下一棵树为止。



1）每个初始结点最终都成为叶结点，且权值越小的结点到根结点的路径长度越大
2）哈夫曼树的结点总数为$2n-1$
3）哈夫曼树中不存在度为1的结点。
4）哈夫曼树并不唯一，但WPL必然相同且为最优

$WPL_{min}=1*7+2*3+3*2+4*1+4*2=31$

王道考研/CSKAOYAN.COM

对应原题本第6页；点击此行返回题目。

2026-06

数据结构 · 单项选择题 · 2 分

若用邻接表保存有向图 $G=(V,E)$，则计算 G 中某顶点入度的时间复杂度是

A. $O(|V|)$

B. $O(min(|V|,|E|))$

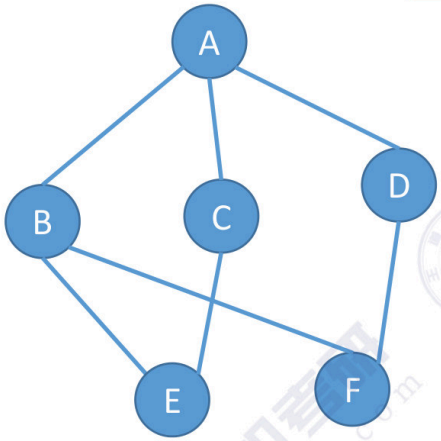
C. $O(|E|)$

D. $O(max(|V|,|E|))$

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】6.2.2 邻接表法 · 原PDF第7页
原课件 6.2.2 邻接表法 · PDF p7

知识回顾与重要考点



邻接表

	data	*first
0	A	1 → 2 → 3 ^
1	B	0 → 4 → 5 ^
2	C	0 → 4 ^
3	D	0 → 5 ^
4	E	1 → 2 ^
5	F	1 → 3 ^

邻接矩阵

	A	B	C	D	E	F
A	0	1	1	1	0	0
B	1	0	0	0	1	1
C	1	0	0	0	1	0
D	1	0	0	0	0	1
E	0	1	1	0	0	0
F	0	1	0	1	0	0

	邻接表	邻接矩阵
空间复杂度	无向图 $O(\|V\|+2\|E\|)$；有向图$O(\|V\|+\|E\|)$	$O(\|V\|^2)$
适合用于	存储稀疏图	存储稠密图
表示方式	不唯一	唯一
计算度/出度/入度	计算有向图的度、入度不方便，其余很方便	必须遍历对应行或列
找相邻的边	找有向图的入边不方便，其余很方便	必须遍历对应行或列

王道考研/CSKAOYAN.COM

对应原题本第7页；点击此行返回题目。

2026-07

数据结构 · 单项选择题 · 2 分

有向图 $G=(V,E)$ 的顶点个数为 n（$n\geq 2$），指定一个顶点为初始点，指定若干个顶点（非初始点）为标记点，且每个顶点都在从初始点到标记点的有向路径上（一条有向路径上允许含有多个标记点）。每条有向边上均有一个字符，从初始点到任一标记点的有向路径上的所有字符组成一个字符串，所有这样的字符串构成字符串集 S。下列关于 S 的叙述中，错误的是

A. 若 G 无环，则 S 是有穷集

B. 若 G 无环，则 S 中存在长度等于 n 的字符串

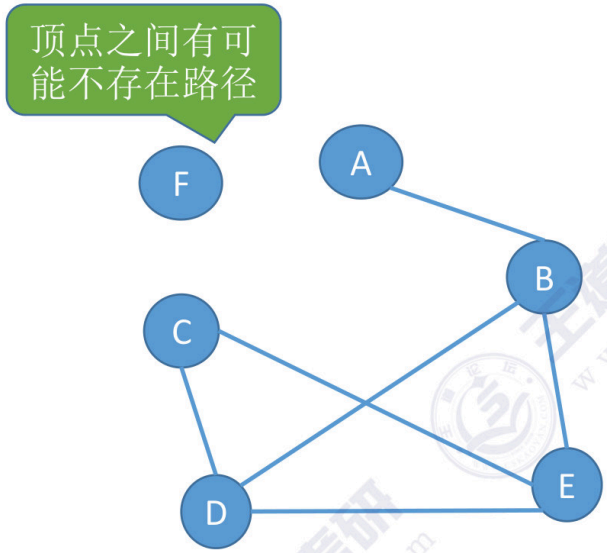
C. 若 G 有环，则 S 中存在长度大于 n 的字符串

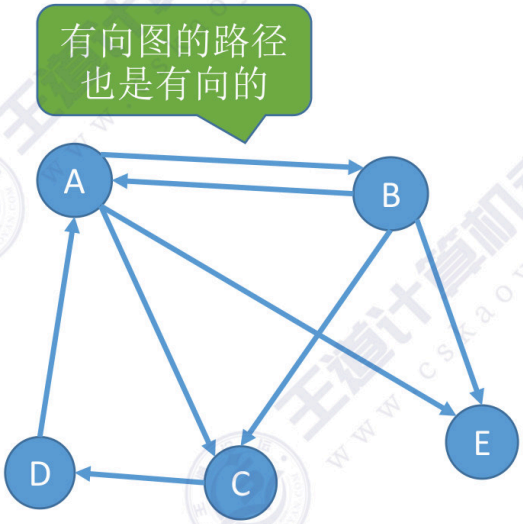
D. 若 G 有环，则 S 中存在长度小于 $2n$ 的字符串

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】6.1.1 图的基本概念 · 原PDF第11页
原课件 6.1.1 图的基本概念 · PDF p11

顶点-顶点的关系描述





- 路径——顶点v_p到顶点v_q之间的一条路径是指顶点序列，$v_p, v_{i_1}, v_{i_2}, \cdots, v_{i_m}, v_q$
- 回路——第一个顶点和最后一个顶点相同的路径称为回路或环
- 简单路径——在路径序列中，顶点不重复出现的路径称为简单路径。
- 简单回路——除第一个顶点和最后一个顶点外，其余顶点不重复出现的回路称为简单回路。
- 路径长度——路径上边的数目
- 点到点的距离——从顶点u出发到顶点v的最短路径若存在，则此路径的长度称为从u到v的距离。若从u到v根本不存在路径，则记该距离为无穷（∞）。

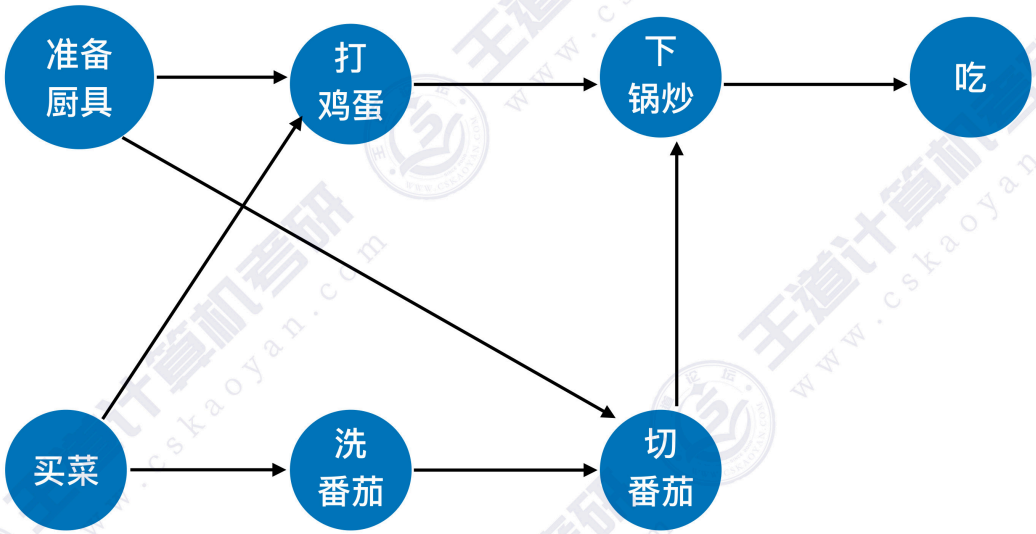
- 无向图中，若从顶点v到顶点w有路径存在，则称v和w是连通的
- 有向图中，若从顶点v到顶点w和从顶点w到顶点v之间都有路径，则称这两个顶点是强连通的

王道考研/CSKAOYAN.COM

对应原题本第8页；点击此行返回题目。

【对应知识】6.4.4 拓扑排序 · 原PDF第4页
原课件 6.4.4 拓扑排序 · PDF p4

拓扑排序



拓扑排序：在图论中，由一个**有向无环图**的顶点组成的序列，当且仅当满足下列条件时，称为该图的一个拓扑排序：
① 每个顶点出现且只出现一次。
② 若顶点A在序列中排在顶点B的前面，则在图中不存在从顶点B到顶点A的路径。

或定义为：拓扑排序是对有向无环图的顶点的一种排序，它使得若存在一条从顶点A到顶点B的路径，则在排序中顶点B出现在顶点A的后面。每个AOV网都有一个或多个拓扑排序序列。

王道考研/CSKAOYAN.COM

对应原题本第8页；点击此行返回题目。

2026-08

数据结构 · 单项选择题 · 2 分

高度为 4 的平衡二叉树中，根结点左、右子树的结点个数相差最多是

A. 1

B. 2

C. 3

D. 5

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】7.3.2_1 平衡二叉树 · 原PDF第28页
原课件 7.3.2_1 平衡二叉树 · PDF p28

查找效率分析

若树高为h，则最坏情况下，查找一个关键字最多需要对比 h 次，即查找操作的时间复杂度不可能超过 O(h)

平衡二叉树——树上任一结点的左子树和右子树的高度之差不超过1。

假设以n_h表示深度为h的平衡树中含有的最少结点数。

则有$n_0 = 0, n_1 = 1, n_2 = 2$，并且有$n_h = n_{h-1} + n_{h-2} + 1$

可以证明含有n个结点的平衡二叉树的最大深度为$O(\log_2 n)$，平衡二叉树的平均查找长度为$O(\log_2 n)$

王道考研/CSKAOYAN.COM

对应原题本第9页；点击此行返回题目。

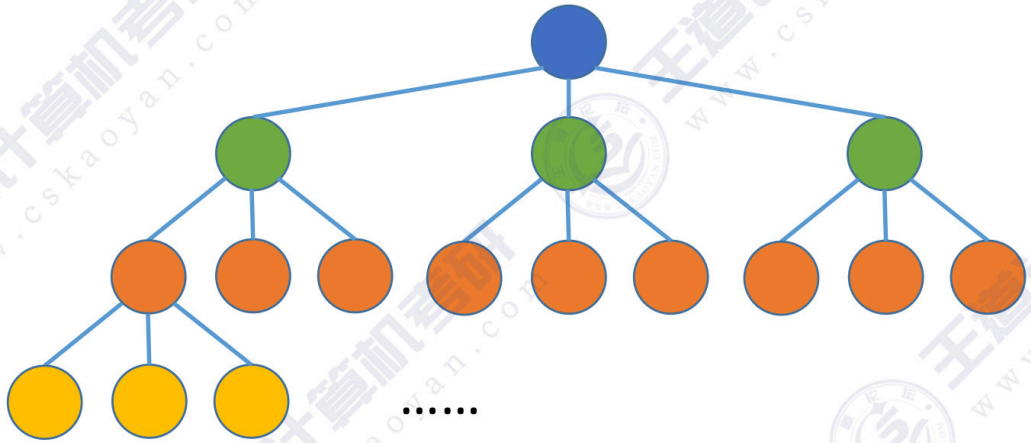
【对应知识】5.2.1_2 二叉树的性质 · 原PDF第4页
原课件 5.2.1_2 二叉树的性质 · PDF p4

二叉树的常考性质

常见考点3：高度为h的二叉树至多有 2^h-1 个结点（满二叉树）

高度为h的m叉树至多有 $\frac{m^h-1}{m-1}$ 个结点

等比数列求和公式：$a+aq+aq^2+\cdots+aq^{n-1}=\frac{a(1-qn)}{1-q}$



第 1 层：m^0

第 2 层：m^1

第 3 层：m^2

第 4 层：m^3

王道考研/CSKAOYAN.COM

对应原题本第9页；点击此行返回题目。

2026-09

数据结构 · 单项选择题 · 2 分

使用直接插入排序算法分别对下列数据进行升序排序，元素之间比较次数最少的是

A. 30，27，56，41，80，95，69

B. 31，43，26，55，63，99，77

C. 61，84，51，23，34，91，40

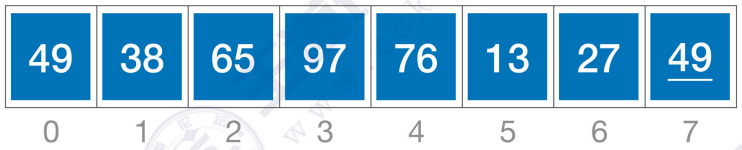
D. 93，32，48，81，50，21，72

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】8.2.1+8.2.2 插入排序 · 原PDF第2页
原课件 8.2.1+8.2.2 插入排序 · PDF p2

插入排序

算法思想：每次将一个待排序的记录按其关键字大小插入到前面已排好序的子序列中，直到全部记录插入完成。



王道考研/CSKAOYAN.COM

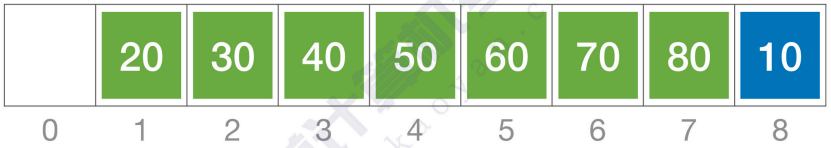
对应原题本第10页；点击此行返回题目。

【对应知识】8.2.1+8.2.2 插入排序 · 原PDF第41页
原课件 8.2.1+8.2.2 插入排序 · PDF p41

算法效率分析

```
//直接插入排序（带哨兵）
void InsertSort(int A[],int n){
    int i,j;
    for(i=2;i<=n;i++)              //依次将A[2]~A[n]插入到前面已排序序列
        if(A[i]<A[i-1]){           //若A[i]关键码小于其前驱，将A[i]插入有序表
            A[0]=A[i];             //复制为哨兵，A[0]不存放元素
            for(j=i-1;A[0]<A[j];--j)//从后往前查找待插入位置
                A[j+1]=A[j];       //向后挪位
            A[j+1]=A[0];           //复制到插入位置
        }
}
```

最坏情况：原本为逆序



最坏情况：

第1趟：对比关键字2次，移动元素3次

第2趟：对比关键字3次，移动元素4次

第 i 趟：对比关键字 i+1次，移动元素 i+2 次

…

第n−1趟：对比关键字 n 次，移动元素 n+1 次

最坏时间复杂度——$O(n^2)$

王道考研/CSKAOYAN.COM

对应原题本第10页；点击此行返回题目。

2026-10

数据结构 · 单项选择题 · 2 分

某考试考查了课程 C1 和 C2，C1、C2 成绩及总成绩（C1、C2 成绩之和）均保存在数组 M 中。现需根据考生的总成绩从高到低进行排名，总成绩相同者的排名由 C1 成绩从高到低决定，C1 成绩也相同者的排名由初始相对次序决定。假设先按 C1 成绩对 M 排序，再按总成绩对 M 排序，能得到正确的排名结果，则下列算法中，采用的是

A. 基数排序

B. 快速排序

C. 希尔排序

D. 选择排序

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】王道2027数据结构 · 原PDF第385页
王道2027数据结构 · 排序的时间、空间、稳定性和存储方式 · PDF p385

8.6 各种内部排序算法的比较及应用

8.6.1 内部排序算法的比较

前面讨论的排序算法很多，各种排序算法的对比是考研常考的内容。通常从五个方面进行对比：时间复杂度、空间复杂度、稳定性、适用性和过程特征。

考点追踪 ▸▸ 各种排序算法的特点和对比（2009、2010、2017、2020、2022、2025）

从时间复杂度看：简单选择排序、直接插入排序和冒泡排序在平均情况下的时间复杂度均为 $O(n^2)$，且实现较为简单。其中，直接插入排序和冒泡排序在最好情况下可达 $O(n)$，而简单选择排序的时间复杂度与序列初始状态无关。希尔排序作为插入排序的改进，对较大规模的数据通常具有较高效率，但目前未知其精确的渐近时间。堆排序利用堆这种数据结构，可在 $O(n)$时间内完成建堆，整个排序过程的时间复杂度为 $O(n\log_2 n)$。快速排序基于分治思想，最坏情况下的时间复杂度为 $O(n^2)$，但平均性能可达 $O(n\log_2 n)$，在实际应用中快速排序通常表现优异。归并排序同样基于分治思想，其分割过程与初始序列无关，因此最好、最坏和平均时间复杂度均为 $O(n\log_2 n)$。

从空间复杂度看：简单选择排序、插入排序、冒泡排序、希尔排序和堆排序仅需常数级辅助空间，空间复杂度为 $O(1)$。快速排序需要递归工作栈，平均空间复杂度为 $O(\log_2 n)$，最坏情况下可能达到 $O(n)$。2 路归并排序在合并操作中需要额外的辅助数组用于元素复制，空间复杂度为 $O(n)$。虽有方法可减少辅助空间，但通常会导致算法复杂度显著增加，甚至影响时间效率。

【对应知识】王道2027数据结构 · 原PDF第385页（续）
王道2027数据结构 · 排序的时间、空间、稳定性和存储方式 · PDF p385

考点追踪 ▸▸ 各种排序算法的稳定性及分析（2021、2023）

从稳定性看：插入排序、冒泡排序、归并排序和基数排序是稳定的排序算法，而简单选择排序、快速排序、希尔排序和堆排序都是不稳定的。在平均时间复杂度为 $O(n\log_2 n)$的排序算法中，只有归并排序是稳定的。判断排序算法是否稳定，关键在于分析其操作是否会改变相等元素的相对次序；对于不稳定的算法，通常可通过构造一个反例加以验证。

考点追踪 ▸▸ 各种排序算法的适用性（2017）

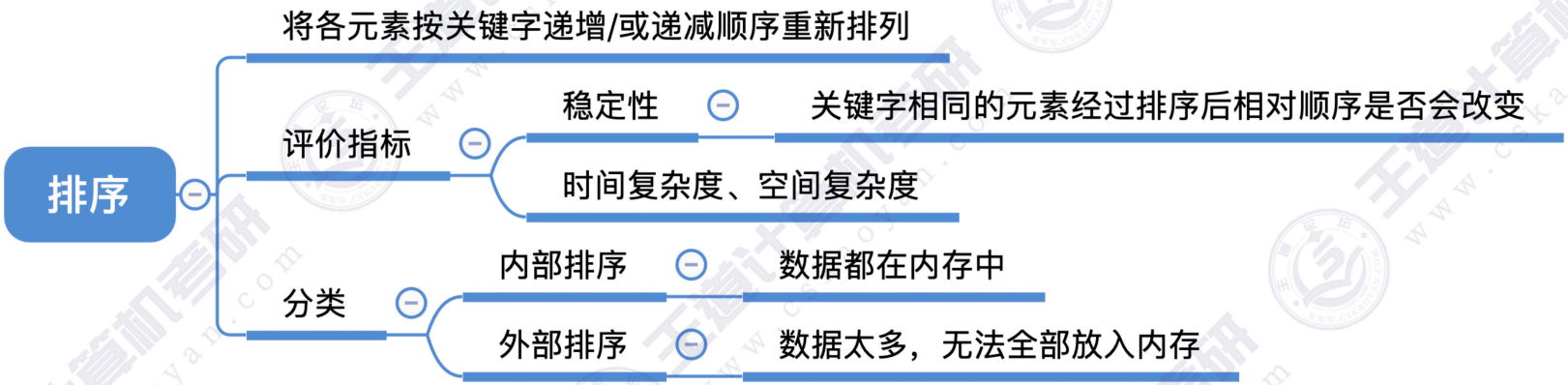
从适用性看：折半插入排序、希尔排序、快速排序和堆排序适用于顺序存储。直接插入排序、冒泡排序、简单选择排序、归并排序和基数排序既适用于顺序存储，也适用于链式存储。

考点追踪 ▸▸ 各种排序算法的过程特征（2012）

从过程特征看：不同排序算法在一趟或几趟处理后的中间结果通常是不同的。考研题目常给出初始序列和部分排序后的结果，要求判断所用的排序方法。这就要求考生熟练掌握各类算法的过程特征。如冒泡排序、简单选择排序和堆排序在每趟处理后都能确定当前最大（或最小）元素的最终位置，而快速排序在一趟划分后，至少能确定一个元素（枢轴）的最终位置等。

【对应知识】8.1 排序的基本概念 · 原PDF第7页
原课件 8.1 排序的基本概念 · PDF p7

知识回顾与重要考点



一个神奇的学习网站：https://www.cs.usfca.edu/~galles/visualization/Algorithms.html

旧金山大学 (University of San Francisco)

王道考研/CSKAOYAN.COM

对应原题本第11页；点击此行返回题目。

【对应知识】王道2027数据结构 · 原PDF第386页
王道2027数据结构 · 内部排序算法比较表与选用条件 · PDF p386

表 8.1 列出了各种排序算法的时空复杂度和稳定性情况。其中，空间复杂度仅列出平均情况；由于希尔排序的时间复杂度依赖于所选的增量序列，因此无法给出精确表达式。

表 8.1　各种排序算法的性质

算法种类	时间复杂度			空间复杂度	是否稳定
	最好情况	平均情况	最坏情况		
直接插入排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	是
冒泡排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	是
简单选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	否
希尔排序				$O(1)$	否
快速排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n^2)$	$O(\log_2 n)$	否
堆排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(1)$	否
2 路归并排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n)$	是
基数排序	$O(d(n+r))$	$O(d(n+r))$	$O(d(n+r))$	$O(r)$	是

8.6.2　内部排序算法的应用

通常情况，对排序算法的比较和应用应考虑以下情况。

考点追踪 ▶▶ 选取排序算法时需要考虑的因素（2019）

1．选取排序算法需要考虑的因素

1）待排序元素的个数 n。
2）待排序序列的初始状态（如是否基本有序）。
3）关键字的结构及其分布情况。
4）对稳定性的要求。
5）存储结构类型及辅助空间的限制等。

2．排序算法选型小结

1）若 n 较小，可采用直接插入排序或简单选择排序。直接插入排序的记录移动次数通常多于简单选择排序，因此当记录本身信息量较大时，简单选择排序更为合适。
2）若 n 较大，应采用时间复杂度为 $O(n\log_2 n)$的排序算法，如快速排序、堆排序或归并排序。当关键字随机分布时，快速排序通常被认为是最高效的基于比较的内部排序算法。堆排序所需辅助空间少于快速排序，且不会退化到最坏情况，但两者均为不稳定排序。若既要求稳定性，又希望达到 $O(n\log_2 n)$的时间复杂度，则应选用归并排序。
3）若初始序列已基本有序，则直接插入或冒泡排序更为高效，因其最好时间复杂度为 $O(n)$。
4）在基于比较的排序算法中，每次比较两个关键字仅有两种可能的结果，因此其执行过程可用一棵判定树来描述。由此可证明：对于 n 个关键字的随机排列，任何基于比较的排序算法，在最坏情况下至少需要 $O(n\log_2 n)$的时间。
5）若 n 很大，且关键字位数较少、可分解为多个子关键字（如整数、字符串），则基数排序是更优选择，因其时间复杂度可接近线性。
6）当记录本身信息量较大（如包含大量字段或大对象）时，为避免因频繁移动记录而产生高昂开销，宜采用链表作为存储结构，此时插入类或归并类排序更具优势。

对应原题本第11页；点击此行返回题目。

2026–11

数据结构 · 单项选择题 · 2 分

采用 k-路归并对外存中保存的数据进行排序，排序的趟数记为 d。下列叙述中，正确的是

I. 增大 k 的值可能减小 d 的值

II. 初始归并段的个数不影响 d 的值

III. 可用内存大小限制了初始归并段的最大长度

A. 仅 I

B. 仅 I、II

C. 仅 I、III

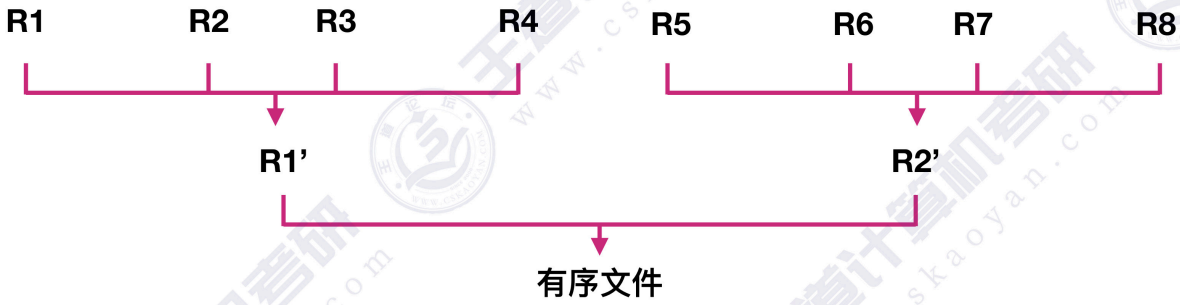
D. 仅 II、III

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】8.7.1+8.7.2 外部排序 · 原PDF第89页
原课件 8.7.1+8.7.2 外部排序 · PDF p89

优化：多路归并

外部排序时间开销=读写外存的时间+内部排序所需时间+内部归并所需时间



采用4路归并，只需进行两趟归并即可
读、写磁盘次数= 32+32*2 = 96 次

重要结论：采用多路归并可以减少归并趟数，从而减少磁盘I/O(读写)次数

对 r 个初始归并段，做k路归并，则归并树可用 k 叉树表示
若树高为h，则归并趟数 = h-1 = $\lceil log_k r \rceil$

k越大，r越小，归并趟数越少，读写磁盘次数越少

推导：k叉树第h层最多有k^{h-1} 个结点
则 $r \le k^{h-1}$，$(h-1)_{最小} = \lceil log_k r \rceil$

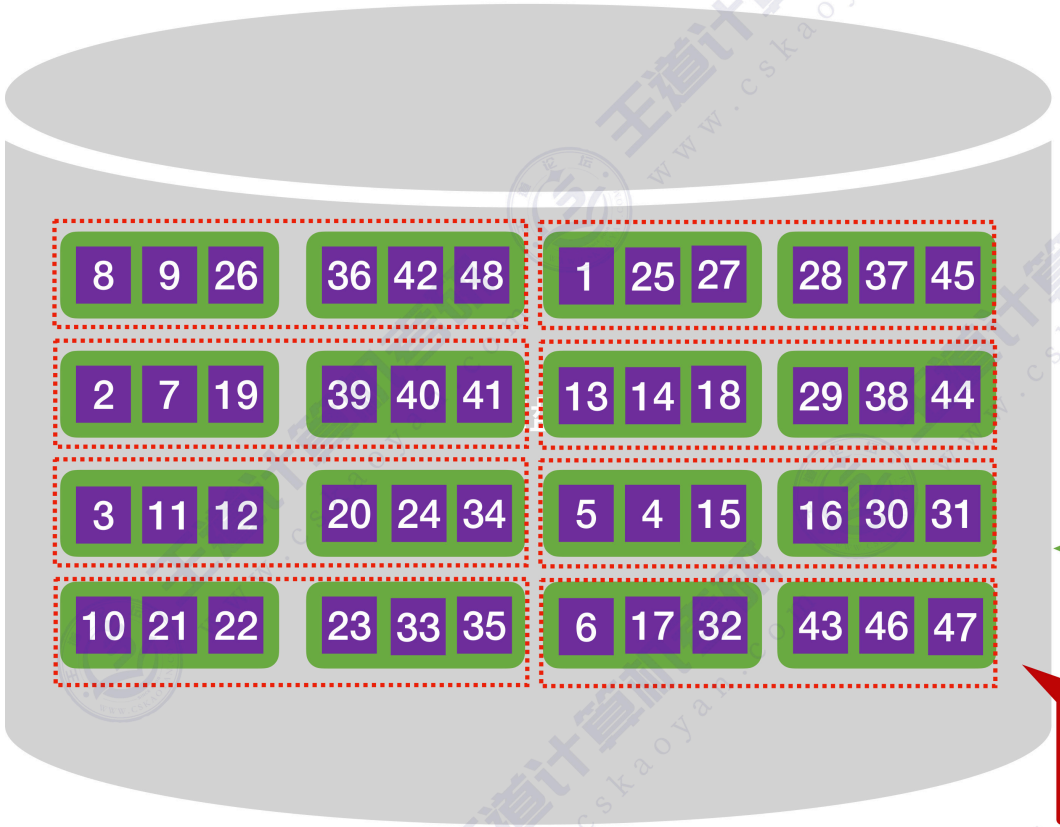
多路归并带来的负面影响：
①k路归并时，需要开辟k个输入缓冲区，内存开销增加。
②每挑选一个关键字需要对比关键字（k-1）次，内部归并所需时间增加

王道考研/CSKAOYAN.COM

对应原题本第12页；点击此行返回题目。

【对应知识】8.7.1+8.7.2 外部排序 · 原PDF第19页
原课件 8.7.1+8.7.2 外部排序 · PDF p19

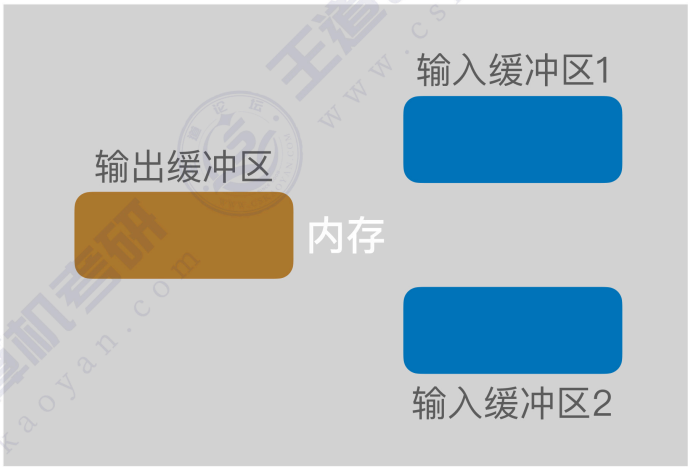
构造初始“归并段”



读磁盘

写磁盘

“归并排序”要求各个子序列有序，每次读入两个块的内容，进行内部排序后写回磁盘



构造初始归并段：
需要16次“读”和16次“写”

王道考研/CSKAOYAN.COM

对应原题本第12页；点击此行返回题目。

2026-12

组成原理 · 单项选择题 · 2 分

下列关于计算机系统层次结构的叙述中，错误的是

A. 最上层是应用软件层，最下层是器件及物理封装层

B. 指令集体系结构是软件和硬件之间的接口，且必不可少

C. 计算机组成（即微架构）属于指令集体系结构的物理实现层

D. 操作系统可通过指令集体系结构进行相应抽象，向上层软件提供服务

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】王道2027计算机组成原理 · 原PDF第17页
系统层次结构图

图 1.2　计算机系统的层次结构示意图

【对应知识】王道2027计算机组成原理 · 原PDF第18页
微体系结构的定义

4．微体系结构

微体系结构（又称微架构）是处理器内部的硬件组织方式，用于实现 ISA 定义的功能。如果说 ISA 定义了“做什么”，那么微架构则决定了“怎么做”。**其核心设计包括**数据通路组织、控制单元实现、流水线级数、缓存层次结构以及分支预测机制等。

例如，加法操作可能通过串行进位加法器、超前进位加法器，甚至专用的 SIMD 单元来实现，这些都属于微体系结构的范畴。**相同的 ISA 可对应多种不同的微构架**。以 Intel x86 为例，不同代际的处理器（如 Core、Skylake、Alder Lake）均遵循同一套 ISA 规范，但内部组织方式差异显著，体现了微架构的多样性与演进性。

【对应知识】王道2027计算机组成原理 · 原PDF第17页
操作系统与ISA

2．操作系统

所有的语言处理系统都必须在操作系统提供的运行环境中执行；操作系统通过对计算机硬件及其底层结构的抽象，构建出一台可供程序员使用的虚拟机。

3．指令集体系结构

指令集体系结构（Instruction Set Architecture，ISA）是计算机软/硬件之间的**关键接口**，它从程序员和编译器的视角，**完整地定义了软件可直接使用的硬件功能**。主要包括：指令格式、操作类型、寻址方式，以及可访问的寄存器等硬件资源。

因此，ISA 构成了软件所能“感知”到的计算机功能视图，也被称为软件可见部分。我们编写的机器语言程序，本质上就是一串严格遵循该 ISA 规范的指令序列；而硬件执行程序的过程，就是逐条解释并完成这些指令所规定操作的过程。

2026–13

组成原理　单项选择题 · 2 分

对机器数 1010 0110B 先算术右移 3 位，再算术左移 2 位，最终结果是

A. 1101 0000B

B. 1101 0011B

C. 0101 0000B

D. 0101 0011B

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】王道2027计算机组成原理 · 原PDF第47页
补码算术左右移规则

2．算术移位

考点追踪 ▶▶ 算术移位运算（2012、2017、2018）

算术移位需要考虑符号位的问题，即将操作数视为有符号整数。有符号整数采用补码表示，因此对于有符号整数的移位操作应采用补码算术移位方式。算术移位的规则：**左移时**，高位移出，低位补 0。若移出的高位与原符号位不同（左移后符号位改变），则发生溢出。**右移时**，低位移出，高位补符号位。若低位的 1 移出，则影响精度。

例如，4 位补码 0010（+2）左移一位变为 0100（+4），未溢出；1001（−7）左移一位变为 0010，符号由负变正，表明发生溢出（因为−14 超出了 4 位补码的表示范围）。又如，1001（−7）右移一位变为 1100（−4），保留了符号位，但丢失了最低有效位，影响精度。

2026–14

组成原理 · 单项选择题 · 2 分

已知 float 型变量用 IEEE 754 单精度浮点格式表示，采用就近舍入（中间值取偶数）方式。若 float 型变量 x 值为 12.1（提示：0.1 = 0.000 1100 1100 1100 1100 1100…B），则 x 的机器数是

A. 4141 9999H

B. 4141 999AH

C. 41E0 CCCCH

D. 41E0 CCCDH

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】王道2027计算机组成原理 · 原PDF第66页
IEEE754单精度格式、隐藏位与偏置值

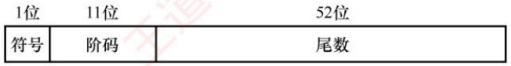
2.3.1 IEEE 754 标准的浮点数

1. IEEE 754 标准的浮点数格式

考点追踪 ▶▶ IEEE 754 单精度数大小的比较（2014）

现代系统**普遍采用 IEEE 754 浮点数标准**。该标准定义了两种常用格式：**32 位单精度浮点数**（float 型）和 **64 位双精度浮点数**（double 型），其基数隐含为 2，其格式如图 2.14 所示。

(a) 32位单精度格式



(b) 64位双精度格式

图 2.14 IEEE 754 标准浮点数的格式

32 位单精度格式包含 1 位符号 s、8 位阶码 e 和 23 位尾数 f；64 位双精度格式包含 1 位符号 s、11 位阶码 e 和 52 位尾数 f。基数隐含为 2；尾数用原码表示。对于规格化的二进制浮点数，尾数的最高位恒为 1。为提升精度，IEEE 754 不显式存储该位，而是将其**隐含在小数点之前**，称为隐藏位。因此，单精度格式的 23 位尾数实际提供了 24 位有效数字，双精度格式的 52 位尾数实际提供了 53 位有效数字。例如，$(12)_{10}=(1100)_2$，规格化后为 1.1×2^3。其中，小数点前的“1”不

【对应知识】王道2027计算机组成原理 · 原PDF第66页（续）
IEEE754单精度格式、隐藏位与偏置值

实际存储，尾数 f 仅保存小数部分“100…0”，而阶码保存的是指数 3 的编码值。

IEEE 754 标准的阶码采用移码表示，但偏置值并不是通常 n 位移码所用的 2^{n-1}，而是 $2^{n-1}-1$。因此，单精度和双精度格式的**偏置值分别为 127 和 1023**。上例中，指数真值为 3，因此在单精度格式中，阶码为 127 + 3 = 130（82H）；在双精度格式中，阶码为 1023 + 3 = 1026（402H）。

IEEE 754 标准的规格化单精度浮点数的真值为

$$(-1)^s\times1.f\times2^{e-127}$$

规格化双精度浮点数的真值为

$$(-1)^s\times1.f\times2^{e-1023}$$

其中，规格化单精度浮点数的阶码 e 的取值范围为 1～254（8 位，全 0 和全 1 保留用于特殊值）；规格化双精度浮点数的阶码 e 的取值范围为 1～2046（11 位，保留用途相同）。

【对应知识】王道2027计算机组成原理 · 原PDF第69页
就近舍入与中间值取偶数

4．舍入

在对阶和右规过程中，尾数右移可能导致低位丢失。为保证精度，移出的低位通常被保留用于中间计算。最终结果需通过舍入处理，还原为标准的 IEEE 754 格式。

为此，IEEE 754 引入三个辅助位以指导精确舍入。

1）保护位：紧邻尾数最低有效位之后的第一位，用于初步判断舍入方向。

2）舍入位：位于保护位之后，与保护位和粘滞位共同构成完整的舍入信息。

3）粘滞位：只要舍入位之后被移出的位中存在至少一个 1，粘滞位就置为 1，否则为 0。

IEEE 754 定义了四种可选的舍入模式。

1）就近舍入（默认方式）：选择最接近真实值的可表示数。若真实值恰好位于两个可表示数的正中间，则选择尾数最低有效位为 0 的那个（偶数）。具体规则：① 若保护位 = 0，直接舍去；② 若保护位 = 1 且（舍入位 = 1 或粘滞位 = 1），则尾数加 1；③ 若保护位 = 1、舍入位 = 0、粘滞位 = 0，则在尾数末位为奇数时向其加 1，以符合**向偶数舍入**的要求。

例如，运算后得到浮点数的临时尾数 M_1，舍入过程如下：M_1

$$M_1 = 1.\underline{1011\ 0011\ 1100\ 1100\ 1101\ 010}\quad \mathbf{1\ 0\ 1}$$

注意，下划线部分为保留的 23 位尾数，其后依次为保护位、舍入位、粘滞位。

由于保护位 = 1、舍入位 = 0、粘滞位 = 1，结果属于非中间值，需要向尾数加 1。加 1 后的 23 位尾数为 1011 0011 1100 1100 1101 011。

若运算后得到临时尾数 M_2，则舍入过程如下：

$$M_2 = 1.\underline{1011\ 0011\ 1100\ 1100\ 1101\ 010}\quad \mathbf{1\ 0\ 0}$$

由于保护位 = 1、舍入位 = 0、粘滞位 = 0，结果恰好位于两个可表示数的正中间。此时尾数最低有效位为偶数，无须加 1。最终的 23 位尾数保持为 1011 0011 1100 1100 1101 010。

对应原题本第15页；点击此行返回题目。

2026–15

组成原理 · 单项选择题 · 2 分

若用 8 个 $64M\times 8$ 位的 DRAM 芯片按交叉编址方式构成主存储器，并与一个宽度为 64 位的存储器总线相连，主存每次最多读写 64 位，且按字节编址，则下列地址中，与主存地址 0018 001DH 位于同一芯片中的是

A. 0000 01D5H

B. 000F A020H

C. 0018 001EH

D. 0F02 0014H

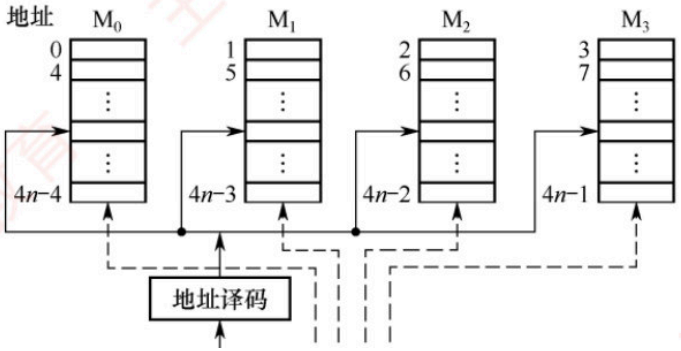
王道知识与解析 · 未定位原解析，仅附知识

【对应知识】王道2027计算机组成原理 · 原PDF第98页
交叉编址：低位体号、地址模模块数

2. 交叉编址（低位交叉）方式

考点追踪 ▶▶ 交叉存储器中数据的存放方式（2017）

低位地址用作模块号（体号），高位地址作为模块内地址。假设有 m 个模块，每个模块含 k 个存储单元，则模块编号由地址对 m 取模决定，即**模块号 = 单元地址%m**。如图 3.9 所示，单元 $0, m, \cdots, (k-1)m$ 位于模块 M_0；单元 $1, m+1, \cdots, (k-1)m+1$ 位于模块 M_1；以此类推。



【对应知识】王道2027计算机组成原理 · 原PDF第98页（续）
交叉编址：低位体号、地址模模块数

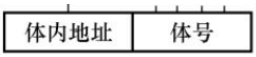


图 3.9　交叉编址的多体并行存储器

在交叉编址方式下，由于连续地址的数据被依次分布到不同模块中，程序或数据块在物理上是“**交叉存放**”的，采用此方式的多模块存储器被称为交叉存储器。通过多个结构完全相同的存储模块并行工作，这种设计能够在访问连续地址时显著提高存储器的吞吐率。

对应原题本第16页；点击此行返回题目。

2026–16

组成原理 · 单项选择题 · 2 分

对于下列选项中给出的相关内容，**不是**由指令集体系结构规定的是

A. 指令集中是否包含输入和输出指令

B. 采用向量中断还是非向量中断方式

C. 采用段式还是页式虚拟存储管理方式

D. 指令流水线是否采用超级流水线技术

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】王道2027计算机组成原理 · 原PDF第161页
ISA规定的内容

ISA 规定的内容主要包括：

1）指令格式、寻址方式、操作类型，以及各操作所需操作数的个数、类型和寻址约束。

2）操作数的数据类型，以及是按大端还是小端方式存放。

3）程序可访问的寄存器编号、数量和位数，存储空间的大小及编址方式。

4）程序员可见的控制状态，如程序计数器、条件码的定义与行为等。

ISA 规定了机器级程序的格式，因此机器语言或汇编语言程序员必须熟悉所用机器的 ISA。然而，大多数程序员使用高级语言（如 C/C++/Java）编程，因其开发效率高且不易出错。但高级语言的抽象层较高，隐藏了许多机器级细节，导致程序员难以利用与硬件结构相关的优化手段提升性能。若能充分理解 ISA 及底层硬件特性，则能开发出性能更优的程序。

【对应知识】王道2027计算机组成原理 · 原PDF第18页
微体系结构包含流水线级数

4．微体系结构

微体系结构（又称微架构）是处理器内部的硬件组织方式，用于实现 ISA 定义的功能。如果说 ISA 定义了“做什么”，那么微架构则决定了“怎么做”。**其核心设计包括**数据通路组织、控制单元实现、流水线级数、缓存层次结构以及分支预测机制等。

例如，加法操作可能通过串行进位加法器、超前进位加法器，甚至专用的 SIMD 单元来实现，这些都属于微体系结构的范畴。**相同的 ISA 可对应多种不同的微构架**。以 Intel x86 为例，不同代际的处理器（如 Core、Skylake、Alder Lake）均遵循同一套 ISA 规范，但内部组织方式差异显著，体现了微架构的多样性与演进性。

2026–17

组成原理 · 单项选择题 · 2 分

下列指令中，执行后有可能按存放顺序执行其下一条指令的是

I. 条件跳转指令

II. 过程调用指令

III. 陷阱（Trap）指令

IV. 过程返回指令

A. 仅 I、II

B. 仅 I、IV

C. 仅 II、III

D. 仅 III、IV

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】王道2027计算机组成原理 · 原PDF第163页
条件转移、调用和返回指令

4．顺序控制指令

考点追踪 ▶▶ 转移指令、调用和返回指令、条件转移指令的区分（2019）

顺序控制指令主要有无条件转移（JMP）、条件转移（BRANCH）、调用（CALL）、返回（RET）、陷阱（TRAP）等。无条件转移指令在任何情况下都执行转移操作，而条件转移指令仅在特定条件满足时执行转移操作，转移条件通常是某个标志位的值或多个标志位的组合。

调用指令与转移指令的区别在于：调用指令会保存下一条指令的地址（返回地址），以便子程序执行结束后能返回到主程序继续执行；而转移指令则不返回。

【对应知识】王道2027计算机组成原理 · 原PDF第255页
自陷指令的触发和返回行为

（2）自陷（Trap）

考点追踪 ▶▶ 自陷的原理和性质（2020）

自陷是一种**预先安排的**、用于主动转入操作系统内核的同步事件，也称陷阱或陷入。与故障不同，陷阱是在指令**正常执行完毕之后**才被触发的，因此 CPU 能够完整提交该指令的结果。典型应用包括：x86 计算机架构中的断点调试与单步跟踪机制、系统调用指令（如 syscall），以及 MIPS 架构中的条件自陷指令（如 teq）。它就像程序中预先布设的一个“陷阱”：通过特殊指令（如断点指令）或特定控制标志（如单步跟踪标志）显式设置。当执行到此类指令时，CPU 会根据其类型自动转移至相应的处理程序。处理完成后，CPU 总是**从自陷指令的下一条指令**继续执行。

故障异常和自陷异常属于程序性异常（软件中断）。

2026-18

组成原理 · 单项选择题 · 2 分

某计算机按字节编址，数据 Cache 共有 1024 行，采用 4 路组相联映射方式，主存块大小为 32 B。若需要访问主存地址为 1028 的 4 字节数据，则该数据所在主存块对应的 Cache 组号是

A. 4

B. 16

C. 32

D. 64

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】王道2027计算机组成原理 · 原PDF第125页
组相联映射：组数、组号取模和地址字段

3. 组相联映射

考点追踪 ▶▶ 组相联映射的原理（2009、2016、2018～2020、2023）

将 Cache 划分为 Q 个大小相等的**组**，每个主存块只能映射到**固定组**中的任意一行，即**组间采用直接映射，组内采用全相联映射**，如图 3.21 所示。它是直接映射与全相联映射的一种折中方案：当 $Q=1$（整个 Cache 为一个组）时，退化为**全相联映射**；当 $Q=$ Cache 总行数（每组仅 1 行）时，退化为**直接映射**。设每组包含 r 个 Cache 行，则称为 r 路组相联映射。

路数 r 越大，组内可选位置越多，块冲突概率越低，但所需的比较器数量和控制逻辑也越复杂。合理选择 r，可在硬件成本接近直接映射的同时，获得接近全相联映射的性能。

主存

【对应知识】王道2027计算机组成原理 · 原PDF第125页（续）
组相联映射：组数、组号取模和地址字段

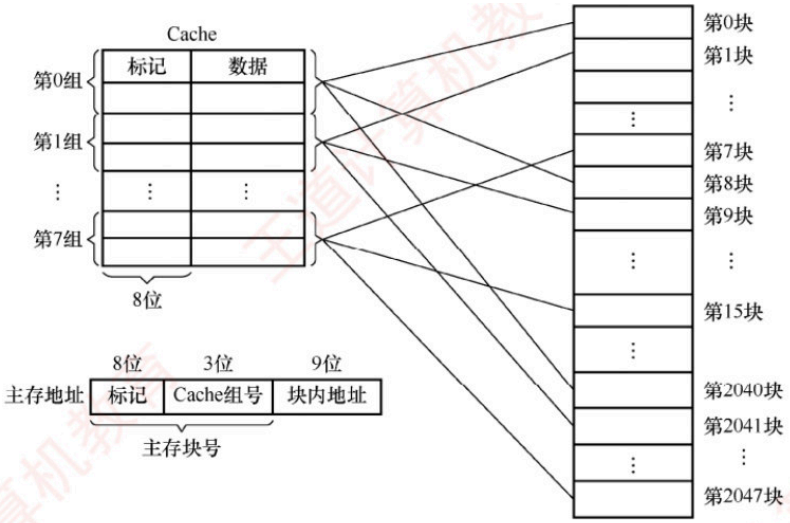


图 3.21　Cache 和主存之间的 2 路组相联映射方式

考点追踪 ▶▶ 组相联映射的地址结构及映射关系的分析（2025）

组相联映射关系可表示为

$$\text{Cache 组号} = \text{主存块号} \bmod \text{Cache 组数}（Q）$$

组相联映射的地址结构如下

标记	组号	块内地址

2026–19

组成原理 · 单项选择题 · 2 分

某计算机按字节编址，虚拟地址为 16 位，页大小为 256 B，页表项中包含装入位（P）、页框号（PPN）等字段，TLB 采用 4 路组相联映射方式，共有 16 个 TLB 表项，TLB 表项中包含标记（Tag）、有效位（V）等字段。在 TLB 表项与主存页表同步后，若主存页表中页号 22 对应的页表项中 P = 0、PPN = 2AH，则下列选项中，**不可能**出现在组号为 2 的 TLB 表项中的是

A. Tag = 05H、V = 1、PPN = 1CH

B. Tag = 06H、V = 1、PPN = 2AH

C. Tag = 16H、V = 0、PPN = 2AH

D. Tag = 1AH、V = 0、PPN = 1CH

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】王道2027计算机组成原理 · 原PDF第143页
页表装入位的含义

1．页表

图 3.27 是一个页表示例。有效位（也称装入位），表示对应虚页是否已调入主存，若为 1，表示该页已在主存，页表项中存放其物理页号；若为 0，表示未调入，页表项通常存放该页在外存（如磁盘）中的地址。脏位（也称修改位），表示页面是否被修改过，在采用回写策略的虚拟存储系统中，置换页面时根据脏位决定是否需将其写回磁盘。引用位（也称使用位），记录页面是否被访问过，主要用于实现基于使用历史的页面替换算法（如 Clock 或 LRU 算法）。

【对应知识】王道2027计算机组成原理 · 原PDF第144页
TLB组相联映射的标记与组索引

3．快表（TLB）

由地址转换过程可知，每次访存需先访问主存中的页表以获取物理页号，再访问主存取得实际数据，因此采用虚拟存储机制后，**平均访存次数增加，性能下降**。

考点追踪 ▶▶ TLB 的硬件实现（2018），TLB 和 Cache 的比较（2020）

根据程序访问的局部性原理，在一段时间内 CPU 往往集中访问少数页面。若将这些页面对应的页表项缓存在由**高速 SRAM 构成**的快表（TLB）中，则可在地址转换时避免访问主存中的页表，从而显著提升效率。相应地，主存中的页表常被称为慢表（Page）。

考点追踪 ▶▶ TLB 映射方式、地址划分与标记字段的分析（2016、2021）

TLB 的工作原理类似于 Cache，通常采用全相联或组相联映射。TLB 表项包含**虚拟页号**（作为标记）和对应的**物理页号**及**控制位**（如有效位、脏位等）。在全相联映射下，TLB 标记即为完整的虚拟页号；在组相联映射下，虚拟页号的高位作为标记，低位作为组索引。

2026-20

组成原理 · 单项选择题 · 2 分

在不考虑异常/中断处理和访存额外开销的前提下，下列关于数据通路结构与 CPI 之间关系的叙述中，正确的是

I. 单周期数据通路计算机的 CPI 等于 1

II. 多周期数据通路计算机的 CPI 大于 1

III. 流水线数据通路计算机的 CPI 等于 1

A. 仅 I、II

B. 仅 I、III

C. 仅 II、III

D. I、II、III

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】王道2027计算机组成原理 · 原PDF第215页
单周期、多周期、流水线处理器及CPI

5.2.4 处理器指令执行模型

考点追踪 ▶▶ 单周期和多周期 CPU 的 CPI（2016、2020、2025）

一个指令周期通常由若干依次执行的步骤组成，各步骤协同完成指令的全部功能。不同处理器对这些步骤的组织方式存在显著差异，主要可分为以下三类模型。

1．单周期处理器

单周期处理器为所有指令分配相同的执行时间，每条指令在一个时钟周期内完成（CPI = 1）。指令之间严格串行执行，即下一条指令必须等待前一条指令完全结束后才能启动。因此，时钟周期的长度由执行时间最长的指令决定。对于原本可在更短时间内完成的指令，也要占用整个周期，导致硬件资源在部分时间内空闲，限制了系统的整体性能。

2．多周期处理器

多周期处理器根据指令类型动态分配执行周期数，不同指令可占用不同数量的时钟周期（各指令的 CPI 不同，平均 CPI 通常大于 1）。该方案不再要求所有指令具有相同的执行时间，从而提高了时钟频率和资源利用效率。但是，指令之间仍是串行执行的，无法实现重叠处理。

【对应知识】王道2027计算机组成原理 · 原PDF第215页（续）
单周期、多周期、流水线处理器及CPI

3．流水线处理器

流水线处理器采用指令级并行策略，目标是在每个时钟周期完成一条指令的吞吐（理想情况下 CPI = 1）。其实现机制是：每个时钟周期启动一条新指令，使多条指令在流水线中重叠执行，

【对应知识】王道2027计算机组成原理 · 原PDF第216页
流水线模型的延续说明

各自处于不同的执行阶段（如取指、译码、执行、写回等）。尽管单条指令从开始到完成仍需要多个周期（执行延迟未减少），但整体吞吐率显著提高，大幅提升了处理器效率。

2026–21

组成原理 · 单项选择题 · 2 分

I/O 子系统中，由驱动程序和中断服务程序直接控制外设与主机之间的输入/输出操作，其中需要用到一些特权指令。下列指令中，不属于特权指令的是

A. I/O 指令

B. 关中断指令

C. 中断返回指令

D. 系统调用指令

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】王道2027计算机组成原理 · 原PDF第163页
I/O指令与CPU控制指令

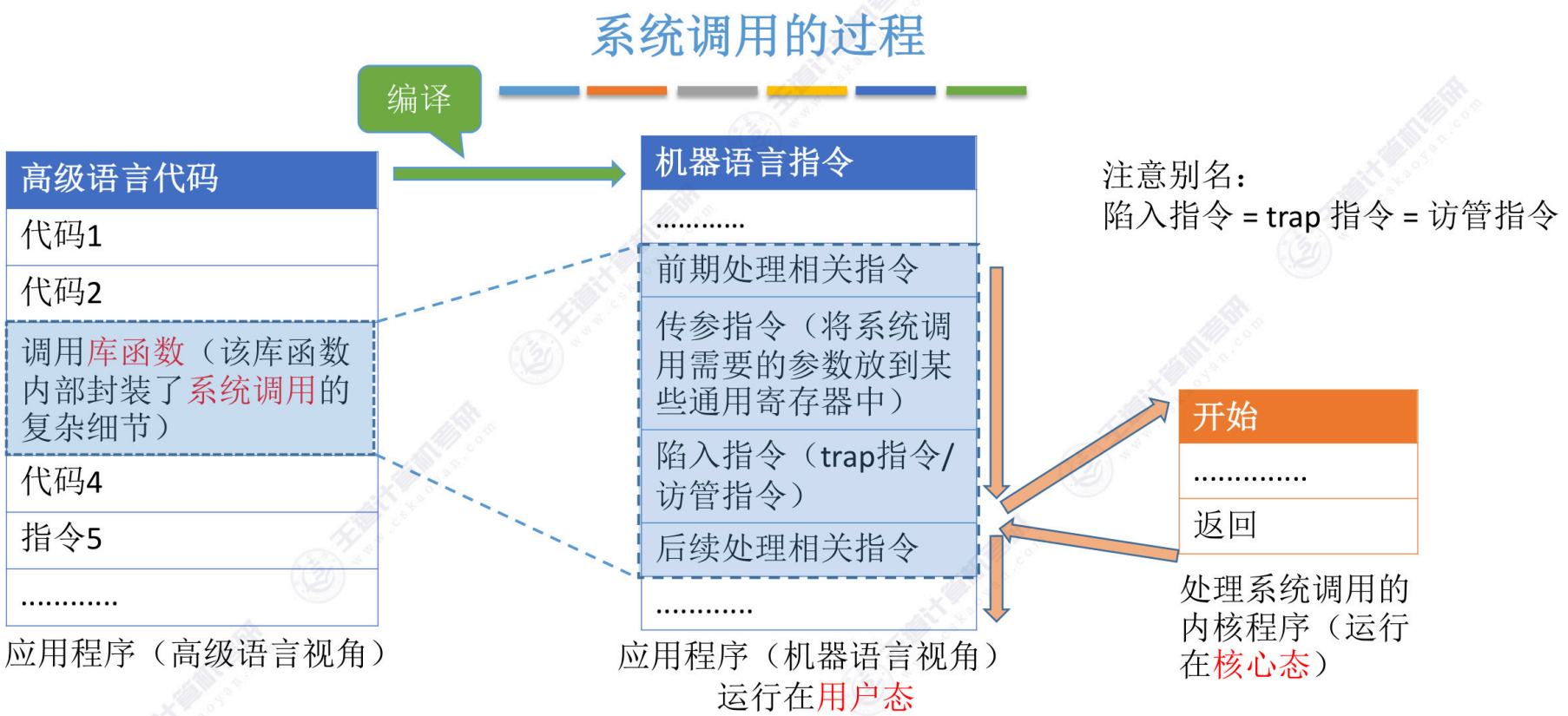
5．输入输出指令

输入输出指令用于完成 CPU 与外部设备交换数据或传送控制命令及状态信息。

6．CPU 控制指令

CPU 控制指令主要有停机、开中断、关中断、系统模式切换以及进入特殊处理程序等。这类指令只能在操作系统内核代码中使用，以防止用户误用，对系统运行造成危害。

【对应知识】1.3.3 系统调用 · 原PDF第8页
系统调用：陷入指令在用户态执行



传递系统调用参数 → 执行陷入指令（用户态） → 执行相应的内请求核程序处理系统调用（核心态） → 返回应用程序

注意：1. 陷入指令是在用户态执行的，执行陷入指令之后立即引发一个内中断，使CPU进入核心态
2. 发出系统调用请求是在用户态，而对系统调用的相应处理在核心态下进行

王道考研/CSKAOYAN.COM

对应原题本第22页；点击此行返回题目。

2026–22

组成原理 · 单项选择题 · 2 分

中断控制 I/O 方式下，实现 I/O 需要硬件和软件协同完成。中断响应及处理过程所包含的下列工作中，必须由硬件完成的是

A. 开中断

B. 中断判优

C. 保存断点

D. 保存通用寄存器

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】王道2027计算机组成原理 · 原PDF第256页
异常和中断响应由硬件完成；软件轮询判别中断源

中断响应过程。该过程指从事件发生到转移至相应处理程序之间的硬件自动操作序列，主要包括**关中断、保存断点和程序状态、以及识别事件类型并转移至处理程序**三个阶段。

（1）关中断

为确保上下文保存的原子性，在响应异常或中断的初始阶段，CPU 会自动屏蔽可屏蔽中断（通常通过清零标志寄存器中的 IF 位）。IF = 0 表示关中断，IF = 1 表示开中断。需要注意的是，不可屏蔽中断和部分高优先级异常仍可能在此期间发生，具体行为取决于体系结构设计。

（2）保存断点和程序状态

为支持处理完成后正确返回原程序，CPU 必须保存两类关键上下文信息：① 断点（返回地址），对于异常通常为当前指令地址，对于中断则为指令执行完毕后的下一条指令地址；② 程序状态字（PSW），包含条件码、中断允许标志、特权级等运行状态。这些信息通常被压入**内核栈**，既保障了上下文的安全性，又天然支持中断与异常的嵌套处理。

（3）识别事件类型并转移至处理程序

现代处理器普遍采用**硬件向量机制**（称为向量中断）来识别和响应异常与中断：每类事件都被分配唯一的类型号；系统初始化时，将对应服务程序的入口地址（称为中断向量）按类型号顺序填入中断向量表；当事件发生时，CPU 自动获取类型号，并以此为索引直接访问向量表，获取目标地址并转移至相应处理程序，全程无须软件介入。

相比之下，早期系统常采用**软件轮询方式**（称为非向量中断）：CPU 响应请求后转移至统一入口，操作系统需要依次读取各设备的状态寄存器，通过查询确定中断源。该方式依赖软件判别、效率较低，现代系统已极少使用。

整个响应过程由**硬件自动完成**，具有高度原子性。响应结束后，CPU 开始执行对应服务程序

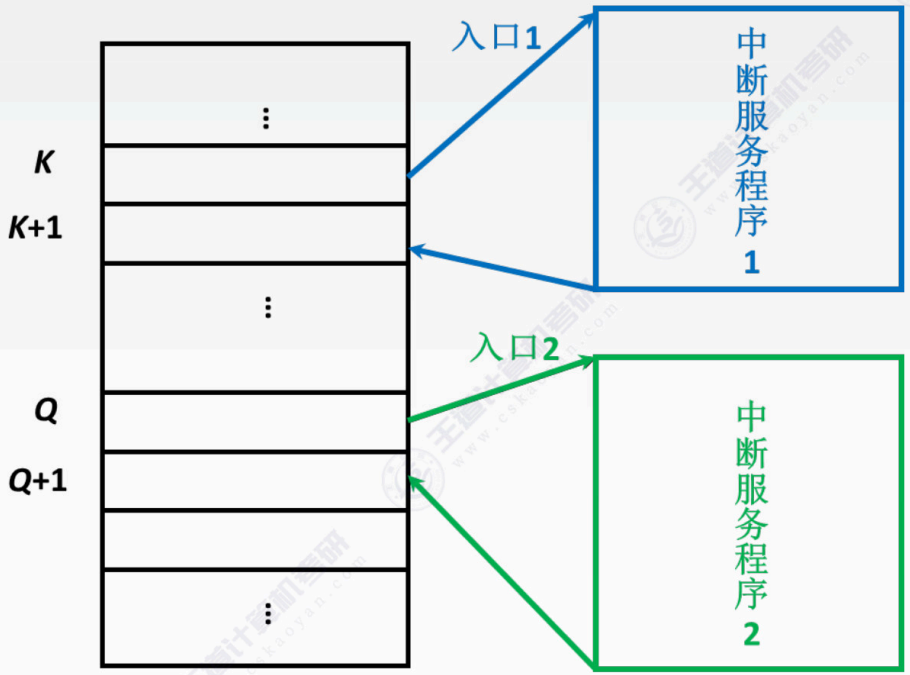
【对应知识】王道2027计算机组成原理 · 原PDF第256页（续）
异常和中断响应由硬件完成；软件轮询判别中断源

的第一条指令。后续处理逻辑（如缺页页面调入、系统调用分发等）由操作系统内核的软件实现，因此，完整的异常与中断处理是**软硬件紧密协同**的结果。

【对应知识】7.3.2_1 中断的作用和原理 · 原PDF第9页
中断隐指令自动保存断点

中断处理过程-中断隐指令

并不是一条具体的指令，而是CPU在检测到中断请求时自动完成的一系列动作



中断隐指令的主要任务：

① **关中断**。在中断服务程序中，为了保护中断现场（即CPU主要寄存器中的内容）期间不被新的中断所打断，必须关中断，从而保证被中断的程序在中断服务程序执行完毕之后能接着正确地执行下去。

② **保存断点**。为了保证在中断服务程序执行完毕后能正确地返回到原来的程序，必须将原来程序的断点（即程序计数器（PC）的内容）保存起来。可以存入堆栈，也可以存入指定单元。

③ **引出中断服务程序**。引出中断服务程序的实质就是取出中断服务程序的入口地址并传送给程序计数器（PC）。

- 软件查询法
- 硬件向量法

王道考研/CSKAOYAN.COM

对应原题本第23页；点击此行返回题目。

2026-23

操作系统 单项选择题 · 2 分

下列程序中，在内核模式执行的是

A. 编译程序

B. 链接程序

C. 装入程序

D. 命令解释程序

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】王道2027操作系统 · 原PDF第27页
王道2027操作系统：内核的系统资源管理功能

2. 系统资源管理功能

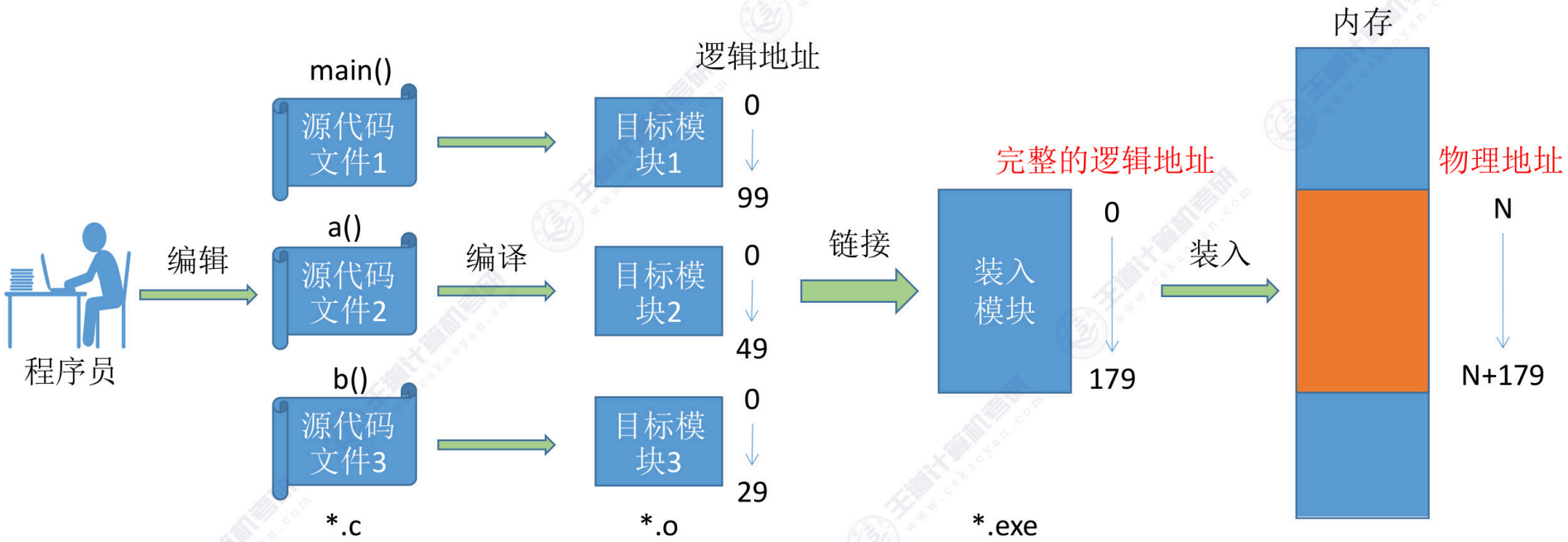
操作系统通过一系列核心数据结构（如进程控制块、设备控制块、内存分配表、缓冲区等）对系统资源进行统一管理。为实现高效、安全的资源使用，内核需提供以下三类基本管理功能：

1）进程管理。负责进程的创建与撤销、状态转换、调度与分派，并维护进程控制块（PCB）等核心数据结构。

2）存储器管理。实现内存的分配与回收、地址映射、内存保护及页面置换等机制，确保多道程序安全、高效地共享主存资源。

3）设备管理。完成设备的分配与回收、缓冲区管理、I/O 调度及设备驱动调用，屏蔽硬件差异，为用户提供统一的设备访问接口。

【对应知识】3.1.1_1 内存的基础知识 · 原PDF第16页
王道原课件：3.1.1_1 内存的基础知识



编译：由编译程序将用户源代码编译成若干个目标模块（编译就是把高级语言翻译为机器语言）
链接：由链接程序将编译后形成的一组目标模块，以及所需库函数链接在一起，形成一个完整的装入模块
装入（装载）：由装入程序将装入模块装入内存运行

王道考研/CSKAOYAN.COM

对应原题本第24页；点击此行返回题目。

2026-24

操作系统 单项选择题 · 2 分

下列是支持虚拟存储管理的系统中指令执行过程的相关叙述，其中正确的是

A. 虚实地址转换由操作系统完成

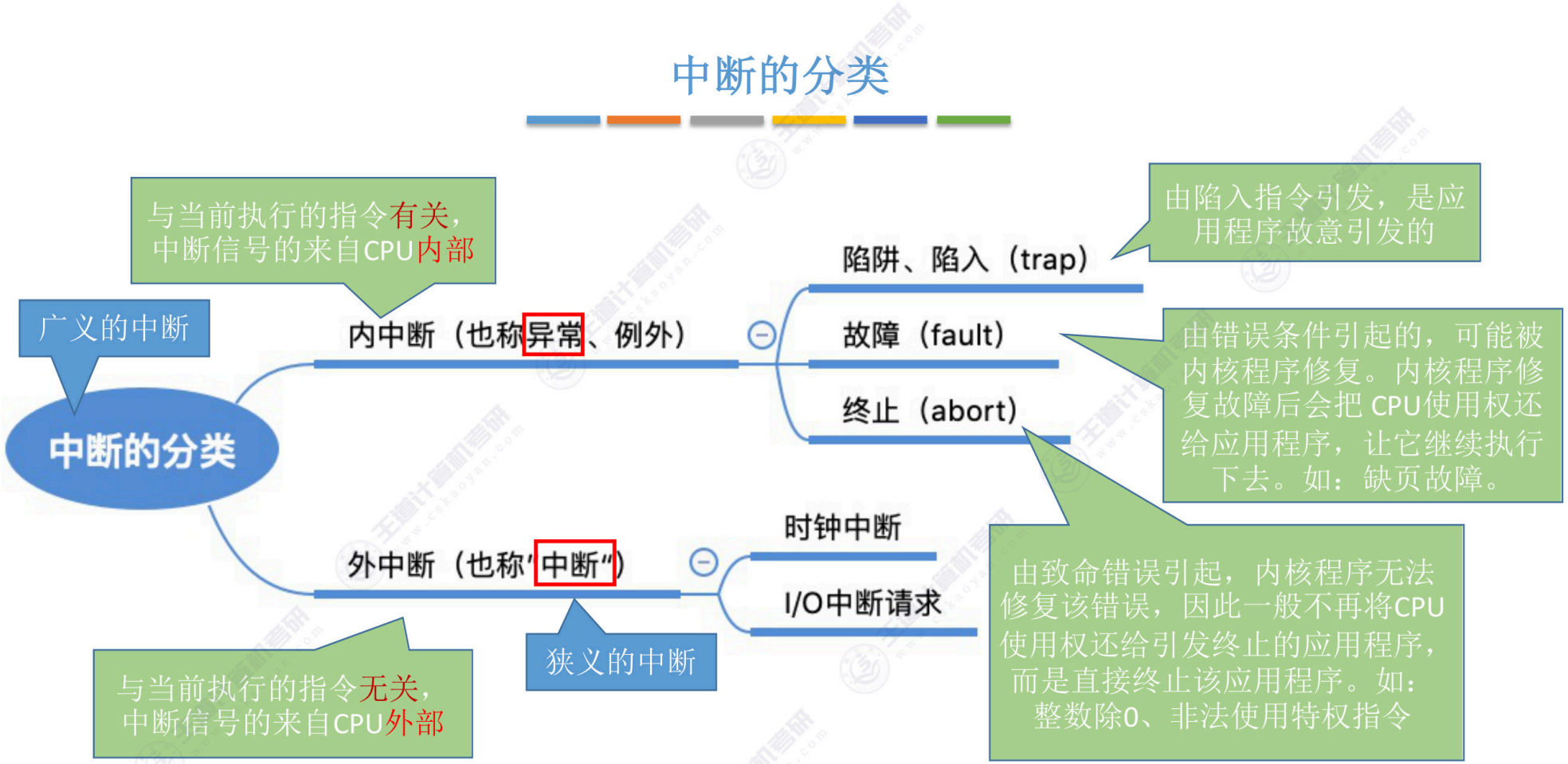
B. 存储访问权限由操作系统检测

C. 指令的合法性由操作系统验证

D. CPU 产生的异常由操作系统处理

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】1.3.2 中断和异常 · 原PDF第8页
王道原课件：1.3.2 中断和异常



大多数的教材、试卷中，“中断”特指狭义的中断，即外中断。而内中断一般称为“异常”

王道考研/CSKAOYAN.COM

对应原题本第25页；点击此行返回题目。

【对应知识】3.1.3_3 具有快表的地址变换机构 · 原PDF第5页
王道原课件：3.1.3_3 具有快表的地址变换机构

引入快表后，地址的变换过程

① CPU给出逻辑地址，由某个硬件算得页号、页内偏移量，将页号与快表中的所有页号进行比较。
② 如果找到匹配的页号，说明要访问的页表项在快表中有副本，则直接从中取出该页对应的内存块号，再将内存块号与页内偏移量拼接形成物理地址，最后，访问该物理地址对应的内存单元。因此，若快表命中，则访问某个逻辑地址仅需一次访存即可。
③ 如果没有找到匹配的页号，则需要访问内存中的页表，找到对应页表项，得到页面存放的内存块号，再将内存块号与页内偏移量拼接形成物理地址，最后，访问该物理地址对应的内存单元。因此，若快表未命中，则访问某个逻辑地址需要两次访存（注意：在找到页表项后，应同时将其存入快表，以便后面可能的再次访问。但若快表已满，则必须按照一定的算法对旧的页表项进行替换）

由于查询快表的速度比查询页表的速度快很多，因此只要快表命中，就可以节省很多时间。
因为局部性原理，一般来说快表的命中率可以达到 90% 以上。

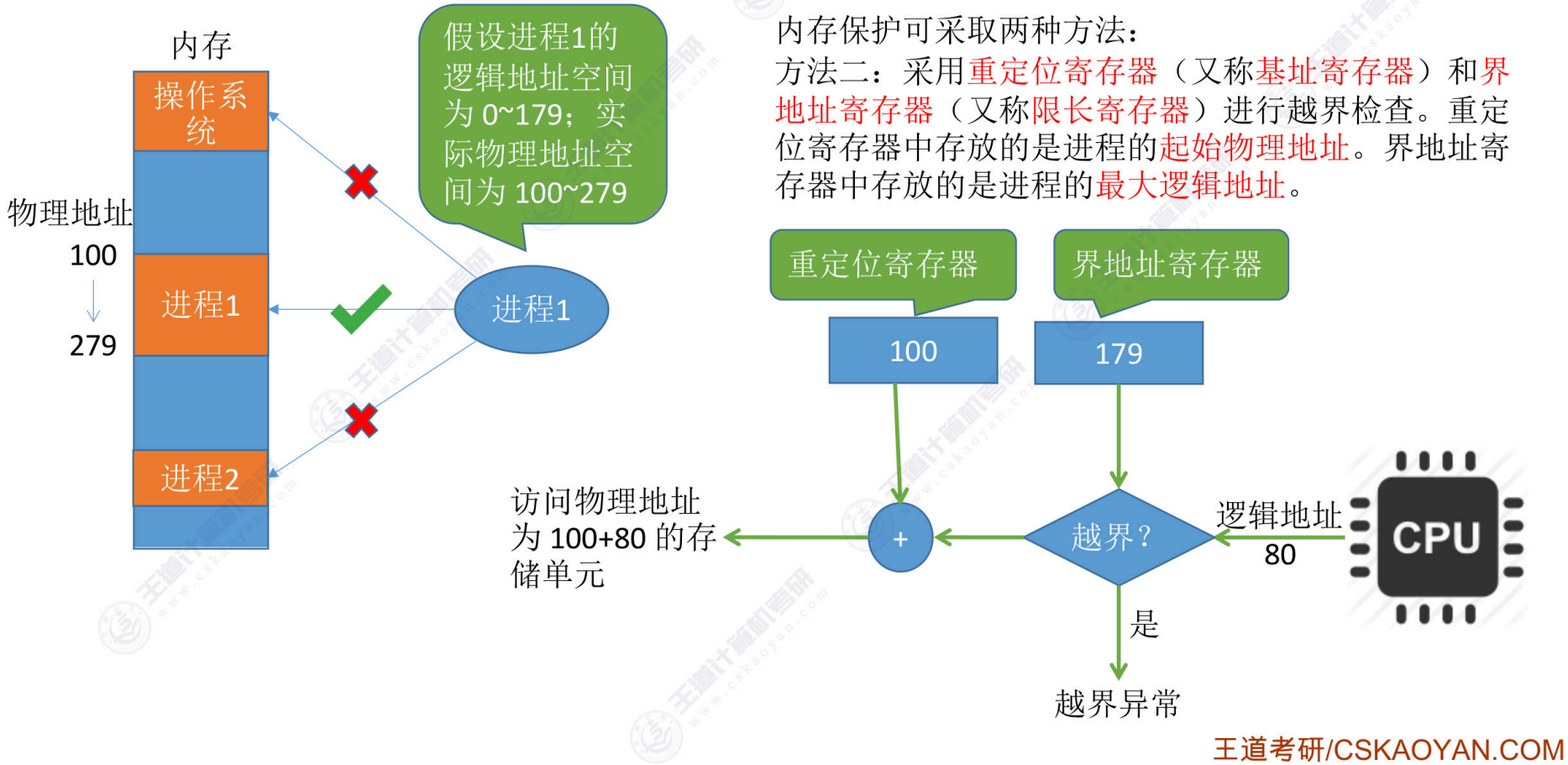
例：某系统使用基本分页存储管理，并采用了具有快表的地址变换机构。访问一次快表耗时 1us，访问一次内存耗时 100us。若快表的命中率为 90%，那么访问一个逻辑地址的平均耗时是多少？
(1+100) * 0.9 + (1+100+100) * 0.1 = 111 us
有的系统支持快表和慢表同时查找，如果是这样，平均耗时应该是 (1+100) * 0.9 + (100+100) * 0.1 = 110.9 us
若未采用快表机制，则访问一个逻辑地址需要 100+100 = 200us
显然，引入快表机制后，访问一个逻辑地址的速度快多了。

王道考研/CSKAOYAN.COM

对应原题本第25页；点击此行返回题目。

【对应知识】3.1.1_2 内存管理的概念 · 原PDF第9页
王道原课件：3.1.1_2 内存管理的概念

内存保护



对应原题本第25页；点击此行返回题目。

2026-25

操作系统 · 单项选择题 · 2 分

下列关于线程的叙述中，正确的是

A. 用户级线程和内核级线程均由操作系统创建

B. 多个内核级线程可以映射到同一个用户级线程

C. 同一进程的不同内核级线程共用该进程的内核栈

D. 同一进程的不同线程共享该进程虚拟地址空间中的堆

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】王道2027操作系统 · 原PDF第58页
王道2027操作系统：线程属性、私有状态与共享资源

考点追踪 ▶▶ 线程的特点（2019、2011、2024）

线程可被通俗地理解为“轻量级进程”，它是**程序执行流的最小单元**，也是**处理器调度的基本单位**。在多线程操作系统中，进程不再作为基本的执行实体，但仍保留与执行相关的状态。所谓进程处于“执行”状态，实际上是指该进程中的某个线程正在运行。线程的主要属性如下。

（1）轻量级执行实体

线程本身不拥有独立的系统资源（如地址空间、打开的文件等），但拥有运行所必需的私有数据结构，包括**线程 ID**、**程序计数器**、**寄存器集合**和**栈**。每个线程都具有唯一的标识符和一个线程控制块（TCB），用于记录其执行上下文（如寄存器状态、栈指针等）。

（2）共享代码段

同一进程中的多个线程可并发执行相同的程序代码。例如，当多个用户同时访问一个 Web 服务器时，服务器进程可创建多个线程，各自服务不同的用户请求，显著提升并发能力。

（3）资源共享

同一进程内的所有线程共享该进程的代码段、数据段、堆、文件描述符等系统资源。这种机制使得线程间的通信和数据共享非常高效，无须复杂的进程间通信（IPC）机制。

（4）独立调度单位

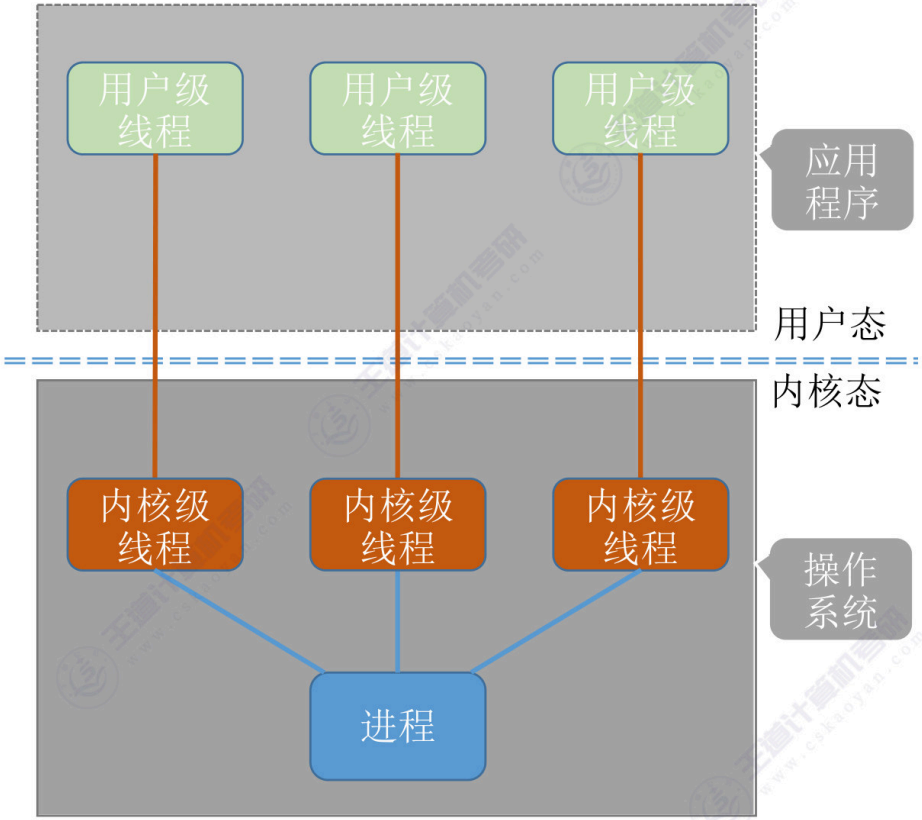
线程是 CPU 调度和执行的基本单位。在单 CPU 系统中，多个线程通过时间片轮转等方式轮流使用 CPU；在多 CPU 系统中，不同线程可并行运行于多个 CPU 核心上。若多个核心同时执行同一进程的不同线程，则能显著缩短该进程的整体处理时间。

【对应知识】2.1.6_2 线程的实现方式和多线程模型 · 原PDF第8页
王道原课件：2.1.6_2 线程的实现方式和多线程模型

线程的实现方式

内核级线程（Kernel-Level Thread, KLT, 又称“内核支持的线程”）

由操作系统支持的线程



1. 内核级线程的管理工作由操作系统内核完成。
2. 线程调度、切换等工作都由内核负责，因此内核级线程的切换必然需要在核心态下才能完成。
3. 操作系统会为每个内核级线程建立相应的TCB（Thread Control Block，线程控制块），通过TCB对线程进行管理。“内核级线程”就是“从操作系统内核视角看能看到的线程”
4. 优缺点

优点：当一个线程被阻塞后，别的线程还可以继续执行，并发能力强。多线程可在多核处理机上并行执行。

缺点：一个用户进程会占用多个内核级线程，线程切换由操作系统内核完成，需要切换到核心态，因此线程管理的成本高，开销大。

王道考研/CSKAOYAN.COM

对应原题本第26页；点击此行返回题目。

2026-26

操作系统 · 单项选择题 · 2 分

某系统中有 8 个进程均会执行如右图所示的代码片段，其中信号量 S 的初值为 5。若某时刻 S 的值为 -2，已执行到代码 Q 中的进程有 m 个，因执行 wait(S) 而被阻塞的进程有 n 个，则 m 和 n 的值分别是

```
wait(S);
代码Q;
signal(S);
```

A. 5、2

B. 5、3

C. 6、2

D. 7、1

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】2.3.4_1 信号量机制 · 原PDF第7页
王道原课件：2.3.4_1 信号量机制

信号量机制——记录型信号量

```
/*记录型信号量的定义*/
typedef struct {
  int value;              //剩余资源数
  Struct process *L;      //等待队列
} semaphore;

/*某进程需要使用资源时，通过 wait 原语申请*/
void wait (semaphore S) {
  S.value--;
  if (S.value < 0 ) {
    block (S.L);
  }
}

/*进程使用完资源后，通过 signal 原语释放*/
void signal (semaphore S) {
  s.value++;
  if (S.value <= 0) {
    wakeup(S.L);
  }
}
```

在考研题目中 wait(S)、signal(S) 也可以记为 P(S)、V(S)，这对原语可用于实现系统资源的“申请”和“释放”。

S.value 的初值表示系统中某种资源的数目。

对信号量 S 的一次 P 操作意味着进程请求一个单位的该类资源，因此需要执行 S.value--，表示资源数减1，当 S.value < 0 时表示该类资源已分配完毕，因此进程应调用 block 原语进行自我阻塞（当前运行的进程从运行态→阻塞态），主动放弃处理机，并插入该类资源的等待队列 S.L 中。可见，该机制遵循了“让权等待”原则，不会出现“忙等”现象。

对信号量 S 的一次 V 操作意味着进程释放一个单位的该类资源，因此需要执行 S.value++，表示资源数加1，若加1后仍是 S.value <= 0，表示依然有进程在等待该类资源，因此应调用 wakeup 原语唤醒等待队列中的第一个进程（被唤醒进程从阻塞态→就绪态）。

王道考研/CSKAOYAN.COM

对应原题本第27页；点击此行返回题目。

操作系统 · 单项选择题 · 2 分

若进程 P 读、写数据的集合分别记为 R(P) 和 W(P)，进程 Q 读、写数据的集合分别记为 R(Q) 和 W(Q)，则进程 P 和 Q 并发执行时**不会**发生并发错误的充要条件是

I. $W(P)\cap W(Q)=\varnothing$

II. $R(P)\cap W(Q)=\varnothing$

III. $W(P)\cap R(Q)=\varnothing$

IV. $R(P)\cap R(Q)=\varnothing$

A. I、IV 同时成立

B. II、III 同时成立

C. I、II、III 同时成立

D. I、II、III、IV 同时成立

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】2.3.5_2 读者-写者问题 · 原PDF第3页
王道原课件：并发读写与互斥条件

问题分析

有读者和写者两组并发进程，共享一个文件，当两个或两个以上的读进程同时访问共享数据时不会产生副作用，但若某个写进程和其他进程（读进程或写进程）同时访问共享数据时则可能导致数据不一致的错误。因此要求：①允许多个读者可以同时对文件执行读操作；②只允许一个写者往文件中写信息；③任一写者在完成写操作之前不允许其他读者或写者工作；④写者执行写操作前，应让已有的读者和写者全部退出。

1. 关系分析。找出题目中描述的各个进程，分析它们之间的同步、互斥关系。
2. 整理思路。根据各进程的操作流程确定P、V操作的大致顺序
3. 设置信号量。设置需要的信号量，并根据题目条件确定信号量初值。（互斥信号量初值一般为1，同步信号量的初始值要看对应资源的初始值是多少）

两类进程：写进程、读进程
互斥关系：写进程—写进程、写进程—读进程。读进程与读进程不存在互斥问题。

王道考研/CSKAOYAN.COM

对应原题本第28页；点击此行返回题目。

2026–28

操作系统 · 单项选择题 · 2 分

某 64 位系统采用三级页式存储管理方式，虚拟地址结构如下图所示。

保留 25位	一级页号 9位	二级页号 9位	三级页号 9位	页内偏移 12位

若各级页表项均占 8 字节，则进程的第三级页表占用的页框数最多是

A. 512

B. 32 K

C. 256 K

D. 128 M

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】3.1.3_4 两级页表 · 原PDF第10页
王道原课件：3.1.3_4 两级页表

需要注意的几个细节

1. 若分为两级页表后，页表依然很长，则可以采用更多级页表，一般来说各级页表的大小不能超过一个页面

例：某系统按字节编址，采用 40 位逻辑地址，页面大小为 4KB，页表项大小为 4B，假设采用纯页式存储，则要采用（）级页表，页内偏移量为（）位？

页面大小 = 4KB =2^{12}B，按字节编址，因此页内偏移量为12位
页号 = 40 - 12 = 28 位
页面大小 = 2^{12}B，页表项大小 = 4B，则每个页面可存放 2^{12} / 4 = 2^{10} 个页表项
因此各级页表最多包含 2^{10} 个页表项，需要 10 位二进制位才能映射到 2^{10} 个页表项，因此每一级的页表对应页号应为10位。总共28位的页号至少要分为三级

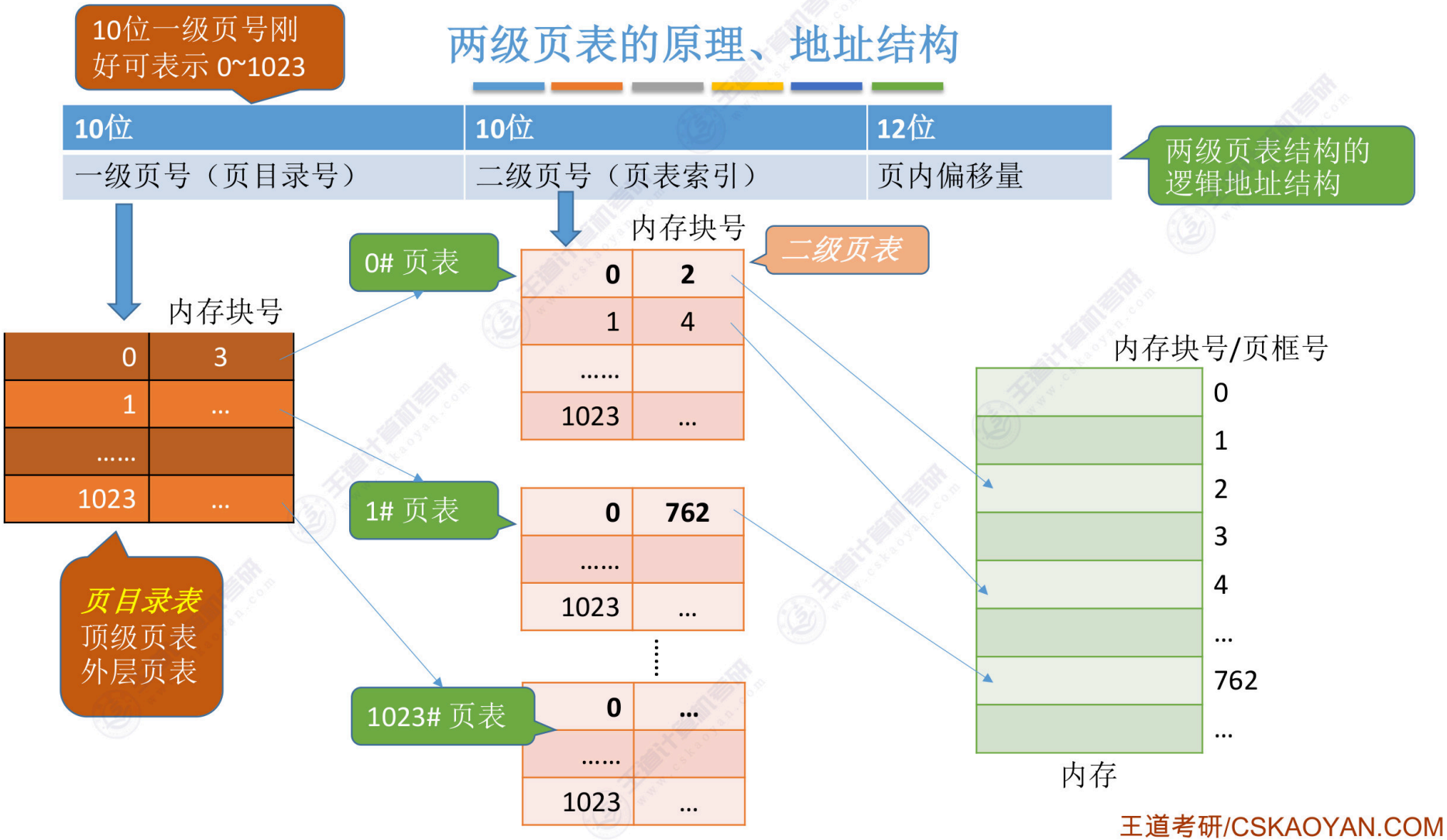
2. 两级页表的访存次数分析（假设没有快表机构）
第一次访存：访问内存中的页目录表
第二次访存：访问内存中的二级页表
第三次访存：访问目标内存单元

如果只分为两级页表，则一级页号占 18 位，也就是说页目录表中最多可能有 2^{18} 个页表项，显然，一个页面是放不下这么多页表项的。

王道考研/CSKAOYAN.COM

对应原题本第29页；点击此行返回题目。

【对应知识】3.1.3_4 两级页表 · 原PDF第7页
王道原课件：3.1.3_4 两级页表



对应原题本第29页；点击此行返回题目。

2026–29

操作系统 · 单项选择题 · 2 分

下列页式虚拟存储管理系统的技术中，可用于降低系统平均访存时间的是

I. TLB

II. 多级页表

III. 工作集模型

IV. 页缓冲队列

A. 仅 I、II

B. 仅 I、IV

C. 仅 II、III

D. 仅 I、III、IV

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】3.1.3_3 具有快表的地址变换机构 · 原PDF第5页
王道原课件：3.1.3_3 具有快表的地址变换机构

引入快表后，地址的变换过程

① CPU给出逻辑地址，由某个硬件算得页号、页内偏移量，将页号与快表中的所有页号进行比较。
② 如果找到匹配的页号，说明要访问的页表项在快表中有副本，则直接从中取出该页对应的内存块号，再将内存块号与页内偏移量拼接形成物理地址，最后，访问该物理地址对应的内存单元。因此，若快表命中，则访问某个逻辑地址仅需一次访存即可。
③ 如果没有找到匹配的页号，则需要访问内存中的页表，找到对应页表项，得到页面存放的内存块号，再将内存块号与页内偏移量拼接形成物理地址，最后，访问该物理地址对应的内存单元。因此，若快表未命中，则访问某个逻辑地址需要两次访存（注意：在找到页表项后，应同时将其存入快表，以便后面可能的再次访问。但若快表已满，则必须按照一定的算法对旧的页表项进行替换）

由于查询快表的速度比查询页表的速度快很多，因此只要快表命中，就可以节省很多时间。
因为局部性原理，一般来说快表的命中率可以达到 90% 以上。

例：某系统使用基本分页存储管理，并采用了具有快表的地址变换机构。访问一次快表耗时 1us，访问一次内存耗时 100us。若快表的命中率为 90%，那么访问一个逻辑地址的平均耗时是多少？
(1+100) * 0.9 + (1+100+100) * 0.1 = 111 us
有的系统支持快表和慢表同时查找，如果是这样，平均耗时应该是 (1+100) * 0.9 + (100+100) * 0.1 = 110.9 us
若未采用快表机制，则访问一个逻辑地址需要 100+100 = 200us
显然，引入快表机制后，访问一个逻辑地址的速度快多了。

王道考研/CSKAOYAN.COM

对应原题本第30页；点击此行返回题目。

【对应知识】3.2.5+3.2.3 页面分配策略 · 原PDF第10页
王道原课件：3.2.5+3.2.3 页面分配策略

工作集

驻留集：指请求分页存储管理中给进程分配的内存块的集合。
工作集：指在某段时间间隔里，进程实际访问页面的集合。

操作系统会根据“窗口尺寸”来算出工作集。例：
某进程的页面访问序列如下，窗口尺寸为 4，各时刻的工作集为？

24，15，18，23，24，17，18，24，18，17，17，15

工作集大小可能小于窗口尺寸，实际应用中，操作系统可以统计进程的工作集大小，根据工作集大小给进程分配若干内存块。如：窗口尺寸为5，经过一段时间的监测发现某进程的工作集最大为3，那么说明该进程有很好的局部性，可以给这个进程分配3个以上的内存块即可满足进程的运行需要。
一般来说，驻留集大小不能小于工作集大小，否则进程运行过程中将频繁缺页。

拓展：基于局部性原理可知，进程在一段时间内访问的页面与不久之后会访问的页面是有相关性的。因此，可以根据进程近期访问的页面集合（工作集）来设计一种页面置换算法——选择一个不在工作集中的页面进行淘汰。

王道考研/CSKAOYAN.COM

对应原题本第30页；点击此行返回题目。

【对应知识】王道2027操作系统 · 原PDF第254页
王道2027操作系统：缺页率因素与页缓冲队列

页置换算法会影响缺页率，例如，LRU 算法的缺页率通常要比 FIFO 算法的缺页率低，排除选项 A。工作集的大小决定了分配给进程的物理块数，分配给进程的物理块数越多，缺页率就越低，排除选项 B。进程的数量越多，对内存资源的竞争越激烈，每个进程被分配的物理块数越少，缺页率也就越高，排除选项 C。页缓冲队列是将被淘汰的页面缓存下来，暂时不写回磁盘，队列长度会影响页面置换的速度，但不会影响缺页率。

对应原题本第30页；点击此行返回题目。

2026-30

操作系统 · 单项选择题 · 2 分

假设两个进程共享一个页 S，其页表分别为 PT1 和 PT2，S 在两个进程虚拟地址空间中的起始地址分别为 VA1 和 VA2，则下列叙述中，正确的是

A. VA1 和 VA2 相同

B. VA1 和 VA2 映射到的物理地址相同

C. S 在 PT1 和 PT2 中的页表项内容相同

D. S 在 PT1 和 PT2 中的页表项虚拟地址相同

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】王道2027操作系统 · 原PDF第219页
王道2027操作系统：共享内存的页框与进程页号

进程 R 和 S 共享数据 data，说明它们都映射了同一个共享内存段，于是这个段在物理内存中的位置（页框号）必然相同，但这两个段在不同进程的地址空间中的位置（页号）可以不同。因

【对应知识】王道2027操作系统 · 原PDF第220页
王道2027操作系统：共享内存的页框与进程页号（续）

此，p1 和 p2 不一定相等，f1 和 f2 一定相等。

2026-31

操作系统 · 单项选择题 · 2 分

下列关于驱动程序的叙述中，错误的是

A. 驱动程序中包含启动设备的代码

B. 驱动程序中包含一组读/写设备的代码

C. 驱动程序与内核之间需要定义统一的接口

D. 块设备和字符设备驱动程序采用相同的 I/O 控制方式

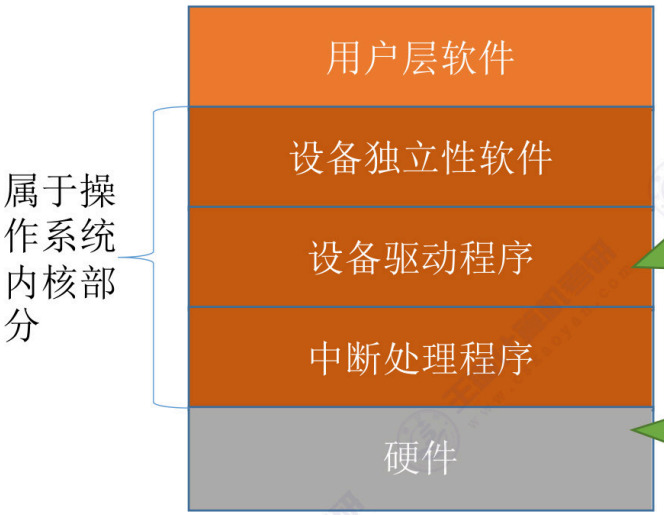
王道知识与解析 · 未定位原解析，仅附知识

【对应知识】5.1_4_IO软件层次结构 · 原PDF第16页
王道原课件：5.1_4_IO软件层次结构

设备驱动程序

逻辑设备表（LUT）

逻辑设备名	物理设备名	驱动程序入口地址
/dev/打印机1	3	1024
/dev/打印机2	5	2046
…	…	…



主要负责对硬件设备的具体控制，将上层发出的一系列命令（如read/write）转化成特定设备“能听得懂”的一系列操作。包括设置设备寄存器；检查设备状态等

不同的I/O设备有不同的硬件特性，具体细节只有设备的厂家才知道。因此厂家需要根据设备的硬件特性设计并提供相应的驱动程序。

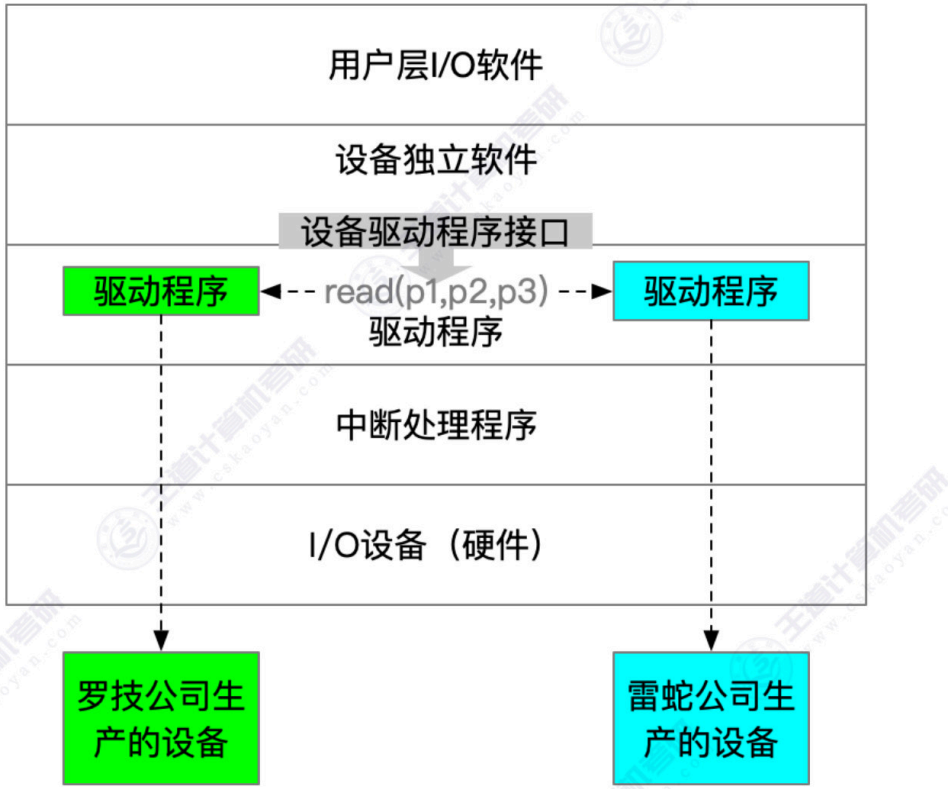
注：驱动程序一般会以一个独立进程的方式存在。

王道考研/CSKAOYAN.COM

对应原题本第32页；点击此行返回题目。

【对应知识】5.1_5_输入输出应用程序接口和驱动程序接口 · 原PDF第16页
王道原课件：5.1_5_输入输出应用程序接口和驱动程序接口

统一标准的设备驱动程序接口



操作系统规定好设备驱动程序的接口标准，各厂商必须按要求开发设备驱动程序

王道考研/CSKAOYAN.COM

对应原题本第32页；点击此行返回题目。

【对应知识】5.1_3_IO控制方式 · 原PDF第13页
王道原课件：5.1_3_IO控制方式

知识点回顾与重要考点

	完成一次读/写的过程	CPU干预频率	每次I/O的数据传输单位	数据流向	优缺点
程序直接控制方式	CPU发出I/O命令后需要不断轮询	极高	字	设备→CPU→内存 内存→CPU→设备	每一个阶段的优点都是解决了上一阶段的最大缺点。 总体来说，整个发展过程就是要尽量减少CPU对I/O过程的干预，把CPU从繁杂的I/O控制事务中解脱出来，以便更多地去完成数据处理任务。
中断驱动方式	CPU发出I/O命令后可以做其他事，本次I/O完成后设备控制器发出中断信号	高	字	设备→CPU→内存 内存→CPU→设备	
DMA方式	CPU发出I/O命令后可以做其他事，本次I/O完成后DMA控制器发出中断信号	中	块	设备→内存 内存→设备	
通道控制方式	CPU发出I/O命令后可以做其他事。通道会执行通道程序以完成I/O，完成后通道向CPU发出中断信号	低	一组块	设备→内存 内存→设备	

难点理解：
通道=弱鸡版CPU
通道程序=任务清单

王道考研/CSKAOYAN.COM

对应原题本第32页；点击此行返回题目。

2026–32

操作系统 · 单项选择题 · 2 分

下列操作中，由鼠标中断处理程序完成的是

A. 将当前进程的状态由执行态改为阻塞态

B. 将 CPU 运行模式由用户模式改为内核模式

C. 将接收到的输入数据存放到设备接口的数据寄存器中

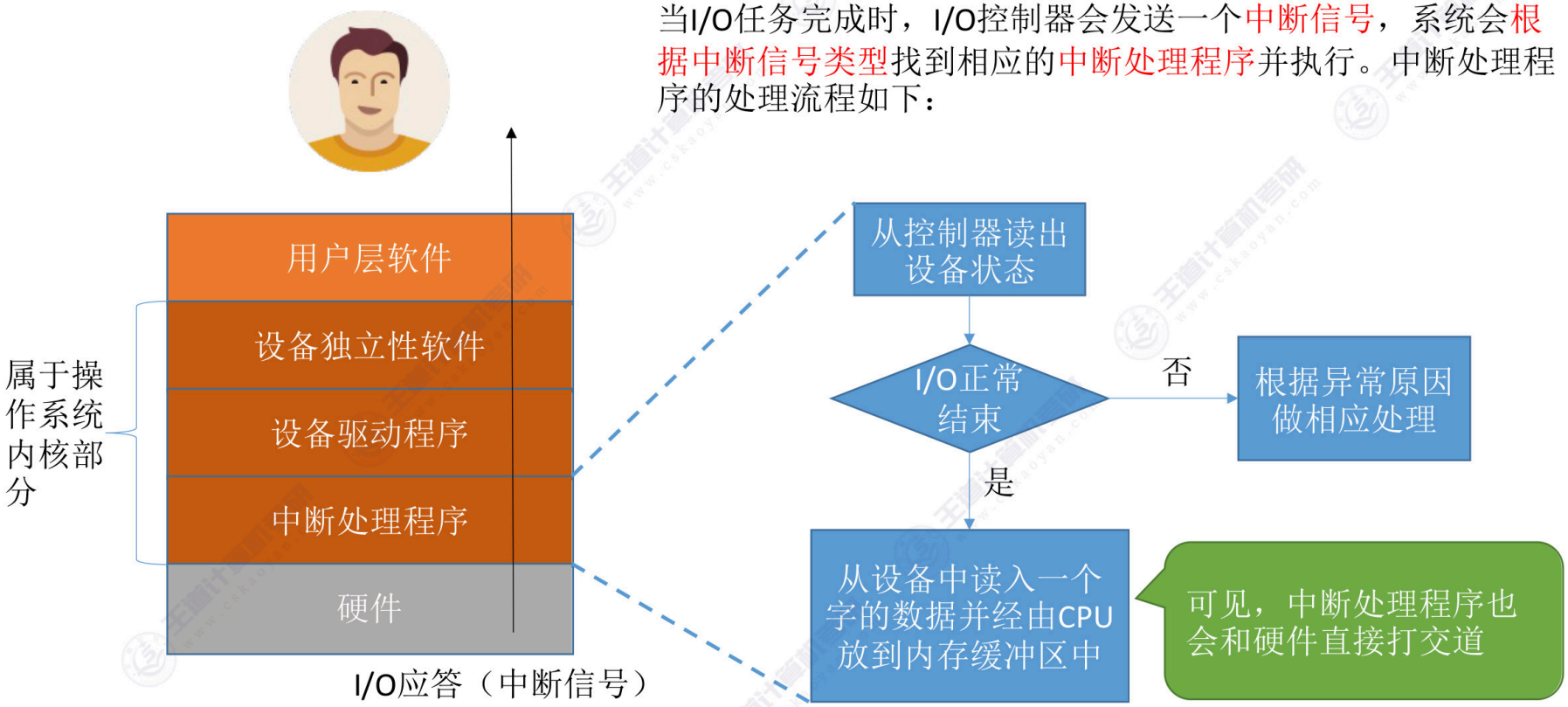
D. 将输入数据从设备接口的数据寄存器传送到内核数据区

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】5.1_4_IO软件层次结构 · 原PDF第17页
王道原课件：5.1_4_IO软件层次结构

中断处理程序

当I/O任务完成时，I/O控制器会发送一个中断信号，系统会根据中断信号类型找到相应的中断处理程序并执行。中断处理程序的处理流程如下：



王道考研/CSKAOYAN.COM

对应原题本第33页；点击此行返回题目。

2026–33

计算机网络 · 单项选择题 · 2 分

下列关于分层网络体系结构的叙述中，错误的是

A. 有利于标准化

B. 层次越多效率越高

C. 有利于各层技术独立演化

D. 降低网络系统设计与实现的复杂度

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】王道2027计算机网络 · 原PDF第28页
王道知识：各层封装PDU与协议控制开销

网络层的 PDU 称为 IP 分组，传输层的 PDU 称为 TCP 报文段或 UDP 数据报。

当数据在各层之间传递时，将从第 $n+1$ 层接收到的 PDU 作为第 n 层的 SDU，加上第 n 层的 PCI 后封装成第 n 层的 PDU，再交由第 $n-1$ 层作为其 SDU 发送。接收方收到后做相反的处理。

因此，三者的关系可表示为 n-SDU + n-PCI = n-PDU = ($n-1$)-SDU，如图 1.10 所示。

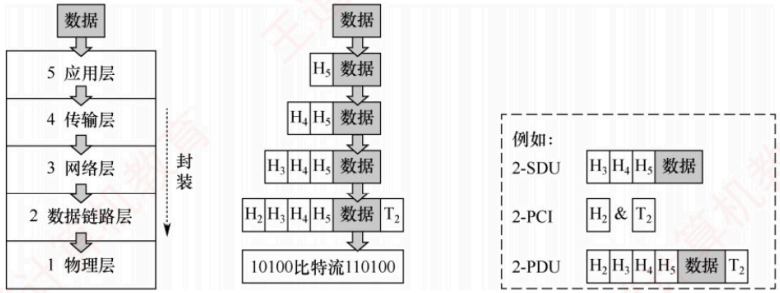


图 1.10　网络各层数据单元之间的关系

【对应知识】王道2027计算机网络 · 原PDF第27页
王道知识：分层思想、独立性与标准化

1.2.1　计算机网络分层结构

计算机网络是一个非常复杂的系统，因此在早期就采用了分层的设计思想。分层可将庞大而复杂的问题转换为若干较小的局部问题，而这些局部问题更易于研究和处理。

考点追踪 ▶▶ 网络体系结构的概念（2010）

计算机网络的各层及其协议的集合称为网络的体系结构（Architecture）。换言之，计算机网络的体系结构是对该网络及其所应完成功能的精确定义。需要强调的是，这些功能究竟由何种硬件或软件实现，属于实现（Implementation）问题。**体系结构是抽象的，而实现是具体的**，体现在实际运行的计算机硬件和软件中。计算机网络体系结构通常具有可分层的特性，能将复杂的大系统分解为若干较易实现的层次。

分层的**基本原则**如下：

1）每层实现一种相对独立的功能，以降低整个系统的复杂度。

2）各层之间的接口清晰、简洁，相互依赖尽可能少，便于理解与维护。

3）各层功能的定义独立于具体实现方法，可采用最合适的技术来实现。

4）保持下层对上层的独立性，上层单向使用下层提供的服务。

5）整个分层结构应有利于标准化工作。

2026–34

计算机网络 · 单项选择题 · 2 分

若在带宽为 200 kHz、信噪比 $S/N=1023$ 的信道上发送一个长度为 1500 B 的分组，则发送该分组的传输时延至少是

A. 0.375 ms

B. 0.75 ms

C. 3 ms

D. 6 ms

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】王道2027计算机网络 · 原PDF第19页
王道知识：发送时延与传播时延公式

4）时延（Delay）。指数据（一个报文或分组）从网络（或链路）的一端传送到另一端所需的**总时间**，它由四部分构成：发送时延、传播时延、处理时延和排队时延。

考点追踪 ▶▶ 网络时延的分析（2010、2013、2023、2025）

- 发送时延（也称传输时延）。节点将分组的所有比特推入链路所需的时间，即从发送分组的第一个比特开始，到最后一个比特发送完毕为止的时间。

$$\text{发送时延}=\text{分组长度}/\text{发送速率}$$

- 传播时延。电磁波在信道（传输介质）中传播一定距离所需的时间，即一个比特从链路一端传播到另一端所需的时间。

$$\text{传播时延}=\text{信道长度}/\text{电磁波在信道上的传播速率}$$

【对应知识】王道2027计算机网络 · 原PDF第44页
王道知识：香农定理、单位和信噪比

2．香农定理

考点追踪 ▶▶ 香农定理的应用（2014、2016）

实际信道存在**高斯白噪声**。香农定理给出了**带宽受限且有高斯噪声干扰的信道**的极限数据传输速率。当速率不超过该极限值时，理论上可以实现任意低的误码率。香农定理定义为

$$\text{信道的极限数据传输速率}=W\log_2(1+S/N)\quad\text{（单位为 b/s）}$$

式中，W 为信道的频率带宽（单位为 Hz），S 为信号的平均功率，N 为噪声功率。S/N 为信噪比，即信号的平均功率与噪声的平均功率之比。

信噪比有两种表示形式：无单位记法和分贝（dB）记法。当采用**无单位记法**时，信噪比 $=S/N$；当采用**分贝记法**时，信噪比 $=10\log_{10}(S/N)$（单位为 dB），例如，当 $S/N=1000$ 时，信噪比为 30dB。注意，使用香农定理计算信道的极限数据传输速率时，信噪比应采用无单位记法。

考点追踪 ▶▶ 信道带宽的影响因素（2014）

对于香农定理，有以下结论：

1）**信道带宽**或**信噪比越大**，信息的极限传输速率越高。

2）对于给定的带宽和信噪比，**信息传输速率的上限是确定的**。

3）只要实际信息传输速率低于该极限值，就能找到某种方法实现近似无差错传输。

4）香农定理得出的是**理论极限**，实际信道能达到的传输速率要比它低不少。

假设某采用 CSMA/CA 的 IEEE 802.11 无线局域网，数据传输速率为 300 Mbps，DIFS $=128\ \mu s$，SIFS $=28\ \mu s$，忽略除数据帧外的其他帧的传输时延以及信号传播时延。若主机 H 发送一个总长度为 1500 B 的数据帧，则 H 从开始发送数据帧时刻起，到确认接收方接收到该帧为止，所需的时间至少是

A. $40\ \mu s$

B. $68\ \mu s$

C. $168\ \mu s$

D. $196\ \mu s$

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】王道2027计算机网络 · 原PDF第19页
王道知识：发送时延=分组长度/发送速率

- 发送时延（也称传输时延）。节点将分组的所有比特推入链路所需的时间，即从发送分组的第一个比特开始，到最后一个比特发送完毕为止的时间。

发送时延=分组长度/发送速率

【对应知识】王道2027计算机网络 · 原PDF第108页
王道知识：DATA—SIFS—ACK时序与信道预约

3）A 站收到 CTS 帧后，再等待时间 SIFS，即可发送数据帧。其首部写入时间[DATA + SIFS + ACK]，即从开始接收数据帧起[①]，到 AP 完成 ACK 发送为止的时间。

考点追踪 ▶ NAV 的工作原理（2024）

任何站点只要收到 RTS 帧、CTS 帧或数据帧之一，就会根据其首部的持续期字段设置自己的 NAV，如图 3.27 所示。虽然 B 站收不到 A 站发送给 AP 的 RTS 帧，但能收到 AP 广播的 CTS 帧，B 站根据 CTS 帧首部的内容设置自己的 NAV，因此在这段时间内不会发送数据、干扰信道。因此，虚拟载波监听机制能够有效**减少由隐蔽站引发的冲突问题**。

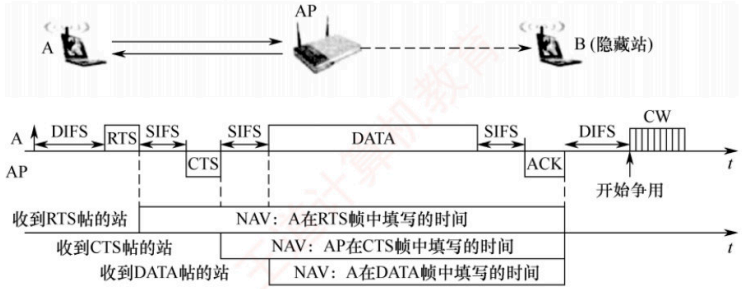
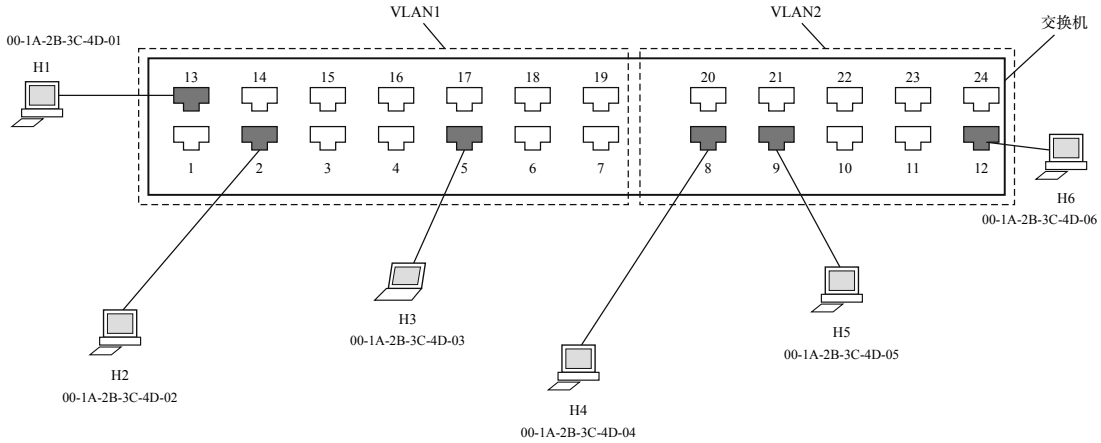


图 3.27 使用 RTS 和 CTS 帧的冲突避免

2026–36

计算机网络　单项选择题 · 2 分

支持 VLAN 划分的以太网交换机，已按端口划分了 2 个 VLAN。VLAN 划分结果及部分端口连接主机的 MAC 地址如下图所示。



下列具有不同目的 MAC 地址（DA）和源 MAC 地址（SA）的以太网帧 F1～F4 中，H3 会接收到的是

F1：DA = 00-1A-2B-3C-4D-03，SA = 00-1A-2B-3C-4D-01

F2：DA = 00-1A-2B-3C-4D-03，SA = 00-1A-2B-3C-4D-05

F3：DA = FF-FF-FF-FF-FF-FF，SA = 00-1A-2B-3C-4D-02

F4：DA = FF-FF-FF-FF-FF-FF，SA = 00-1A-2B-3C-4D-06

A. 仅 F1、F2

B. 仅 F1、F3

C. 仅 F2、F4

D. 仅 F3、F4

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】王道2027计算机网络 · 原PDF第111页
王道知识：每个VLAN独立广播域、同VLAN直接通信

3.6.4 VLAN 基本概念与基本原理

一个以太网构成一个广播域。当网络中包含的计算机数量过多时，往往会导致：

- 网络中出现大量广播帧，尤其是频繁使用的 ARP 和 DHCP 报文（见第 4 章）。
- 同一单位的不同部门共享一个局域网，不利于信息保密与安全隔离。

通过虚拟局域网（Virtual LAN，VLAN），可将一个大型局域网划分为若干与地理位置无关的逻辑上的 VLAN，**每个 VLAN 构成一个独立且较小的广播域**。同一 VLAN 内的主机可以直接通信，而不同 VLAN 之间的主机则无法直接通信。

目前主要有以下三种 VLAN 划分方式：

1）**基于端口**（接口）。将交换机的若干物理接口划归同一个逻辑组。该方法最简单、最常用。若主机更换了接入端口，则可能被划分到新的 VLAN（新子网）。

2）**基于 MAC 地址**。根据主机的 MAC 地址划分 VLAN。当主机从一个交换机移动到另一个交换机时，它仍属于原 VLAN。

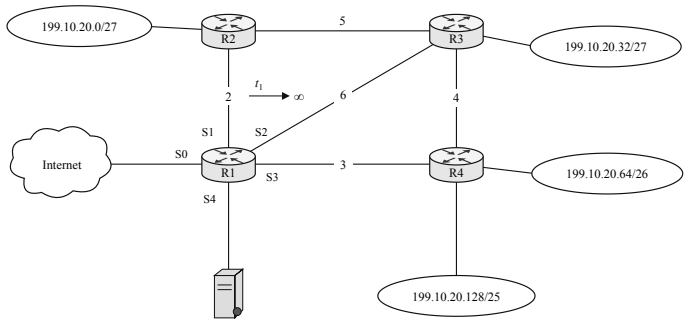
3）**基于 IP 地址**。根据网络层地址或协议类型划分 VLAN。此类 VLAN 可跨越路由器进行扩展，将多个物理局域网中的主机纳入同一逻辑 VLAN。

对应原题本第37页；点击此行返回题目。

2026–37

计算机网络 · 单项选择题 · 2 分

某网络在 t_0 时刻的网络拓扑以及 R1 的路由表如下图所示，R1～R4 为路由器，基于链路状态路由算法计算路由，S0～S4 为 R1 的接口，链路上的数值为链路费用。若 t_1（t_1>t_0）时刻 R1 检测到连接 R1 和 R2 的链路断开，则 R1 经过重新计算路由并进行充分路由聚合后的路由表中路由项（入口）的数量是



R1 路由表

目的网络	接口
199.10.20.0/27	S1
199.10.20.32/27	S2
199.10.20.64/26	S3
199.10.20.128/25	S3
199.10.21.125/32	S4
0.0.0.0/0	S0

A. 3

B. 4

C. 5

D. 6

【对应知识】王道2027计算机网络 · 原PDF第155页
王道知识：同下一跳路由聚合与最长前缀匹配

5．路由聚合

一个大的 CIDR 地址块中包含多个较小的地址块，因此在路由表中可以用这个较大的 CIDR 地址块来代替多个较小的地址块。这种方法称为路由聚合，它使得路由表中的一个项目能够代表原来基于传统分类地址的多条路由，从而压缩路由表规模，提升网络性能。

考点追踪 ▶▶ 路由聚合的原理（2009、2011、2013、2014、2018）

例如，在如图 4.8 所示的网络中，若不使用路由聚合，则 R1 的路由表中需要分别为网络 1 和网络 2 配置两条独立的路由项。观察可知，这两个网络的地址前缀在二进制表示下的前 16 位完全相同，第 17 位分别为 0 和 1，且 R1 到这两个网络的下一跳均为 R2。在此情况下，R1 可将这两个网络聚合成一个更大的地址块 206.1.0.0/16，从而用**一条聚合路由**替代原有的两条路由。

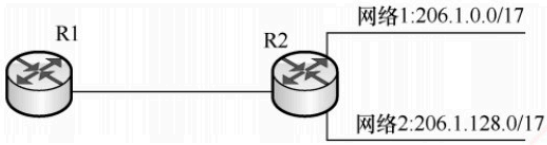


图 4.8　路由聚合的例子

考点追踪 ▶▶ 最长前缀匹配机制（2013、2015）

最长前缀匹配（也称最佳匹配）：使用 CIDR 时，路由表项由“网络前缀”和“下一跳地址”组成。当查找路由表时，目的地址可能匹配多个路由项。此时，应选择**网络前缀最长**（前缀位数最多）的那条路由，因为前缀越长，对应的地址块越小，路由信息就越具体、越精确。

CIDR 查找路由表的方法：为高效实现最长前缀匹配，路由器通常将 CIDR 路由表组织成层次式数据结构（如 Trie 树），通过自上而下的方式快速定位最优匹配项。

CIDR 的主要优势在于网络前缀长度的灵活性：在**网络外部**，可通过较短前缀进行路由聚合，显著减少全局路由表项；在**网络内部**，可延长前缀以灵活划分子网，满足不同规模的需求。这种**“外部聚合、内部细分”**的机制，兼顾了路由可扩展性与地址分配的精细化控制。

【对应知识】王道2027计算机网络 · 原PDF第199页
王道知识：链路状态变化后以Dijkstra重算最短路

3．链路状态算法

链路状态（Link State，LS）指的是：本路由器与哪些邻居相连，以及对应链路的代价。链路状态算法要求每个节点都掌握完整的**全网拓扑结构图**。为此，每个节点执行以下两项任务：

- 主动探测所有**相邻节点的状态**；
- 定期将自身的链路状态信息，通过洪泛法**传播给全网所有节点**。

因此，每个节点都知道全网拓扑、连接关系及各链路代价，并可基于此使用 Dijkstra 最短路径算法，独立计算到其他所有节点的最短路径；每当收到链路状态报文时，节点会更新本地的拓扑视图，并在链路状态发生变化时，立即重新运行 Dijkstra 算法以更新最短路径。

2026-38

计算机网络 · 单项选择题 · 2 分

下列路由协议中，可以将一个自治系统划分为多个区域（Area）的内部网关协议是

I. OSPF

II. RIP

III. BGP

A. 仅 I

B. 仅 II

C. 仅 I、III

D. 仅 II、III

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】王道2027计算机网络 · 原PDF第204页
王道知识：OSPF将自治系统划分区域

为支持大规模网络，OSPF 将一个自治系统划分为若干更小的**区域**（Area）。划分区域的好处是：将洪泛法交换链路状态信息的范围限制在各区域内，而非整个 AS，从而显著减少网络通信开销。每个区域由一个或多个**区域边界路由器**负责为进出该区域的分组提供路由。AS 内必须有一个区域配置成**主干区域**，它包含 AS 内的所有区域边界路由器（可能还包含部分非边界路由器），用于连通其他区域。当分组需在不同区域之间传送时，其转发路径为：**源区域→本地区域边界路由器→主干区域→目的区域边界路由器→目的地**。在图 4.19 中，R3、R4 和 R7 均为区域边界路由器，每个区域至少包含一个区域边界路由器。此外，主干区域中还需指定一个**自治系统边界路由器**（如 R6），专门负责与本 AS 外的其他 AS 交换路由信息。

【对应知识】王道2027计算机网络 · 原PDF第204页（续）
王道知识：OSPF将自治系统划分区域

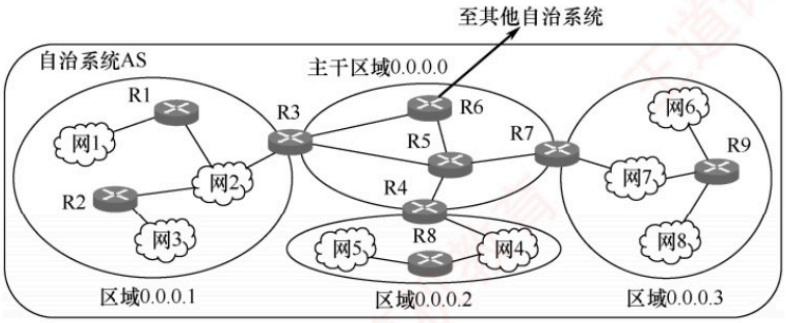


图 4.19　OSPF 划分区域举例

对应原题本第39页；点击此行返回题目。

2026–39

计算机网络 · 单项选择题 · 2 分

若将 IP 网络 123.4.4.0/22 划分为规模均等的 32 个子网，则 IP 地址 123.4.5.211 所在的子网是

A. 123.4.4.192/27

B. 123.4.4.192/28

C. 123.4.5.192/27

D. 123.4.5.192/28

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】王道2027计算机网络 · 原PDF第154页
王道知识：CIDR前缀与子网地址按位与运算

CIDR 采用如下结构表示 IP 地址：

IP 地址:: = {<网络前缀>，<主机号>}。

考点追踪 ▶ CIDR 地址块的分析（2011、2015、2016、2019、2023）

CIDR 还使用斜线记法（也称 CIDR 表示法），格式为“IP 地址/前缀长度”，其中**前缀长度**表示网络前缀所占的位数，它等效于子网掩码中连续 1 的数量。例如，128.14.32.5/20 表示其子网掩码由 20 个连续的 1 和 12 个连续的 0 组成（255.255.240.0）。将该 IP 地址与掩码进行逐位“与”运算，即可得到其**网络前缀**（**子网地址**），它等价于直接截取 IP 地址的前 20 位。

逐位“与”
IP 地址 = 10000000.00001110.00100000.00000101
掩码 = 11111111.11111111.11110000.00000000
网络前缀 = 10000000.00001110.00100000.00000000（128.14.32.0）

【对应知识】王道2027计算机网络 · 原PDF第153页
王道知识：从主机号借位划分子网

划分子网的基本思想如下。

1）对外透明：单位对外仍表现为单一网络，外部路由器无须感知其内部子网结构。

2）内部灵活划分：从原主机号中借用若干位作为子网号，主机号位数相应减少。

3）分层转发：外部路由器根据**网络号**转发数据报至该单位边界路由器；边界路由器再结合**子网号**，将数据报交付至具体子网，最终送到目的主机。

例如，将一个 C 类网络 208.115.21.0 划分为 4 个子网，需借用 2 位作为子网号（因为 $2^2=4$），主机号剩余 6 位，每个子网中的最大主机数为 $2^6-2=62$（扣除网络地址和广播地址），各子网的网络地址为：208.115.21.0、208.115.21.64、208.115.21.128、208.115.21.192。

2026-40

计算机网络 · 单项选择题 · 2 分

电子商务类 Web 应用通常采用 Cookie 技术解决 HTTP 协议无状态的问题。下列叙述中，不属于 Cookie 技术典型用途的是

A. 用户跟踪

B. 个性化推荐

C. 构建虚拟购物车

D. 缩短 Web 对象响应时间

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】王道2027计算机网络 · 原PDF第303页
王道知识：Cookie保持用户状态、身份识别与行为跟踪

考点追踪 ▶▶ Cookie 的工作原理（2015）

HTTP 的无状态特性简化了服务器设计，使其更易于支持大量并发请求。在实际应用中，通常结合 **Cookie 与数据库**来实现用户行为跟踪（如记录用户最近浏览的商品等）。

Cookie 的工作原理：当用户首次访问某个启用 Cookie 的网站时，服务器会为其生成唯一的 Cookie 识别码，如“12345”，并以此为索引在后端数据库中创建一个记录，用于存储该用户的访问信息。随后，服务器在 HTTP 响应报文中添加一个 Set-cookie 首部行：“Set-cookie: 12345”。用户代理（浏览器）收到响应后，会将 Cookie 识别码连同服务器域名一起保存在其本地的 Cookie 文件中。当用户再次访问该网站时，浏览器会在 HTTP 请求报文中自动附加一个 Cookie 首部行：“Cookie: 12345”。服务器根据 Cookie 识别码即可从数据库中检索出该用户的活动记录，从而提供一些个性化服务，例如基于历史浏览记录向用户推荐新商品等。

2026-41

数据结构　综合应用题 · 13 分

已知二叉搜索树采用二叉链表存储，类型定义如下：

```
typedef struct BSTNode                  //二叉搜索树结点
{   int data;                           //保存结点的关键字值
    struct BSTNode * left, * right;     //指向左孩子、右孩子的指针
} BinTNode;
typedef BinTNode * BTree;               //二叉搜索树
```

对于给定的非空二叉搜索树 T（root 为指向 T 的根结点的指针）和整数 k，请设计尽可能高效的查找算法：

```
void searchX( BTree root, int k )
```

查找 T 中其关键字值与 k 之差的绝对值最小的所有结点，输出差值的绝对值和结点中的关键字值。要求：

（1）给出算法的基本设计思想。（4 分）

（2）根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。（9 分）

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】7.3.1 二叉排序树 · 原PDF第3页
原课件 7.3.1 二叉排序树 · PDF p3

二叉排序树的定义

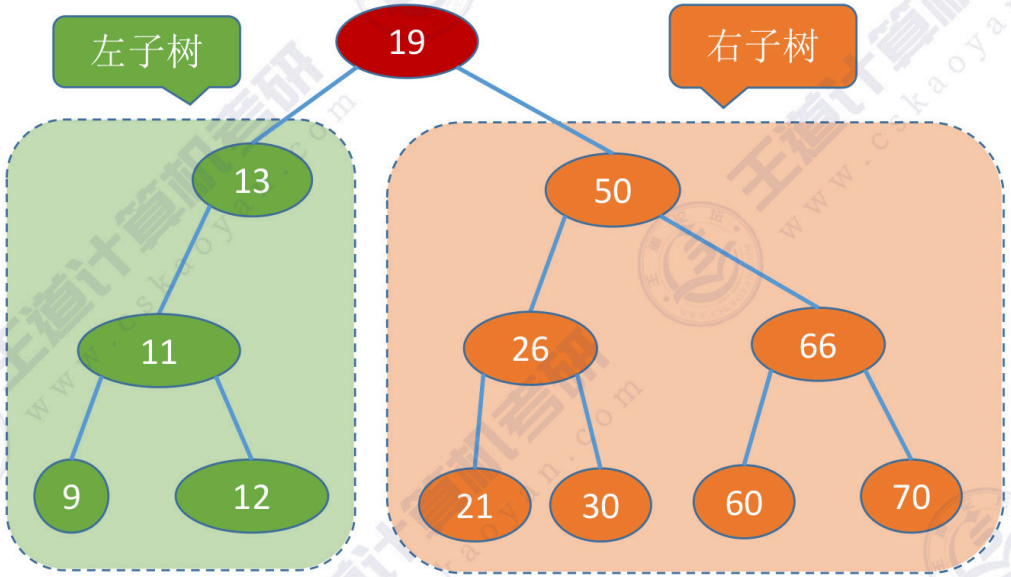
二叉排序树可用于元素的有序组织、搜索

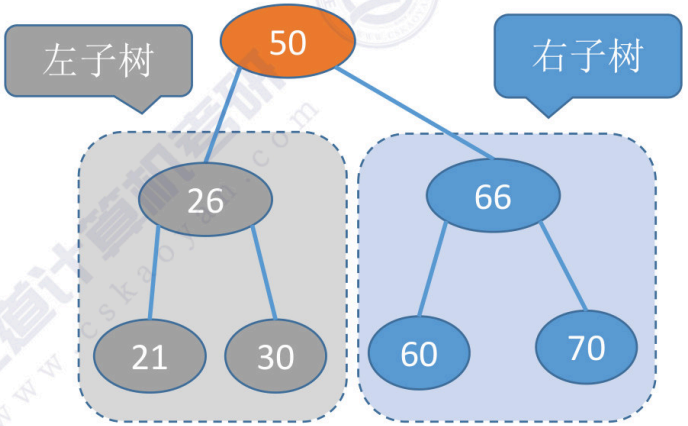
二叉排序树，又称二叉查找树（BST，Binary Search Tree）
一棵二叉树或者是空二叉树，或者是具有如下性质的二叉树：
左子树上所有结点的关键字均小于根结点的关键字；
右子树上所有结点的关键字均大于根结点的关键字。
左子树和右子树又各是一棵二叉排序树。

→ 左子树结点值 < 根结点值 < 右子树结点值

↓

进行中序遍历，可以得到一个递增的有序序列



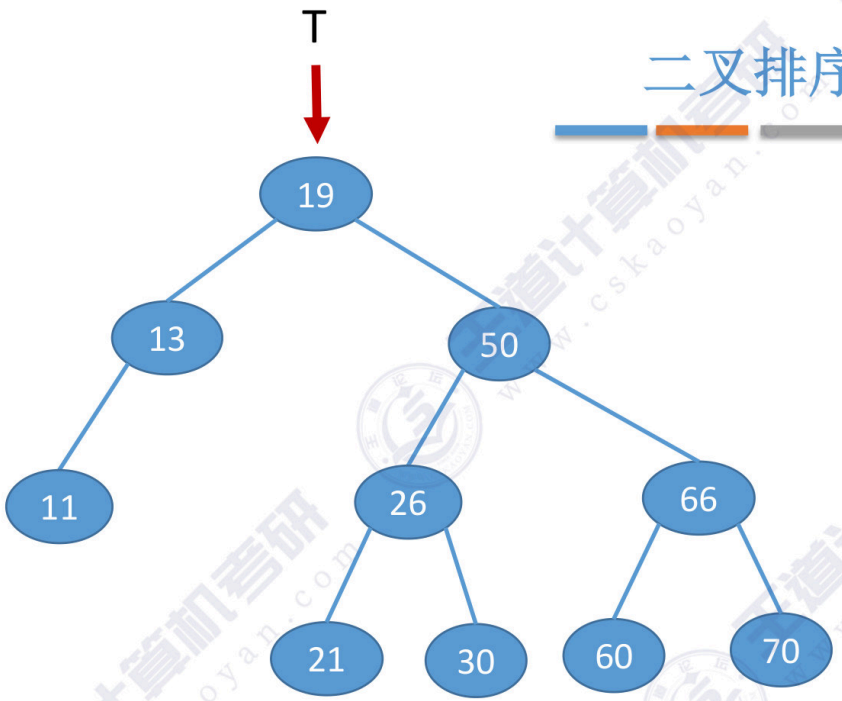


王道考研/CSKAOYAN.COM

对应原题本第42页；点击此行返回题目。

【对应知识】7.3.1 二叉排序树 · 原PDF第4页
原课件 7.3.1 二叉排序树 · PDF p4

二叉排序树的查找



左子树结点值 < 根结点值 < 右子树结点值

若树非空，目标值与根结点的值比较：
若相等，则查找成功；
若小于根结点，则在左子树上查找，否则在右子树上查找。

查找成功，返回结点指针；查找失败返回NULL

例1：查找关键字为30的结点

```
//二叉排序树结点
typedef struct BSTNode{
    int key;
    struct BSTNode  *lchild,*rchild;
}BSTNode,*BSTree;
```

```
//在二叉排序树中查找值为 key 的结点
BSTNode *BST_Search(BSTree T,int key){
    while(T!=NULL&&key!=T->key){     //若树空或等于根结点值，则结束循环
        if(key<T->key) T=T->lchild;      //小于，则在左子树上查找
        else  T=T->rchild;               //大于，则在右子树上查找
    }
    return T;
}
```

王道考研/CSKAOYAN.COM

对应原题本第42页；点击此行返回题目。

2026-42

数据结构　综合应用题 · 10 分

一个栈的入栈序列是 $1,2,\ldots,n$（$n\geq 3$）。请回答下列问题。

（1）当 $n=9$ 时，是否能得到出栈序列 $2,3,1,6,4,7,5,9,8$？是否能得到出栈序列 $2,3,1,5,6,7,4,9,8$？（2 分）

（2）对于由 $1,2,\ldots,n$ 组成的任意序列 $P_1,P_2,\ldots,P_n$ 中的元素 P_i、P_j、P_k（$i<j<k$），当 P_i、P_j、P_k 的大小满足什么关系时，序列 $P_1,P_2,\ldots,P_n$ 不是可以得到的出栈序列？（2 分）

（3）当 $n=4$ 时，可以得到的以 2 开头的出栈序列的个数是多少？（2 分）

（4）若当 $n=k-1$ 时可以得到的出栈序列个数为 M，则当 $n=k$ 时可以得到的以 1 开头的出栈序列的个数是多少？可以得到的以 2 开头的出栈序列的个数是多少？可以得到的所有出栈序列的个数是多少？（4 分）

王道知识与解析 · 未定位原解析，仅附知识

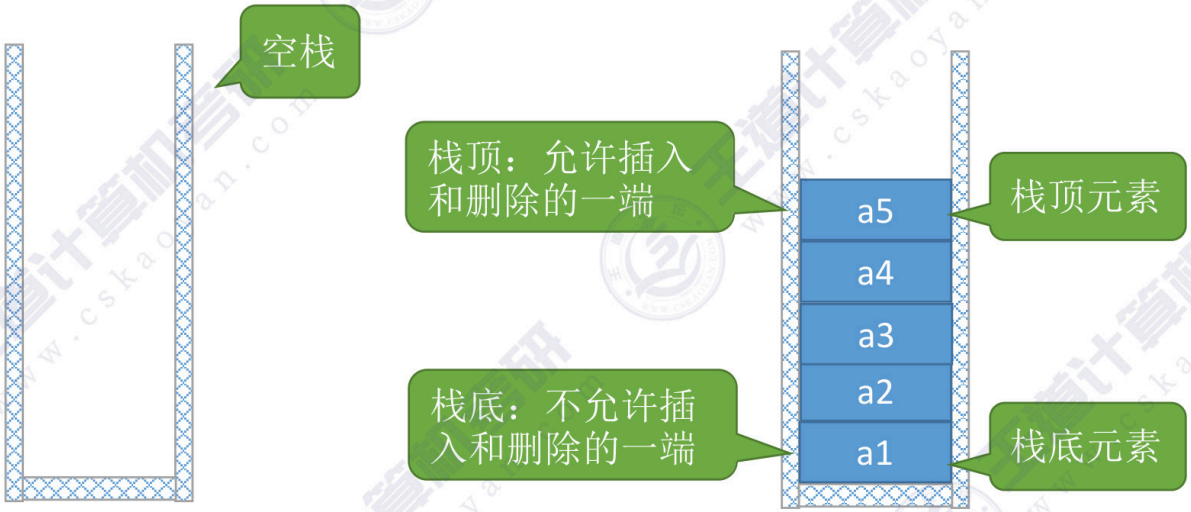
【对应知识】3.1.1_栈的基本概念 · 原PDF第7页
原课件 3.1.1_栈的基本概念 · PDF p7

栈的定义

栈（Stack）是只允许在一端进行插入或删除操作的线性表

逻辑结构：与普通线性表相同
数据的运算：插入、删除操作有区别

重要术语：栈顶、栈底、空栈



进栈顺序：
a1 → a2 → a3 → a4 → a5

出栈顺序：
a5 → a4 → a3 → a2 → a1

特点：后进先出
Last In First Out （LIFO）

王道考研/CSKAOYAN.COM

对应原题本第43页；点击此行返回题目。

【对应知识】3.1.1_栈的基本概念 · 原PDF第10页
原课件 3.1.1_栈的基本概念 · PDF p10

栈的常考题型

进栈顺序：
a → b → c → d → e

有哪些合法的出栈顺序？

n个不同元素进栈，出栈元素不同排列的个数为 $\frac{1}{n+1}C_{2n}^{n}$ 。上述公式称为卡特兰（Catalan）数，可采用数学归纳法证明（不要求掌握）。

$$\frac{1}{5+1}C_{10}^{5} = \frac{10*9*8*7*6}{6*5*4*3*2*1} = 42$$

王道考研/CSKAOYAN.COM

对应原题本第43页；点击此行返回题目。

2026-43

组成原理 · 综合应用题 · 10 分

某 16 位计算机 C 按字节编址，通用寄存器 R0～R15 的编号为 0～15，存储器地址为 16 位，采用定长指令字，指令格式有 R、I 和 M 型，如下所示。

格式	4位	4位	4位	4位	功能说明
R型	0000	rt	rs/num	op1	R[rt]←R[rt] op1 R[rs]，R[rt]←R[rt] op1 num
I型	op2	rt	imm8		R[rt]←R[rt] op2 imm8
M型	op3	offset			R[0]←M[R[15]+offset]，M[R[15]+offset]←R[0]

其中，op1 为 0001、0010 分别表示加、左移指令，op2 为 0100 表示加立即数指令，op3 为 1110、1111 分别表示取数、存数指令，R[r] 表示寄存器 r 中的内容，num 为移位位数，M[addr] 表示存储器地址 addr 中的内容。请回答下列问题。

（1）主存单元和通用寄存器的宽度各是多少位？（2 分）

（2）op1 和 op2 的编码是否可以相同？op2 和 op3 的编码是否可以相同？（2 分）

（3）若 R[2] = ABCDH，R[9] = F001H，则指令 0000 0010 1001 0001 执行后，R2 和 R9 中内容分别是多少（用十六进制表示）？（2 分）

（4）若变量 x、y 均为 16 位带符号整数，在存储器中依次从低地址向高地址连续存放，x 的地址在 R15 中，实现“`y = 16 * x - 5;`”的 4 条指令 I1～I4 如题 43 表所示，写出①～④处的内容。（4 分）

题 43 表

I1:	①	0000	0000	0000
I2:	0000	②		0010
I3:	0100	0000	③	
I4:	1111	④		

【对应知识】王道2027计算机组成原理 · 原PDF第71页
字节、字长和数据宽度

2.3.4 数据的宽度和存储

1．数据的宽度和单位

在计算机中，比特（bit，也称位，符号为 b）是最小的信息单位，表示一个二进制位（0 或 1）；字节（byte，符号为 B）是**基本的存储和寻址单位**，1 字节 = 8 比特。随着信息规模增大，常在 B（字节）或 b（位）前添加前缀以表示更大的容量，如 KB、MB、GB 等。在传统计算机系统中，这些前缀通常按 2 的幂定义，如 $1KB = 2^{10}B = 1024B$。

此外，字（word）也是常用的数据组织单位。它是由体系结构定义的逻辑单位，通常用于表示整数、地址等基本数据类型的宽度，其长度因架构而异，常见的有 2、4 或 8 字节。

与字不同，字长（也称机器字长）指 CPU 内部整数运算的数据通路的宽度，通常等于通用寄存器的宽度。字长反映计算机一次能处理的整数数据的位数，是衡量机器性能的重要指标。日常所说的“32 位机”或“64 位机”，其中的 32 或 64 即指字长。例如，在 Intel x86 架构中，自 80386 起字长为 32 位（32 位机），但其体系结构仍将 16 位定义为一个字，32 位称为双字。这表明：字是架构层面的约定，而字长体现的是硬件的实际处理能力。

【对应知识】王道2027计算机组成原理 · 原PDF第161页
指令格式和字段编码

4.1.2 指令的基本格式

一条指令是机器语言的一个语句，由一组有意义的二进制代码组成。其基本格式如下：

操作码字段	地址码字段

其中，操作码指明指令应执行的操作及功能，是识别指令、理解其作用以及确定地址码含义的关键，例如指示算术加法或减法、程序转移或返回等。地址码给出被操作信息（指令或数据）的地址，包括源操作数地址、目的操作数地址、转移目标地址或子程序入口地址等。

指令字长是指一条指令所包含的二进制位数，取决于操作码长度、地址码长度及地址码个数。在一个指令系统中，若所有指令字长相等，则称为定长指令字结构。定长指令取指和译码简单，有利于流水线实现。若指令字长随功能而异，则称为变长指令字结构。由于主存通常按字节编址，指令字长通常为字节的整数倍。

根据指令中操作数地址码数目的不同，可将指令分为以下几种格式。

【对应知识】王道2027计算机组成原理 · 原PDF第162页
扩展操作码不能互为前缀

考点追踪 ▶▶ **扩展操作码的设计与分析（2017、2021、2022）**

为在有限的指令字长内支持丰富的指令种类，常采用可变长度操作码，即指令的操作码字段位数不固定，且分散在指令的不同位置。这增加了指令译码和分析的难度，使控制器设计更加复杂。最常见的变长操作码方法是扩展操作码（见图 4.1），它使**操作码长度随地址码减少而增加**，不同地址数的指令有不同长度的操作码，从而有效缩短指令字长。

操作码的位数随地址数的减少而增加

OP	A_1	A_2	A_3

指令字长为 16 位，其中 4 位为基本操作码

【对应知识】王道2027计算机组成原理 · 原PDF第162页（续）
扩展操作码不能互为前缀

图 4.1　一种扩展操作码的安排方式

字段 OP，另有 3 个 4 位长的地址字段 A_1、A_2 和 A_3。若 4 位基本操作码全部用于三地址指令，则最多支持 16 条指令。实际使用了 15 条三地址指令，保留 1111 作为扩展操作码；对于二地址指令，保留 1111 1111 作为扩展操作码，共 15 条；对于一地址指令，保留 1111 1111 1111 作为扩展操作码，共 15 条；零地址指令共 16 条。

除这种安排外，还有其他多种扩展方法，例如形成 15 条三地址指令、12 条二地址指令、63 条一地址指令和 16 条零地址指令，共 106 条指令，请读者自行分析。

在设计扩展操作码指令格式时，必须注意以下两点：

1）**不允许短操作码成为长操作码的前缀**，即短操作码不能与长操作码的前面部分相同。

2）各指令的操作码一定不能重复。

通常情况下，对使用频率较高的指令分配较短的操作码，对使用频率较低的指令分配较长的操作码，从而尽可能减少指令译码和分析的时间。

【对应知识】王道2027计算机组成原理 · 原PDF第47页
移位运算与乘2关系

2.2.2　定点数的移位运算

当计算机中没有乘/除运算电路时，可以通过加法和移位相结合的方法来实现乘/除运算。对于任意二进制整数，左移一位，若未发生溢出，相当于乘以 2（类似于十进制数左移一位相当于乘以 10）；右移一位，若忽略因移出而舍去的末位尾数，相当于除以 2。

根据操作数的类型不同，移位运算可以分为**逻辑移位**和**算术移位**。

1．逻辑移位

考点追踪 ▶▶ 逻辑移位运算（2018）

逻辑移位将操作数视为无符号整数。逻辑移位的规则：**左移时**，高位移出，低位补 0。若高位的 1 移出，则发生溢出。**右移时**，低位移出，高位补 0。

例如，4 位无符号数 0001（+1）左移一位变为 0010（+2），相当于乘以 2，未溢出；0001（+1）右移一位变为 0000（0），相当于除以 2 并舍弃小数部分。又如，1000（+8）左移一位变为 0000（0），相当于乘以 2，但结果超出了 4 位无符号数的表示范围，发生溢出。

2．算术移位

考点追踪 ▶▶ 算术移位运算（2012、2017、2018）

算术移位需要考虑符号位的问题，即将操作数视为有符号整数。有符号整数采用补码表示，因此对于有符号整数的移位操作应采用补码算术移位方式。算术移位的规则：**左移时**，高位移出，低位补 0。若移出的高位与原符号位不同（左移后符号位改变），则发生溢出。**右移时**，低位移出，高位补符号位。若低位的 1 移出，则影响精度。

例如，4 位补码 0010（+2）左移一位变为 0100（+4），未溢出；1001（−7）左移一位变为 0010，符号由负变正，表明发生溢出（因为−14 超出了 4 位补码的表示范围）。又如，1001（−7）

【对应知识】王道2027计算机组成原理 · 原PDF第47页（续）
移位运算与乘2关系

右移一位变为1100（−4），保留了符号位，但丢失了最低有效位，影响精度。

【对应知识】王道2027计算机组成原理 · 原PDF第47页
补码加减运算

2.2.3 定点数的加减运算

1．补码加减运算

考点追踪 ▶▶ 补码的加减运算（2009、2011、2017、2025）

补码加减运算规则简单，易于硬件实现。补码加减运算的公式如下（设字长为 $n+1$）。

$$[A+B]_{补}=[A]_{补}+[B]_{补}\ (\mathrm{mod}\ 2^{n+1})$$

$$[A-B]_{补}=[A]_{补}+[-B]_{补}\ (\mathrm{mod}\ 2^{n+1})$$

补码运算具有以下特点：

1）按二进制加法规则运算，逢二进一。

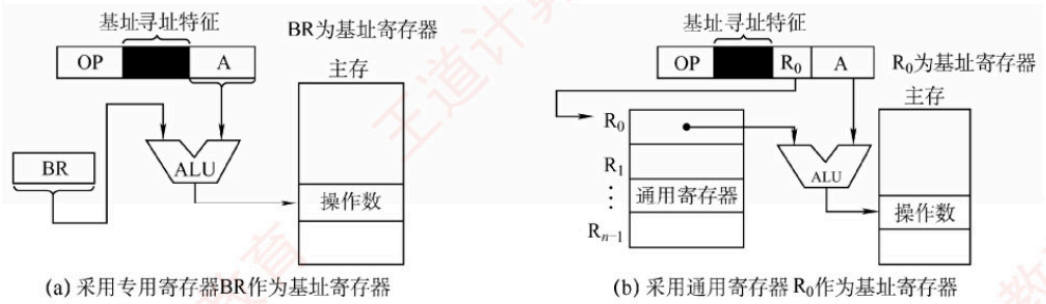
2）若做加法，则两个数的补码直接相加；若做减法，则将被减数与减数的负数补码相加。

【对应知识】王道2027计算机组成原理 · 原PDF第171页
基址寻址与偏移量

8．基址寻址

考点追踪 ▶▶ 基址寻址的 EA 的计算（2019）

基址寻址是指将基址寄存器（BR）的内容与指令中的形式地址 A 相加，形成操作数的有效地址，即 EA = (BR) + A，如图 4.9 所示。BR 可以是专用基址寄存器或指定的通用寄存器。



(a) 采用专用寄存器BR作为基址寄存器　　(b) 采用通用寄存器 R_0 作为基址寄存器

图 4.9　基址寻址

在多道程序环境下，**基址寄存器的内容由操作系统设定**，在程序执行期间保持不变（作为基地址），而形式地址 A 作为偏移量，由用户程序指定并可根据需要变化。当使用通用寄存器作为基址寄存器时，尽管用户可选择哪个寄存器扮演此角色，但其内容仍由操作系统控制。

对应原题本第44页；点击此行返回题目。

2026-44

组成原理　综合应用题 · 13 分

补充条件（来源：2026 年第 43 题）

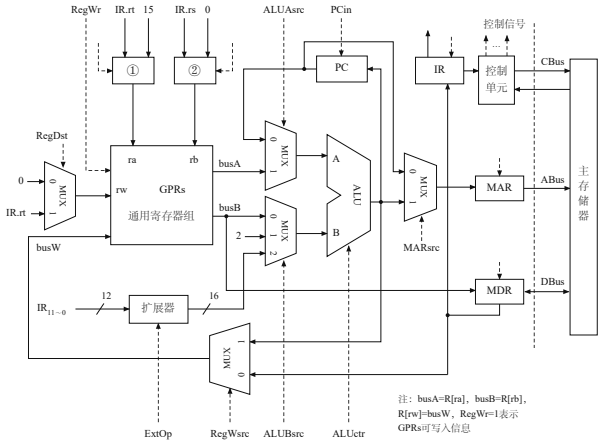
某 16 位计算机 C 按字节编址，通用寄存器 R0～R15 的编号为 0～15，存储器地址为 16 位，采用定长指令字，指令格式有 R、I 和 M 型，如下所示。

格式	4位	4位	4位	4位	功能说明
R型	0000	rt	rs/num	op1	R[rt]←R[rt] op1 R[rs]，R[rt]←R[rt] op1 num
I型	op2	rt	imm8		R[rt]←R[rt] op2 imm8
M型	op3	offset			R[0]←M[R[15]+offset]，M[R[15]+offset]←R[0]

其中，op1 为 0001、0010 分别表示加、左移指令，op2 为 0100 表示加立即数指令，op3 为 1110、1111 分别表示取数、存数指令，R[r] 表示寄存器 r 中的内容，num 为移位位数，M[addr] 表示存储器地址 addr 中的内容。

本题题干

假定题 43 中计算机 C 的部分数据通路如题 44 图所示。



图中带箭头虚线表示控制信号，IR.rt、IR.rs 分别表示 IR 中的 rt、rs 字段，$IR_{11\sim0}$ 为 IR 中低 12 位。要求取指令周期内完成 PC 增量操作。请回答下列问题。

（1）题 44 图中①和②是同一类部件，其名称是什么？（1 分）

（2）已知 I 型指令中 imm8 可以是带符号整数或无符号整数，M 型指令中 offset 是带符号整数，控制信号 ExtOp 至少有几位？为什么？（2 分）

（3）取指周期中，MARsrc、ALUAsrc、ALUBsrc、RegWr 的取值各是什么？（4 分）

（4）左移指令执行周期中，ALUBsrc、RegWsrc、RegDst、RegWr 的取值各是什么？ExtOp 是否可以与 M 型指令的 ExtOp 取值相同？为什么？（6 分）

【对应知识】王道2027计算机组成原理 · 原PDF第218页
数据通路的组合逻辑元件

5.3.1 数据通路的功能

随着技术的发展，越来越多的功能逻辑被集成到 CPU 芯片中，但不论 CPU 的内部结构多么复杂，它都可视为由数据通路（Data Path）和控制部件（Control Unit）两大部分组成。

数据在指令执行过程中所经过的路径，以及路径上涉及的硬件部件，称为数据通路。ALU、通用寄存器、状态寄存器等，都是指令执行时数据流经的部件，属于数据通路的一部分。数据通路描述了数据从何处开始，中间经过哪些部件，最终被传送到哪里。整个数据通路由控制部件控制，后者根据每条指令的功能，生成相应的控制信号，以驱动数据通路完成指定操作。

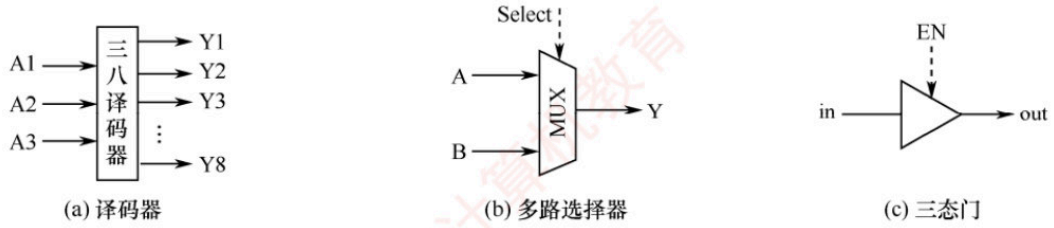
5.3.2 数据通路的组成

数据通路的基本构成元件可分为**组合逻辑元件**和**时序逻辑元件**两大类。

考点追踪 ▶▶ 数据通路的组成部件（2017、2021、2025）

1．组合逻辑元件（操作元件）

组合逻辑元件主要用于执行数据运算与路径选择，也称操作元件。其特征是：任意时刻的输出**仅由当前输入决定**，不依赖历史状态；内部不含记忆单元，不受时钟控制，且无输出到输入的反馈通路，因而具有确定性和即时响应性。数据通路中常用的组合逻辑元件包括加法器、算术逻辑单元（ALU）、译码器、多路选择器（MUX）和三态门等，如图 5.4 所示。



(a) 译码器　(b) 多路选择器　(c) 三态门

图 5.4　数据通路中的几种常用组合逻辑元件

【对应知识】王道2027计算机组成原理 · 原PDF第38页
零扩展与符号扩展

考点追踪 ▶▶ 零扩展和符号扩展的应用（2012、2021、2024）

2）位扩展：当短类型转换为长类型时，系统通过填充高位来保持数值语义不变。具体扩展的方式**取决于源数据的符号性**：

- 零扩展：用于无符号数，在高位补 0。
- 符号扩展：用于补码表示的有符号数，高位重复填充符号位。

对应原题本第45页；点击此行返回题目。

【对应知识】王道2027计算机组成原理 · 原PDF第222页
取指、PC增量和取指数据通路

1. 取指令部件的设计

每条指令的第一步都是完成取指令并计算下一条指令地址的过程。图 5.7 展示了取指令部件的结构示意图。指令存储在指令存储器中，仅支持读操作。读取指令时，只需提供指令地址，经过固定的延迟后即可输出对应的 32 位指令字。指令地址由程序计数器（PC）提供。

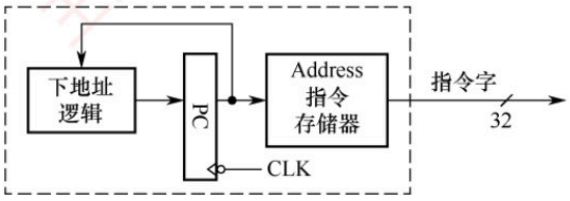


图 5.7　取指令部件的结构示意图

取指令部件包含专用的下地址逻辑电路，用于计算并更新 PC 值，以确定下一条指令的地址。下地址逻辑根据当前指令类型区分不同的执行路径：

- **顺序执行**：当指令为普通顺序指令时，直接计算 PC + 4 得到下一条指令的地址。
- **转移执行**：对于分支或转移指令，需要根据指令类型计算转移目标地址。例如，beq 指令（相等转移）通过比较两个寄存器的内容决定是否转移，并根据偏移量计算新的 PC 值。

【对应知识】王道2027计算机组成原理 · 原PDF第223页
寄存器组端口、写使能和控制位数

2. R 型运算类指令数据通路

R 型指令指的是操作数全部来自通用寄存器、运算结果也写回通用寄存器的算术/逻辑运算指令。以加法指令为例：

```
add rd, rs, rt          #R[rs]+R[rt]→R[rd]
```

图 5.8 展示了 R 型指令相关的数据通路示意图。该数据通路能够对两个寄存器 Rs 和 Rt 的内容进行运算，并将结果写回 Rd 寄存器。例如，add 和 sub 等指令还需判断运算结果是否溢出，仅当不溢出时才将结果写回 Rd 寄存器；发生溢出时，会触发异常处理。

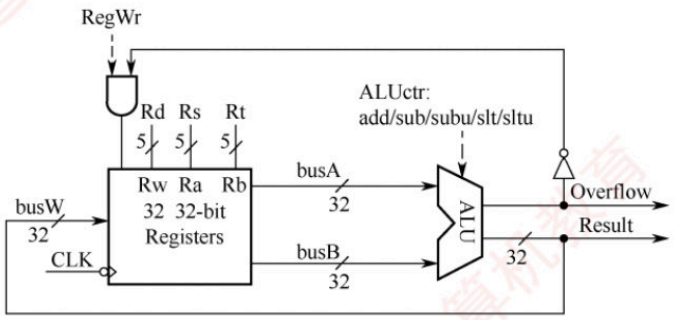


图 5.8　R 型指令相关的数据通路示意图

指令中的 Rs 和 Rt 是两个源操作数寄存器的编号，Rd 是目的寄存器的编号。因此，

- 寄存器堆的两个读地址端 Ra 和 Rb 应分别与 Rs 和 Rt 相连。
- 写地址端 Rw 应与 Rd 相连。
- ALU 的运算结果通过总线 busW 连至寄存器堆的写数据端。

控制信号 RegWr 作为寄存器堆的写使能信号，在 RegWr 为 1 且无溢出的情况下，运算结果才被写入寄存器堆。具体来说，RegWr 信号和溢出标志位 Overflow 通过“与”门组合后，决定是否允许写入寄存器堆。显然，在 R 型指令执行期间，RegWr 信号应始终为 1。

此外，ALU 的操作类型由控制信号 ALUctr 决定。假设 ALU 支持 N 种不同的运算，则 ALUctr 至少需要 $\lceil \log_2 N \rceil$ 位来选择具体的运算类型。

【对应知识】王道2027计算机组成原理 · 原PDF第223页
I型指令扩展器与多路选择器

3．I 型运算指令的数据通路

I 型运算指令（立即数运算类指令）的核心执行逻辑是，先将指令中的 16 位立即数扩展为 32 位，再将寄存器 Rs 的内容送入 ALU 运算，结果写回目的寄存器 Rt。以有符号加法指令为例：

```
addi rt, rs, imm16      #R[rs]+SignExt[imm16]→R[rt]
```

其中，SignExt[imm16]表示对 16 位立即数进行符号扩展。

图 5.9 是在 R 型指令数据通路（见图 5.8）基础上，增加支持 I 型指令的功能模块（如立即数扩展器和多路选择器）后形成的通用数据通路。该通路可同时执行 R 型和 I 型运算指令。

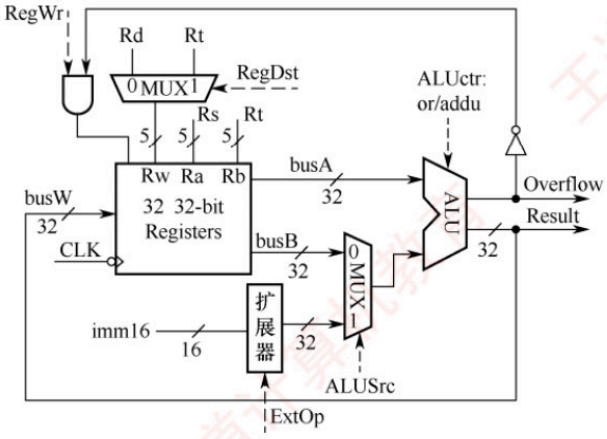


图 5.9　I 型指令相关的数据通路示意图

【对应知识】王道2027计算机组成原理 · 原PDF第224页
RegDst、ExtOp和ALU输入选择

与图 5.8 相比，数据通路主要有以下三处改动，以同时兼容 R 型和 I 型运算指令的执行。

1）**目的寄存器选择**：R 型指令使用目的寄存器 Rd，而 I 型指令使用目的寄存器 Rt。因此，在寄存器堆的写地址端 Rw 处增设多路选择器，由控制信号 RegDst 控制：
- RegDst = 1（执行 R 型指令）时，选择 Rd 为目的寄存器。
- RegDst = 0（执行 I 型指令）时，选择 Rt 为目的寄存器。

2）**立即数扩展**：I 型指令的 16 位立即数需扩展为 32 位才能送入 ALU。数据通路中新增一个立即数扩展器，其输入为指令的 imm16 字段。由控制信号 ExtOp 决定扩展方式：
- 对于算术类指令，采用符号扩展。
- 对于按位逻辑操作指令，采用零扩展。

3）**ALU 第二操作数选择**：R 型指令的第二操作数来自寄存器 Rt，I 型指令则使用扩展后的立即数。因此，在 ALU 的第二输入端增设多路选择器，由控制信号 ALUSrc 控制：
- ALUSrc = 0（执行 R 型指令）时，选择寄存器 Rt 的数据。
- ALUSrc = 1（执行 I 型指令）时，选择扩展后的立即数。

2026–45

操作系统　综合应用题 · 7 分

某系统采用基于抢占式优先权（优先权值越大优先权越高）和时间片轮转的 CPU 调度算法，仅在时钟中断发生时才能抢占 CPU，时钟中断间隔时间为 10 ms。每当进程进入就绪队列时，其时间片大小均设置为 50 ms。进程因时间片用完而再次进入就绪队列时，若其优先权值大于 0，则优先权值减 1。进程因优先权低被抢占 CPU 而再次进入就绪队列时，其优先权值不变。优先权相同时，先进入就绪队列的进程先执行。假设系统中有 4 个进程 P1～P4，各进程的到达时刻、初始优先权值、所需的 CPU 执行时间如题 45 表所示。

题 45 表

进程	到达时刻	初始优先权值	CPU 执行时间
P1	10 ms	3	95 ms
P2	10 ms	4	20 ms
P3	12 ms	2	40 ms
P4	14 ms	5	60 ms

请回答下列问题。

（1）在忽略时钟中断处理及 CPU 调度时间开销的情况下，若将 10 ms 时刻发生的时钟中断记为第一次时钟中断，且从此时开始进行 CPU 调度，则从 10 ms 时刻开始到 4 个进程都运行结束为止，时钟中断和 CPU 调度各发生了多少次？第一次执行 P1、P3、P4 的时刻分别是什么？（5 分）

（2）若时间片由 50 ms 改为 100 ms，则 CPU 调度次数增加、不变还是减少？若时钟中断间隔时间由 10 ms 改为 1 ms，则系统开销增加、不变还是减少？（2 分）

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】2.2.5_2 调度算法：时间片轮转、优先级、多级反馈队列 · 原PDF第11页
王道原课件：2.2.5_2 调度算法：时间片轮转、优先级、多级反馈队列

时间片轮转（RR, Round-Robin）

优先级调度

算法思想	公平地、轮流地为各个进程服务，让每个进程在一定时间间隔内都可以得到响应
算法规则	按照各进程到达就绪队列的顺序，轮流让各个进程执行一个时间片（如 100ms）。若进程未在一个时间片内执行完，则剥夺处理机，将进程重新放到就绪队列队尾重新排队。
用于作业/进程调度	用于进程调度（只有作业放入内存建立了相应的进程后，才能被分配处理机时间片）
是否可抢占？	若进程未能在时间片内运行完，将被强行剥夺处理机使用权，因此时间片轮转调度算法属于抢占式的算法。由时钟装置发出时钟中断来通知CPU时间片已到
优缺点	优点：公平；响应快，适用于分时操作系统； 缺点：由于高频率的进程切换，因此有一定开销；不区分任务的紧急程度。
是否会导致饥饿	不会
补充	时间片太大或太小分别有什么影响？

王道考研/CSKAOYAN.COM

对应原题本第46页；点击此行返回题目。

【对应知识】2.2.5_2 调度算法：时间片轮转、优先级、多级反馈队列 · 原PDF第12页
王道原课件：2.2.5_2 调度算法：时间片轮转、优先级、多级反馈队列

优先级调度算法

算法思想	随着计算机的发展，特别是实时操作系统的出现，越来越多的应用场景需要根据任务的紧急程度来决定处理顺序
算法规则	每个作业/进程有各自的优先级，调度时选择优先级最高的作业/进程
用于作业/进程调度	既可用于作业调度，也可用于进程调度。甚至，还会用于在之后会学习的I/O调度中
是否可抢占？	抢占式、非抢占式都有。做题时的区别在于：非抢占式只需在进程主动放弃处理机时进行调度即可，而抢占式还需在就绪队列变化时，检查是否会发生抢占。
优缺点	
是否会导致饥饿	

王道考研/CSKAOYAN.COM

对应原题本第46页；点击此行返回题目。

【对应知识】2.2.5_2 调度算法：时间片轮转、优先级、多级反馈队列 · 原PDF第15页
王道原课件：2.2.5_2 调度算法：时间片轮转、优先级、多级反馈队列

优先级调度算法

补充：
就绪队列未必只有一个，可以按照不同优先级来组织。另外，也可以把优先级高的进程排在更靠近队头的位置
根据优先级是否可以动态改变，可将优先级分为静态优先级和动态优先级两种。
静态优先级：创建进程时确定，之后一直不变。
动态优先级：创建进程时有一个初始值，之后会根据情况动态地调整优先级。

I/O设备和CPU可以*并行*工作。如果优先让I/O繁忙型进程优先运行的话，则越有可能让I/O设备尽早地投入工作，则资源利用率、系统吞吐量都会得到提升

如何合理地设置各类进程的优先级？

通常：　系统进程优先级 高于 用户进程
　　前台进程优先级 高于 后台进程
　　操作系统更偏好 I/O型进程（或称 I/O繁忙型进程）
注：与I/O型进程相对的是计算型进程（或称 CPU繁忙型进程）

如果采用的是动态优先级，什么时候应该调整？

可以从追求公平、提升资源利用率等角度考虑
如果某进程在就绪队列中等待了很长时间，则可以适当提升其优先级
如果某进程占用处理机运行了很长时间，则可适当降低其优先级
如果发现一个进程频繁地进行I/O操作，则可适当提升其优先级

王道考研/CSKAOYAN.COM

对应原题本第46页；点击此行返回题目。

【对应知识】王道2027操作系统 · 原PDF第27页
王道2027操作系统：时钟管理与时钟中断

考点追踪 ▶▶ 时钟中断服务的内容（2018）

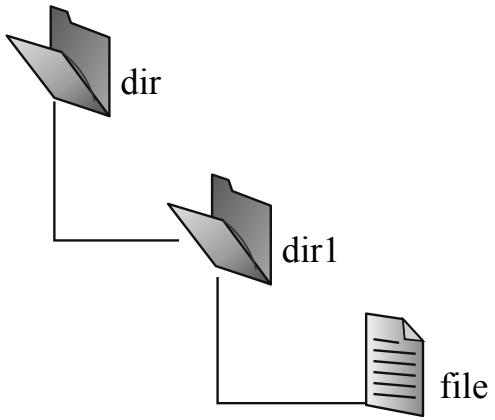
1）时钟管理。时钟是计算机系统的关键部件，其功能包括计时和产生定时中断。操作系统通过时钟管理向用户提供标准系统时间，并利用时钟中断实现任务调度。例如，在分时操作系统中通过时间片轮转进行进程切换，在实时操作系统中依据截止时间控制任务执行，在批处理系统中衡量作业运行进度。因此，**系统的大量活动均依赖于时钟机制**。

对应原题本第46页；点击此行返回题目。

2026–46

操作系统 · 综合应用题 · 8 分

某基于索引节点（inode）的文件系统的目录项由文件名和索引节点号构成。磁盘中存放了索引节点表、位图、目录等元数据以及文件数据，盘块大小为 4 KB，盘块号占 4 B。索引节点表包含了所有文件的索引节点，位于从 100 号盘块开始的 4096 个连续盘块中，索引节点号是索引节点在该表中的序号，从 0 开始编号。每个索引节点占 128 B，包含直接地址项 5 个，一次间接地址项、二次间接地址项、三次间接地址项各 1 个。磁盘块位图和索引节点位图分别用于描述磁盘块和索引节点占用情况，0 表示未占用，1 表示已占用。某子目录树如题 46 图所示，其中各文件的文件名和索引节点号如题 46 表所示，file 大小为 30 KB。



题 46 表

文件名	索引节点号
dir	100
dir1	201
file	1000

请回答下列问题。

（1）file 的索引节点存放在磁盘的几号盘块中？若 file 的索引节点已读入内存，则访问 file 中偏移地址为 21460 的一个字节数据，最多需要访问几个盘块？该文件系统在空间足够大的磁盘中最多能创建多少个文件？（3 分）

（2）若要删除目录 dir1，则文件系统需要对哪些元数据执行哪些操作？（5 分）

【对应知识】4.1_3_文件目录 · 原PDF第15页
王道原课件：4.1_3_文件目录

索引结点（FCB的改进）

文件名	索引结点指针
qianlong	
QMDownLoad	
……	
照片	
……	
对账单4.txt	

索引结点
（包含除了文件名之外的文件描述信息）

思考有何好处？
假设一个FCB是64B，磁盘块的大小为1KB，则每个盘块中只能存放16个FCB。若一个文件目录中共有640个目录项，则共需要占用640/16 = 40 个盘块。因此按照某文件名检索该目录，平均需要查询320 个目录项，平均需要启动磁盘20次（每次磁盘I/O读入一块）。

若使用索引结点机制，文件名占14B，索引结点指针站2B，则每个盘块可存放64个目录项，那么按文件名检索目录平均只需要读入 320/64 = 5 个磁盘块。显然，这将大大提升文件检索速度。

当找到文件名对应的目录项时，才需要将索引结点调入内存，索引结点中记录了文件的各种信息，包括文件在外存中的存放位置，根据“存放位置”即可找到文件。
存放在外存中的索引结点称为“磁盘索引结点”，当索引结点放入内存后称为“内存索引结点”。
相比之下内存索引结点中需要增加一些信息，比如：文件是否被修改、此时有几个进程正在访问该文件等。

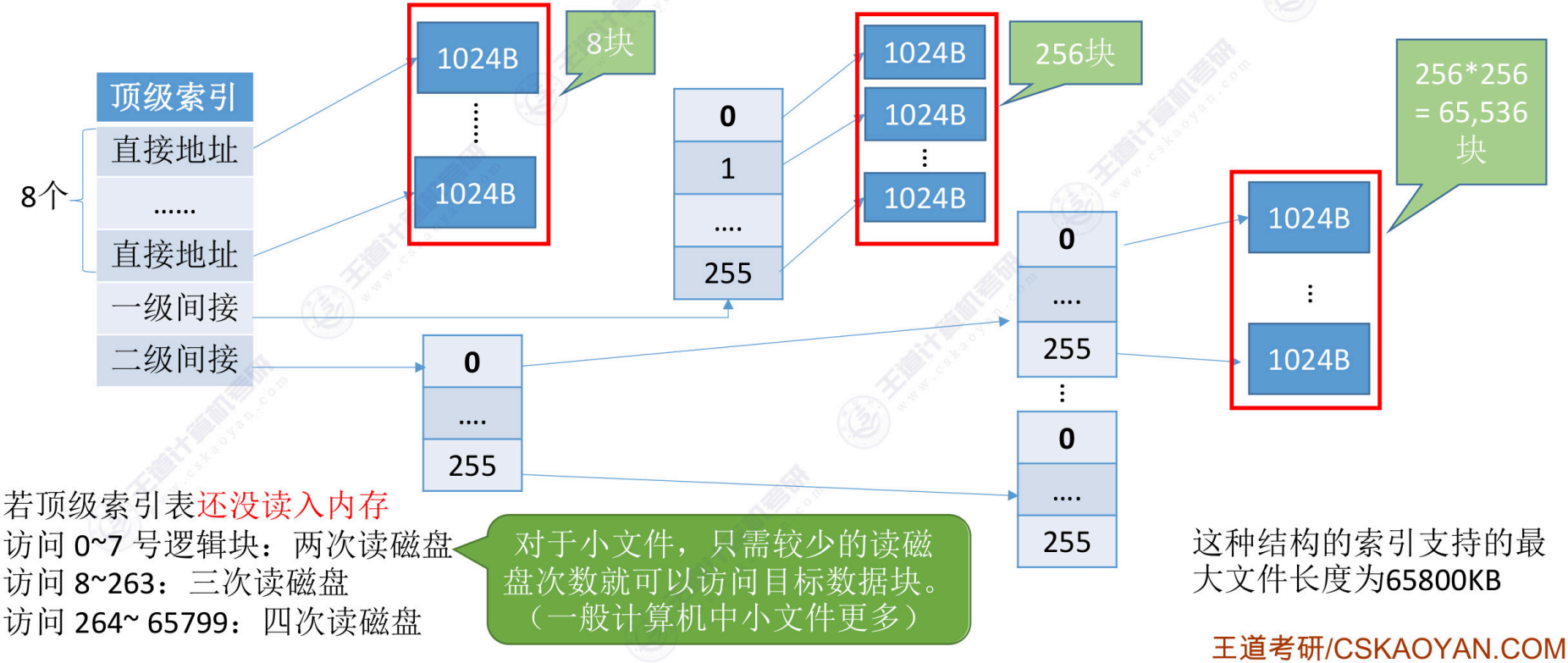
王道考研/CSKAOYAN.COM

对应原题本第47页；点击此行返回题目。

【对应知识】4.1_4_文件的物理结构 · 原PDF第26页
王道原课件：4.1_4_文件的物理结构

文件分配方式——索引分配

③混合索引：多种索引分配方式的结合。例如，一个文件的顶级索引表中，既包含直接地址索引（直接指向数据块），又包含一级间接索引（指向单层索引表）、还包含两级间接索引（指向两层索引表）。



若顶级索引表还没读入内存
访问 0~7 号逻辑块：两次读磁盘
访问 8~263：三次读磁盘
访问 264~ 65799：四次读磁盘

对于小文件，只需较少的读磁盘次数就可以访问目标数据块。（一般计算机中小文件更多）

这种结构的索引支持的最大文件长度为65800KB

王道考研/CSKAOYAN.COM

对应原题本第47页；点击此行返回题目。

【对应知识】4.1_7_文件的基本操作 · 原PDF第5页
王道原课件：4.1_7_文件的基本操作

删除文件

可以“删除文件”（点了“删除”之后，图形化交互进程通过操作系统提供的“删除文件”功能，即 delete 系统调用，将文件数据从外存中删除）

进行 Delete 系统调用时，需要提供的几个主要参数：

1. 文件存放路径（“D:/Demo”）
2. 文件名（“test.txt”）

操作系统在处理 Delete 系统调用时，主要做了几件事：

1. 根据文件存放路径找到相应的目录文件，从目录中找到文件名对应的目录项。
2. 根据该目录项记录的文件在外存的存放位置、文件大小等信息，回收文件占用的磁盘块。（回收磁盘块时，根据空闲表法、空闲链表法、位图法等管理策略的不同，需要做不同的处理）
3. 从目录表中删除文件对应的目录项。

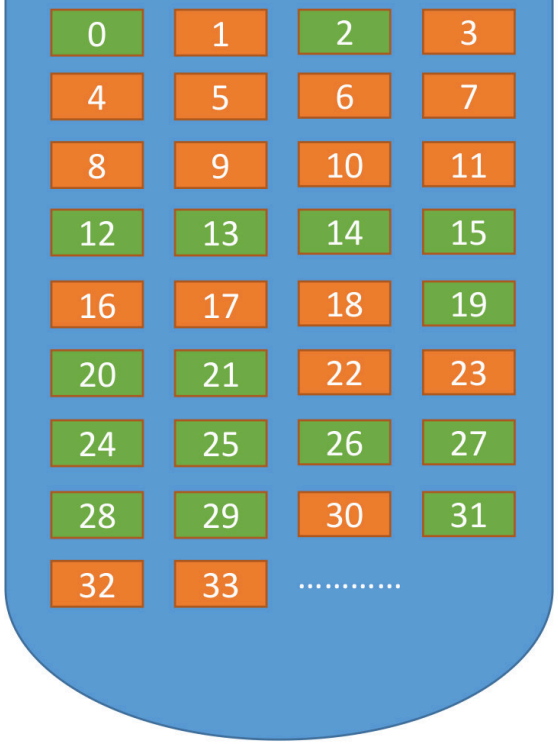
王道考研/CSKAOYAN.COM

对应原题本第47页；点击此行返回题目。

【对应知识】4.1_6_文件存储空间管理 · 原PDF第11页
王道原课件：4.1_6_文件存储空间管理

(0,1) → b=16*0+1=1
(1,10) → b=16*1+10=26

b=13 → i=13/16=0, j=13%16=13
b=31 → i=31/16=1, j=31%16=15



存储空间管理——位示图法

字号 \ 位号	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	1	0	1	1	1	1	1	1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	1	1	0	0	0	0	0	0	1	0
2	1	1	…													
…																

位示图：每个二进制位对应一个盘块。在本例中，“0”代表盘块空闲，“1”代表盘块已分配。位示图一般用连续的“字”来表示，如本例中一个字的字长是16位，字中的每一位对应一个盘块。因此可以用（字号，位号）对应一个盘块号。当然有的题目中也描述为（行号，列号）

重要重要重要：要能自己推出盘块号与（字号,位号）相互转换的公式。

注意题目条件：盘块号、字号、位号到底是从0开始还是从1开始
如本例中盘块号、字号、位号从0开始，若n表示字长，则...

(字号, 位号)=(i, j) 的二进制位对应的 盘块号 b = ni + j

b号盘块对应的字号 i = b/n，位号 j = b%n

注：王道课本上给出的是盘块号、字号、位号都从1开始的算法。

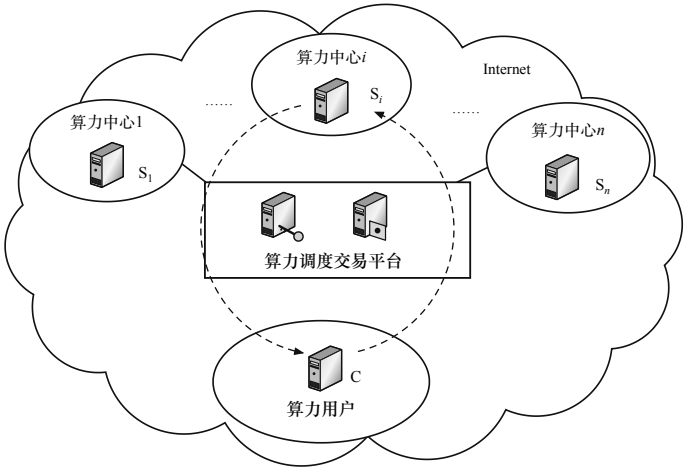
王道考研/CSKAOYAN.COM

对应原题本第47页；点击此行返回题目。

2026–47

计算机网络 · 综合应用题 · 9 分

人工智能（AI）是引领未来的战略性技术，其发展高度依赖强大的算力供给，通过网络互联分散的算力中心，可以充分聚合算力资源，满足人工智能对大规模算力的需求，支撑人工智能国家发展战略。某算力调度交易平台聚合各地的算力中心的算力，向用户提供算力服务的示意场景如题 47 图所示。算力用户 C 向平台申请算力服务，平台通过优化调度各算力中心算力，向 C 授权使用算力中心 i 的算力，S_i 为算力中心 i 的负载均衡服务器，用于接收算力用户的计算任务描述数据文件，并向算力中心 i 内部分派计算任务。为确保可靠传输，假设 C 建立一条 TCP 连接向服务器 S_i 上传一个总长度为 2000 B 的计算任务描述数据文件，拥塞窗口初始阈值为 8 MSS，MSS = 500 B，S_i 对收到的每个 TCP 段进行确认，且确认段不封装数据，接收窗口 rwnd 始终为 1000 B，RTT = 5 ms。C 建立连接时选择的初始序号为 1000，S_i 选择的初始序号为 2000，SYN、ACK、FIN 为标志位，seq 为序号，ack_seq 为确认序号。在整个文件传输过程中未出现任何重传或报文段丢失。



请回答下列问题。

（1）C 与 S_i 建立 TCP 连接过程需要几次握手？C 收到的 SYN = 1、ACK = 1 的 TCP 段的确认序号（ack_seq）是多少？（2 分）

（2）当 C 接收到 S_i 发送的“ACK = 1，seq = 2001，ack_seq = 2001，rwnd = 1000”确认段后，C 的拥塞窗口增加到多少？C 的发送窗口设置为多少？（2 分）

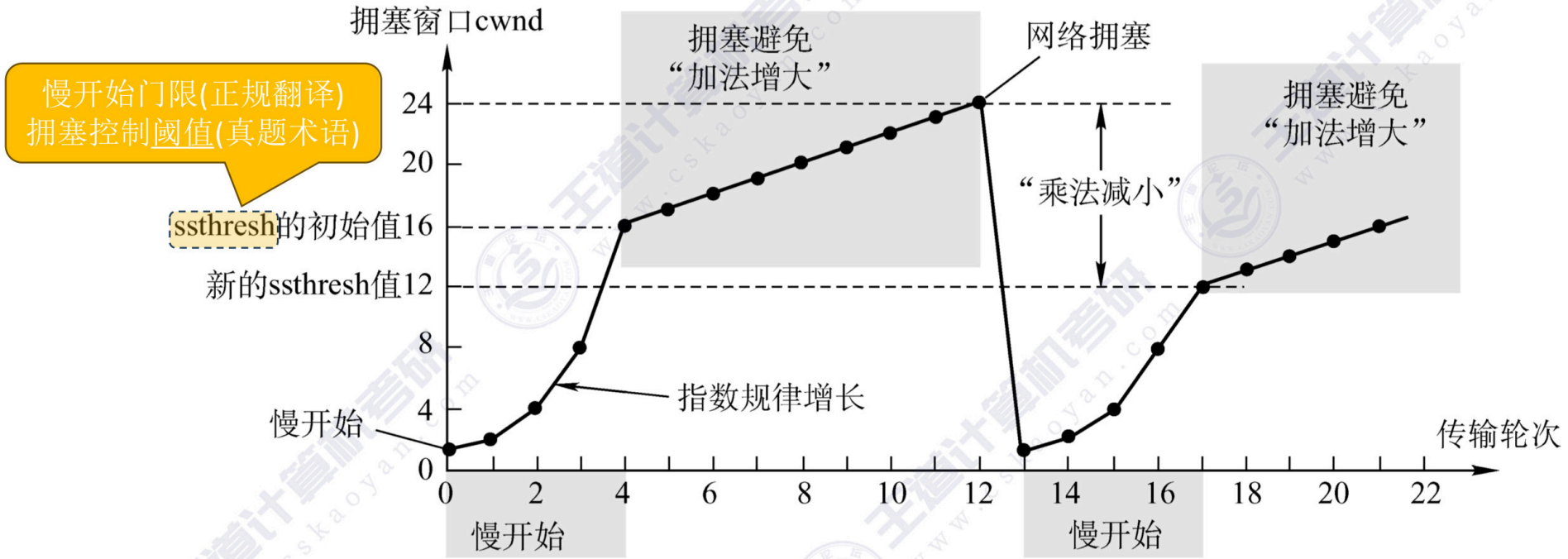
（3）C 与 S_i 释放 TCP 连接过程需要几次挥手？C 收到的最后一个 TCP 报文段的序号（seq）、确认序号（ack_seq）、FIN 的值分别是多少？（4 分）

（4）若忽略报文段传输时延，且时间从 C 请求建立 TCP 连接时刻算起，则 C 确认 S_i 已成功接收到文件的时间是多少？（1 分）

王道知识与解析 · 未定位原解析，仅附知识

【对应知识】5.3.6 TCP拥塞控制 · 原PDF第6页
王道课件：慢开始每收到新ACK增加1 MSS及阈值条件

慢开始、拥塞避免



慢开始算法：cwnd 值从1开始，每收到一个ACK，就让 cwnd+1（当 cwnd<ssthress 时适用）
拥塞避免算法：在一个RTT内，即使收到多个ACK，也只能让 cwnd+1（当 cwnd≥ssthress 时适用）

王道考研/CSKAOYAN.COM

对应原题本第48页；点击此行返回题目。

【对应知识】王道2027计算机网络 · 原PDF第255页
王道知识：发送窗口=min{rwnd,cwnd}

考点追踪 ▶▶ TCP 的滑动窗口机制（2010、2014-2016、2025）

发送窗口的**上限值**应取接收窗口（rwnd）和拥塞窗口（cwnd）中较小的一个，即

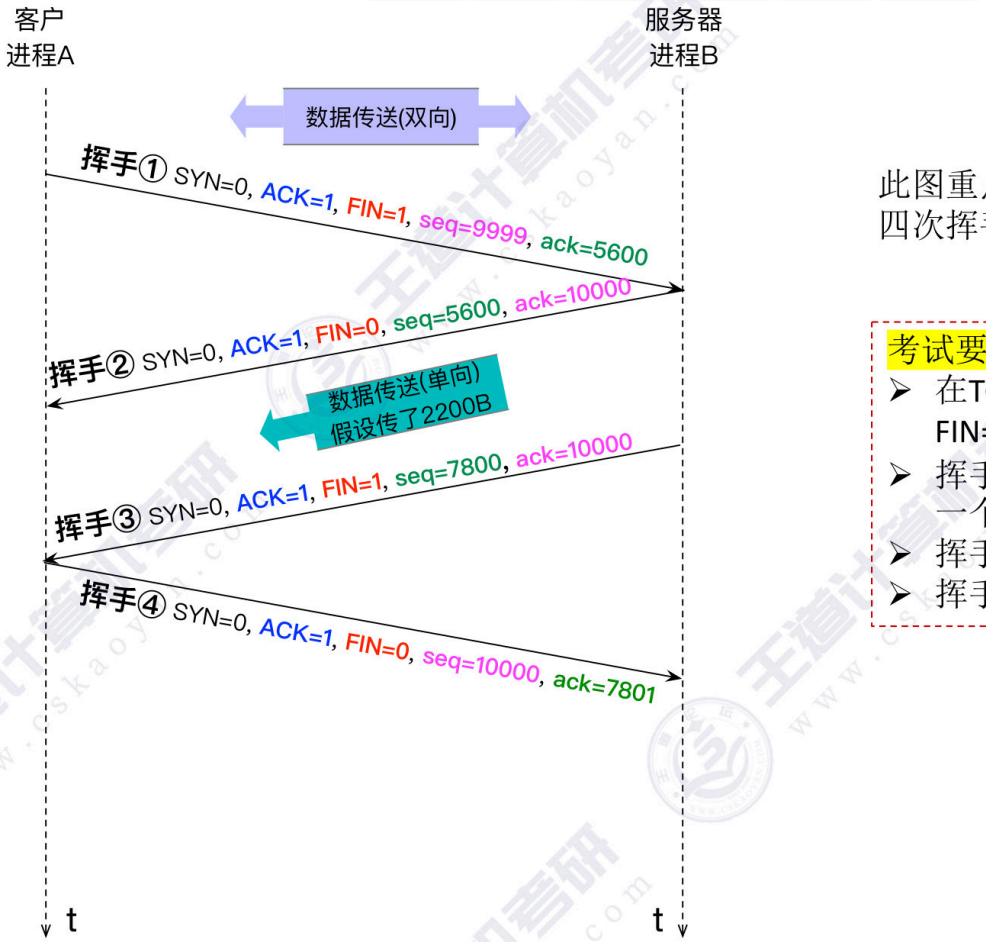
$$\text{发送窗口的上限值} = \min[\text{rwnd}, \text{cwnd}]$$

接收窗口的大小可通过 TCP 首部中的“窗口”字段由接收方通知发送方。那么，发送方如何维护拥塞窗口呢？这正是下面要介绍的慢开始和拥塞避免算法所解决的问题。为便于分析，这里**假设**：数据仅单向传输，接收方只发送确认报文；接收方拥有足够大的缓存空间，因此发送窗口的大小仅受拥塞窗口的限制；并且以最大报文段长度 MSS 作为拥塞窗口大小的单位。

对应原题本第48页；点击此行返回题目。

【对应知识】5.3.3 TCP连接管理 · 原PDF第12页
王道课件：四次挥手字段及FIN消耗序号

释放连接（四次挥手）



此图重点体会：
四次挥手的TCP报文段中，FIN、ACK、seq、ack

考试要点：

- 在TCP全过程中，只有挥手①、挥手③的FIN=1
- 挥手①、挥手③即使不携带数据，也要消耗一个序号
- 挥手②可以携带数据
- 挥手④不可以携带数据

王道考研/CSKAOYAN.COM

对应原题本第48页；点击此行返回题目。

【对应知识】王道2027计算机网络 · 原PDF第251页
王道知识：三次握手序号、第三次可携数据、1RTT后发数据

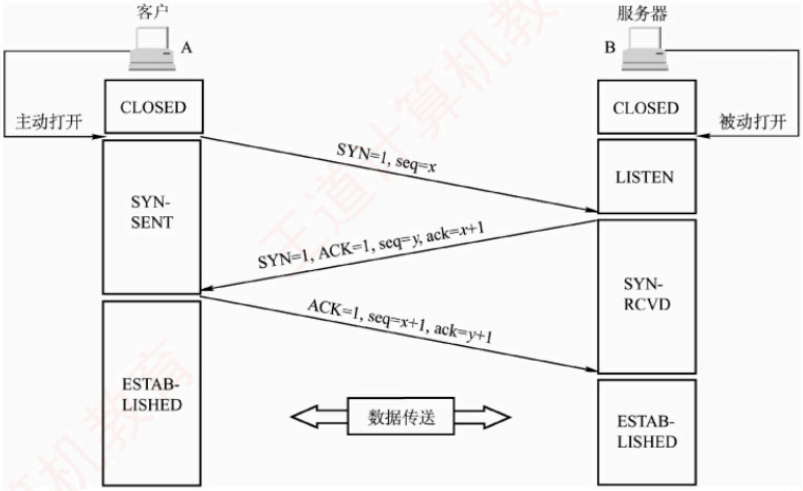


图 5.6　用“三次握手”建立 TCP 连接

考点追踪 ▶▶ TCP 三次握手的字段解析（2011、2012、2016、2019、2023）

第三步：客户收到确认后，再向服务器发送**最终确认**。该确认报文段的 **ACK = 1**，确认序号 **ack** = y + **1**，序号 **seq** = x + **1**。此报文段**可以携带数据**。若不携带数据，则不消耗序号，下一个数据报文段的序号仍为 $x+1$。此时，客户进入 ESTABLISHED（已建立连接）状态。

当服务器收到该最终确认后，也进入 ESTABLISHED 状态，连接正式建立。

注　意

① 仅在前两次握手的报文段中，SYN = 1。TCP 规定：SYN 报文段不能携带数据，但仍消耗一个序号，因此下次发送的报文段序号为其 SYN 报文段序号加 1。

② 仅在第一次握手的报文段中，ACK = 0，因为此时尚未收到任何报文段。

③ 第三次握手的报文段可携带数据；若不携带，则不消耗序号。

从客户发出 SYN 报文段（第一次握手的连接请求报文段）时刻起算，客户最早可在 **1RTT** 后开始发送数据，而服务器最早在 **1.5RTT** 后才能发送数据。

对应原题本第48页；点击此行返回题目。